

Thary: Sistema de apoyo a la decisión basado en predicción de demanda para la gestión de inventarios

Danna Andrea Rodriguez Galarcio

Trabajo de grado presentado para optar al título de ingeniero de sistemas

Asesor:

Ivan Mauricio Cabezas Troyano



Universidad de San Buenaventura
Facultad de Ingeniería (Cali)
Ingeniería de Sistemas
Santiago de Cali, Colombia
2026

Citar/How to cite	Rodriguez Galarcio, D. A. . [1]
Referencia/Reference	[1] Rodriguez Galarcio, D. A. . “Thary: Sistema de apoyo a la decisión basado en predicción de demanda para la gestión de inventarios”, Trabajo de grado profesional , Ingeniería de Sistemas, Universidad de San Buenaventura Santiago de Cali, Colombia, 2026.
Estilo/Style:	
IEEE (2020)	



Biblioteca Digital (Repositorio)
www.bibliotecadigital.usb.edu.co

Bibliotecas Universidad de San Buenaventura

Biblioteca Fray Alberto Montealegre O.F.M. - Bogotá.

Biblioteca Fray Arturo Calle Restrepo O.F.M. - Medellín, Bello, Armenia, Ibagué.

Departamento de Biblioteca - Cali.

Biblioteca Central Fray Antonio de Marchena – Cartagena.

Universidad de San Buenaventura Colombia - www.usb.edu.co

Bogotá - www.usbbog.edu.co

Medellín - www.usbmed.edu.co

Cali - www.usbcali.edu.co

Cartagena - www.usbctg.edu.co

Editorial Bonaventuriana - www.editorialbonaventuriana.usb.edu.co

Revistas científicas – www.revistas.usb.edu.co

Dedicatoria

Me gustaria dedicar este trabajo a mis padres, ya que gracias a ellos y a todo su esfuerzo es que he llegado a este punto de mi carrera. A mis amigos y demas seres queridos que me han brindado su incondicional apoyo, y a mi mascota por su compañía durante el desarrollo de este proyecto.

Agradecimientos

Quiero agradecer a todas las personas que me ayudaron de distintas maneras para lograr completar este trabajo. A mi papá por haber aportado retroalimentacion al proyecto desde su experiencia en la gestion de farmacias, a mi mama por haber escuchado mis ideas y haberme apoyado, a mi director de tesis por la orientación durante el proceso y a todos los profesores que me han enseñado y compartido sus conocimientos durante la carrera.

TABLA DE CONTENIDO

Resumen	14
Abstract	15
Glosario	16
1 Capítulo 1: Introducción	18
1.1 Contexto	18
1.2 Planteamiento del Problema	18
1.3 Justificación del Problema	19
1.3.1 Impactos	19
1.4 Propuesta de Solución	20
1.5 Pregunta de Investigación	20
1.5.1 Preguntas Derivadas	21
1.6 Objetivos	21
1.6.1 Objetivo General	21
1.6.2 Objetivos Específicos	21
1.7 Restricciones	22
1.8 Recursos y Viabilidad Técnica	22
1.8.1 Identificación de Recursos	22
1.8.2 Análisis de Viabilidad	23
2 Capítulo 2: Marco de Referencia	25
2.1 Criterios de Revisión de Literatura	25
2.1.1 Criterios de inclusión	25
2.1.2 Criterios de exclusión	25
2.2 Fundamentos Teóricos	25
2.2.1 Gestión de inventarios	25
2.2.2 Gestión de inventarios con demanda independiente	26
Cantidad Económica de Pedido (EOQ)	26
Punto de reorden y stock de seguridad	26
2.2.3 Clasificación de inventarios	27
Clasificación ABC	27
Coeficiente de variación y clasificación de la demanda	27
2.2.4 Predicción de demanda	28

2.2.5	Series de tiempo	28
2.2.6	Modelos de predicción de demanda	28
	Media Móvil	28
	Redes Neuronales Artificiales (RNA)	29
	Long Short-Term Memory (LSTM) y Gated Recurrent Unit (GRU)	29
2.2.7	Modelos estadísticos y de Machine Learning tradicionales	29
	Regresión lineal	29
	Selección de variables en modelos de regresión	30
	Eliminación Recursiva de Características (RFE)	31
	Significancia estadística (p-valor)	31
	Multicolinealidad y Factor de Inflación de Varianza (VIF)	31
2.2.8	Modelos basados en árboles de decisión	32
	XGBoost	32
2.2.9	Transformaciones de variables	33
	Transformación logarítmica	33
	Codificación cíclica de variables periódicas	34
2.2.10	Combinación de pronósticos	35
2.2.11	Métricas de evaluación de pronósticos	36
	Error Absoluto Medio (MAE)	37
	Raíz del Error Cuadrático Medio (RMSE)	37
	Mediana del Error	38
	Coefficiente de Determinación (R^2)	38
	Error Porcentual Absoluto Medio (MAPE)	38
	Error Porcentual Absoluto Ponderado (WAPE)	39
2.2.12	Validación de modelos de series de tiempo	39
	Validación cruzada temporal (walk-forward)	40
	Prueba de Diebold-Mariano	40
	Corrección de muestra pequeña (Harvey-Leybourne-Newbold)	41
2.3	Antecedentes	43
2.3.1	Comparación de modelos de predicción de demanda	43
2.3.2	Implementaciones de referencia consultadas	45
2.3.3	Brecha identificada	46
2.4	Estándares de Ingeniería Aplicables	47
	ISO/IEC/IEEE 12207 — Procesos del ciclo de vida del software	47
	ISO/IEC/IEEE 29148 — Ingeniería de requisitos	47
	ISO/IEC 25010 — Calidad del producto de software	47

	Modelo C4	48
	Modelado de amenazas STRIDE	48
3	Capítulo 3: Diseño y Desarrollo	49
	3.1 Enfoque Metodológico	49
	3.1.1 Gestión del Proyecto (Planeación)	50
	3.1.2 Identificación y mitigación de riesgos	50
	3.1.3 Ciclo de Vida del Software (Ejecución)	51
	3.2 Procesos Técnicos y Diseño (ISO/IEC/IEEE 12207)	52
	3.2.1 Identificación de Stakeholders	52
	3.2.2 Elicitación de Requisitos	53
	3.2.3 Especificación de Requisitos de Software (SRS)	55
	3.2.4 Diseño de la Arquitectura	60
	3.2.5 Escenarios Arquitecturales	60
	3.2.6 Análisis de Alternativas y Decisiones de Diseño (ADRs)	61
	3.2.7 Representación de la Arquitectura (Modelo C4)	63
	3.2.8 Diseño Detallado	65
	3.2.9 Etapa 1: Carga y limpieza de datos	65
	3.2.10 Etapa 2: Segmentación de productos	68
	3.2.11 Etapa 3: Construcción de variables	69
	3.2.12 Etapa 4: Modelos de predicción	76
	Definición de variables y funciones basicas	76
	Modelos de demanda	78
	Implementación de XgBoost	78
	Implementación de Media Móvil	81
	Implementación de Regresión Lineal	82
	3.2.13 Selección del campeón por producto	87
	3.2.14 Combinación de pronósticos (análisis complementario)	91
	3.2.15 Pronóstico futuro	92
	Ciclo recursivo	93
	3.2.16 Cálculo de reposición (EOQ/ROP)	97
	Lead time	98
	Stock de seguridad y punto de reorden	98
	Cantidad económica de pedido (EOQ)	99
	Cantidad sugerida y presupuesto	99
	Decisión de pedido y fecha estimada	100

3.2.17	Modelo de Datos	101
3.2.18	Diseño UX/UI	106
3.2.19	Análisis de Amenazas y Diseño Seguro	115
3.3	Construcción, Integración y Despliegue	118
3.3.1	Construcción	118
3.3.2	Integración	120
	Integración continua (CI)	122
3.3.3	Despliegue	123
3.4	Plan de Verificación y Validación	127
3.4.1	Estrategia de verificación	127
3.4.2	Estrategia de validación del modelo de predicción	128
3.4.3	Criterios de aceptación	129
3.4.4	Plan de validación con usuarios	130
4	Capítulo 4: Análisis de Resultados y Juicio de Ingeniería	132
4.1	Resultados de Verificación (Pruebas de Software)	132
4.2	Resultados de la Validación del Modelo de Predicción	134
4.2.1	Consistencia mediante validación cruzada temporal	135
4.3	Resultados de Validación (Evaluación del Usuario)	138
4.4	Evaluación de la Seguridad y Estándares	139
4.4.1	Resultados del análisis de amenazas (STRIDE)	139
4.4.2	Nivel de cumplimiento frente a los estándares declarados	140
4.5	Análisis de Impactos de la Solución	141
4.5.1	Impacto técnico	141
4.5.2	Impacto social	141
4.5.3	Impacto económico	142
4.5.4	Impacto ambiental	142
4.5.5	Impacto ético	143
4.6	Conclusiones	143
4.7	Recomendaciones de Operación y Mantenibilidad	146
4.7.1	Continuidad y recuperación de datos	146
4.7.2	Seguridad	147
4.7.3	Verificación continua	147
4.7.4	Accesibilidad y mantenibilidad de la interfaz	147
4.8	Trabajos Futuros	148
4.8.1	Realización de validación con usuarios	148

4.8.2	Conservación del historial de artefactos por corrida	148
4.8.3	Validación en otros dominios de inventario	148
4.8.4	Exploración de modelos de aprendizaje profundo	149
4.8.5	Reevaluación del asistente conversacional	149
4.8.6	Combinación de pronósticos con métodos más sofisticados	149
4.9	Declaración de Uso Responsable de IA	149
Referencias Bibliográficas		151
ANEXOS		156
Anexo A	Documento completo SRS (historias de usuario / backlog)	157
Anexo B	Modelos de datos extensos, diccionarios de datos, mockups	169
Anexo C	Manual de usuario y/o guía de instalación y despliegue	178
Anexo D	Enlaces a repositorios de código fuente y tableros de gestión (Azure DevOps, Jira, GitHub)	181
Anexo E	Protocolo y seguimiento de la revisión sistemática de literatura (si aplica)	182

LISTA DE TABLAS

I	Ejemplo de cuatro meses utilizado para explicar las métricas.	37
II	Ejemplo de la prueba de Diebold-Mariano con y sin la corrección de muestra pequeña ($T = 4, h = 1$).	42
III	Comparación de técnicas de predicción aplicadas a problemas de demanda y series de tiempo	44
IV	Implementaciones de referencia consultadas para el desarrollo de Thary.	45
V	Identificación y mitigación de riesgos del proyecto.	51
VI	Análisis de interés e influencia de los stakeholders	53
VII	Trazabilidad entre casos de uso y requisitos funcionales	58
VIII	Entidades principales del modelo de datos.	102
IX	Resultados de la suite de pruebas automatizadas por archivo.	133
X	Métricas del holdout final por modelo.	134
XI	Resumen de la validación cruzada temporal (promedio $\pm$ desviación, 3 folds). . . .	137
XII	Estado de mitigación de las amenazas identificadas mediante STRIDE.	139
XIII	Nivel de cumplimiento frente a los estándares de ingeniería declarados.	140
XIV	Especificación del caso de uso CU-01: Iniciar sesión	157
XV	Especificación del caso de uso CU-02: Cargar historial de consumo	158
XVI	Especificación del caso de uso CU-03: Procesar archivo	159
XVII	Especificación del caso de uso CU-04: Consultar demanda pronosticada	160
XVIII	Especificación del caso de uso CU-05: Consultar estado del inventario	161
XIX	Especificación del caso de uso CU-06: Consultar recomendaciones de reposición y presupuesto	162
XX	Especificación del caso de uso CU-07: Registrar pedido	163
XXI	Especificación del caso de uso CU-08: Gestionar usuarios	164
XXII	Especificación del caso de uso CU-09: Deshacer pedido	165
XXIII	Especificación del caso de uso CU-10: Consultar actividad de la empresa	166
XXIV	Especificación del caso de uso CU-11: Consultar calendario de pedidos	167
XXV	Especificación del caso de uso CU-12: Consultar perfil	168

LISTA DE FIGURAS

1	Esquema detallado de entradas, proceso y salidas del sistema.	19
2	Esquema detallado de la propuesta de solución.	20
3	Casos de uso del Administrador	56
4	Casos de uso del Personal de Inventario	57
5	Diagrama de contexto del sistema (nivel 1 del modelo C4).	63
6	Diagrama de contenedores del sistema (nivel 2 del modelo C4).	64
7	Diagrama de componentes del sistema (nivel 3 del modelo C4).	65
8	Estructura de la tabla <code>empresas</code> vista desde MySQL Workbench, conectado a la base de datos local.	105
9	Consulta sobre la tabla <code>corridas</code> en MySQL Workbench, mostrando el esquema completo de 12 tablas y el historial acumulado de corridas.	105
10	Logotipo de Thary, compuesto por el nombre del sistema y el icono representativo del búho	106
11	Pantalla de inicio de sesión de Thary	106
12	Pantalla de registro de Thary	107
13	Pantalla de dashboard de Thary	107
14	Pantalla para subir archivos de Thary	107
15	Pantalla de predicción de demanda de Thary	108
16	Pantalla de gestión de usuarios de Thary	108
18	Estado del inventario	109
21	Próximos pedidos	112
22	Calendario de pedidos	112
23	Cerrar sesión	112
24	Alertas visuales	113
25	Dashboard del empleado	113
26	Dashboard en modo claro	114
27	Dashboard en modo mobile	115
28	Servicios desplegados en Railway: aplicación web (Thary) y base de datos MySQL.	124
29	Detalles de construcción y despliegue del servicio Thary en Railway.	125
30	Variables de entorno del servicio Thary en Railway, con los valores ocultos.	126
31	Tablas del esquema relacional de Thary en el entorno de producción, vistas desde el explorador de datos de Railway.	126
32	Pantalla de inicio de sesión de Thary, accedida desde su URL pública en Railway.	127

33	Ejecución detallada de la suite de pruebas mediante <code>pytest tests/ -v</code> , mostrando cada prueba individual.	132
34	Resumen final de la ejecución mediante <code>pytest tests/ -q</code>	133
35	Comparación de WAPE entre los tres modelos candidatos (holdout final).	135

LISTA DE CÓDIGOS

1	Función <code>cargar_y_limpiar</code> y etiquetas de las columnas obligatorias.	66
2	Función <code>calcular_comportamiento_producto</code> : variabilidad y clasificación ABC de cada producto.	68
3	Unión de las categorías ABC y de variabilidad a la tabla mensual.	70
4	Extracción del año y del mes de cada registro.	70
5	Codificación cíclica del mes mediante seno y coseno.	72
6	Cálculo de los rezagos <code>lag_1</code> , <code>lag_2</code> , <code>lag_3</code> y <code>lag_6</code> por producto.	72
7	Cálculo de las estadísticas móviles <code>media_movil_3</code> , <code>media_movil_6</code> y <code>desviacion_movil_3</code>	73
8	Incorporación de la media y la desviación históricas de cada producto.	73
9	Cálculo del inventario del mes anterior (<code>inventario_lag_1</code>).	74
10	Descarte de las filas sin historial suficiente para calcular los rezagos.	74
11	Lista de variables de entrada (<code>feature_cols</code>) y variables opcionales de precio e inventario.	75
12	Relleno final de los valores vacíos de las variables de entrada con 0.	76
13	Nivel de significancia de la prueba de Diebold-Mariano.	76
14	Constantes que definen el tamaño del periodo de prueba (<i>backtest</i>).	76
15	Función <code>tamano_test_backtest</code> , que decide el tamaño del periodo de prueba.	77
16	Parámetros del modelo XGBoost del <i>Ejemplo 3</i>	78
17	Función <code>entrenar_xgboost</code> , con los hiperparámetros del modelo.	79
18	Función <code>prediccion_baseline_movil</code> , que implementa el modelo de media móvil.	82
19	Filtro de las columnas sin variación (función <code>_seleccionar_features_regresion</code>).	82
20	Escalado de las variables con <code>MinMaxScaler</code>	83
21	Selección de variables mediante <i>Recursive Feature Elimination</i> (RFE).	83
22	Cálculo del VIF de las variables seleccionadas.	84
23	Eliminación de variables con VIF infinito o superior al límite.	85
24	Eliminación de variables según el p-valor del modelo OLS.	86
25	Función <code>entrenar_regresion_lineal</code> , que entrena el modelo con las variables seleccionadas.	86
26	Identificación del mejor y el segundo mejor modelo de cada producto (función <code>elegir_campeon_por_producto</code>).	87
27	Diferencia de error entre el mejor y el segundo mejor modelo.	88

28	Prueba de Diebold-Mariano con la corrección de Harvey, Leybourne y Newbold. . . .	89
29	Función <code>tamano_test_backtest</code> , que ajusta el periodo de prueba al historial disponible.	90
30	Cálculo del peso de la combinación (función <code>_peso_minimos_cuadrados</code>). . .	91
31	Cálculo del peso de cada mes sin usar ese mes (función <code>evaluar_combinacion</code>). .	92
32	Definición de las tablas <code>empresas</code> , <code>usuarios</code> , <code>corridas</code> y <code>segmentacion</code> con sus claves foráneas.	103
33	Configuración de la conexión a MySQL (<code>app/services/db.py</code>).	104
34	Función <code>crear_usuario</code> , que almacena la contraseña como hash.	116
35	Función <code>verificar_credenciales</code> , que compara la contraseña ingresada con su hash.	116
36	Función <code>pedido_confirmar</code> , que registra los pedidos con la cantidad y el costo calculados por el sistema.	116
37	Registro de la actividad del usuario al confirmar un pedido (función <code>pedido_confirmar</code>).	117
38	Función <code>carpeta_uploads</code> , que aísla los archivos cargados por cada empresa. . .	117
39	Registro de los blueprints <code>main_bp</code> y <code>auth_bp</code> en la aplicación Flask.	120
40	Función <code>ejecutar_sistema</code> , que orquesta las nueve etapas del pipeline.	120
41	Función <code>carpeta_resultados</code> , que define la carpeta de resultados de cada empresa.	122
42	Decorador <code>admin_required</code> , que restringe el acceso de una ruta al perfil administrador.	122
43	Flujo de trabajo de GitHub Actions para ejecutar las pruebas automatizadas (<code>tests.yml</code>).	123
44	Contenido del archivo <code>Procfile</code> para el servidor Gunicorn.	124
45	Constantes y función <code>_armar_folds</code> de la validación cruzada temporal.	136

Resumen

Palabras clave — Predicción de demanda, gestión de inventarios, sistemas de apoyo a la decisión, series de tiempo

El presente proyecto tiene como objetivo desarrollar un sistema de apoyo a la decisión orientado a la predicción de la demanda de productos en los inventarios.

Esta problemática surge a partir de la falta de información precisa sobre las cantidades necesarias de productos, lo cual dificulta la correcta planificación y puede llevar a que haya desabastecimiento, afectando negativamente la atención de los clientes.

Para abordar esta situación, en este trabajo se propone el uso de modelos de predicción basados en datos históricos de consumo. El sistema permitirá estimar las cantidades requeridas de productos cada mes, así como generar indicadores que apoyen la toma de decisiones para los encargados del manejo del inventario, tales como puntos de reposición y alertas.

Metodológicamente, se emplean diferentes técnicas y enfoques de la ingeniería de sistemas para obtener esta herramienta funcional, que contribuya a mejorar la gestión del inventario, optimizar la planificación de compras y reducir el riesgo de desabastecimiento.

Abstract

Keywords — Demand forecasting, inventory management, decisión support systems, time series

The objective of this project is to develop a decisión support system designed to predict demand for products in inventory.

This problem arises from a lack of accurate information regarding the required quantities of products, which hinders proper planning and can lead to shortages, negatively affecting customer service.

To address this situation, this study proposes the use of forecasting models based on historical consumption data. The system will enable the estimation of the required quantities of products each month, as well as the generation of indicators to support decisión-making for inventory managers, such as reorder points and alerts.

Methodologically, various techniques and approaches from systems engineering are employed to develop this functional tool, which will help improve inventory management, optimize purchasing planning, and reduce the risk of stockouts.

Glosario

Demanda: Cantidad de productos requeridos durante un período determinado, calculada a partir del comportamiento histórico de consumo o ventas.

Predicción de demanda: Proceso mediante el cual se estiman las cantidades futuras de productos utilizando modelos estadísticos o de aprendizaje automático basados en datos históricos.

Gestión de inventarios: Conjunto de actividades orientadas al control, almacenamiento, abastecimiento y seguimiento de los productos disponibles en una organización.

Inventario Conjunto de productos, materiales o recursos almacenados y disponibles para su uso, venta o distribución.

Desabastecimiento: Situación en la que un producto no se encuentra disponible en el inventario cuando es requerido.

Sistema de apoyo a la decisión: Herramienta informática diseñada para asistir a los usuarios en la toma de decisiones mediante el análisis de datos y la generación de información relevante.

Modelo de predicción: Algoritmo o método matemático utilizado para estimar valores futuros a partir de datos históricos y patrones identificados en la información.

Datos históricos: Registros de consumo, ventas, compras o movimientos de inventario ocurridos en períodos anteriores y utilizados para el análisis y la predicción.

Punto de reposición: Nivel de inventario a partir del cual se debe realizar un nuevo pedido para evitar el desabastecimiento.

Tiempo de entrega (Lead Time): Período que transcurre entre la realización de un pedido y la recepción efectiva de los productos solicitados.

Serie de tiempo: Conjunto de observaciones registradas en orden cronológico que permiten analizar patrones, tendencias y comportamientos a lo largo del tiempo.

Aprendizaje automático (Machine Learning): Rama de la inteligencia artificial que permite a los sistemas aprender patrones a partir de datos y realizar predicciones sin ser programados explícitamente para cada situación.

Dominio de inventario: Contexto o sector específico en el que se gestionan inventarios, como

retail, manufactura, logística, salud, restaurantes o comercio electrónico.

1 Capítulo 1: Introducción

1.1 Contexto

La gestión eficiente de inventarios supone un desafío constante en diversos sectores. Problemas como sufrir de debastabecimiento pueden surgir por distintos factores como retrados en la entrega por parte de los proveedores, variaciones inesperadas de la demanda, o deficiencias en los procesos de planificación. Estas situaciones, en muchos casos, se encuentran fuera del control de los administradores, lo que incrementa la complejidad para ellos a la hora de tomar decisiones relacionadas con el abastecimiento.

Actualmente, normalmente en muchas empresas se cuenta con un software para gestión de inventarios tipo ERP, con este software se administra la disponibilidad de los productos mediante el uso de diferentes tipos de movimientos de inventario:

Uno de los tipos de movimientos consiste en las entradas de inventario, las cuales responden a la adquisición de productos a través de proveedores y son registradas mediante facturas que incluyen información como fecha, proveedor y costo.

Por otro lado, están las salidas por consumo, las cuales representan la venta, distribución o consumo de los productos, registrando información relacionada con las cantidades entregadas y el destino de estas. Adicionalmente, existen salidas de inventario asociadas a sucesos como devoluciones, productos dañados, vencimientos, pérdidas o ajustes de inventario, se registra la cantidad de productos que deben de ser retirados del inventario junto con su respectivo motivo. Posteriormente, se realizan ajustes de inventario, los cuales pueden ser tanto positivos como negativos, con el fin de corregir diferencias entre las cantidades registradas en el sistema y las existencias reales, garantizando así la consistencia de la información.

En general, un buen análisis de estos movimientos resulta fundamental para comprender el comportamiento del consumo de productos y así mejorar los procesos de planificación del abastecimiento.

1.2 Planteamiento del Problema

En diferentes entornos, existe una problemática en la que la gestión del inventario de productos se ve afectado debido a la falta de información precisa sobre la demanda mensual de los mismos, lo que

dificulta la planificación adecuada del abastecimiento. En este contexto, surge la necesidad de contar con herramientas que permitan analizar los datos históricos de inventario y generar estimaciones de demanda que apoyen la toma de decisiones relacionadas con el abastecimiento.

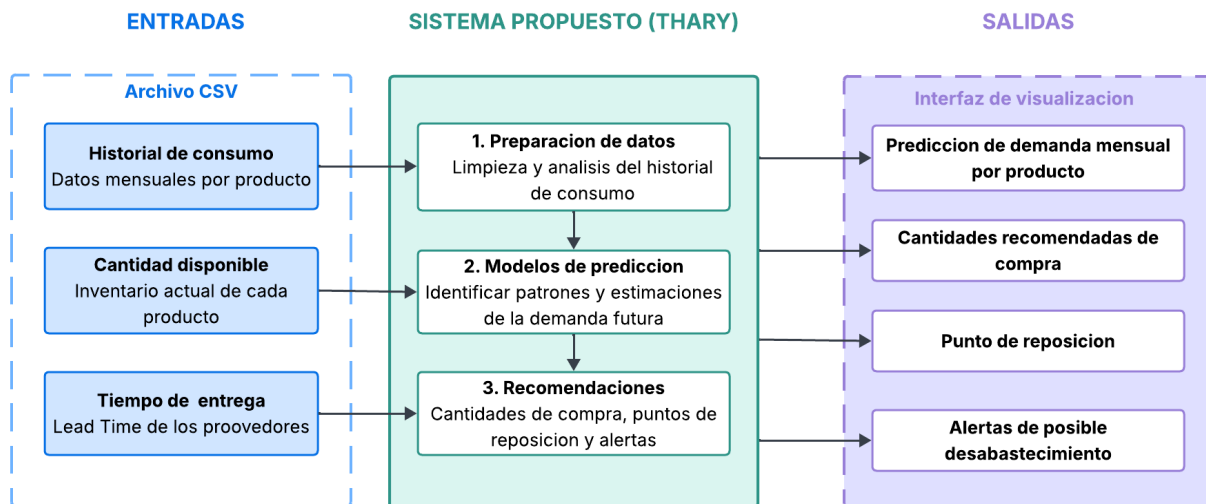


Fig. 1. Esquema detallado de entradas, proceso y salidas del sistema.

1.3 Justificación del Problema

Este proyecto se justifica teniendo en cuenta que esta es una problemática que no se ha visto abordada a profundidad y continua siendo una dificultad para organizaciones de diferentes sectores. Además este proyecto permite aplicar conocimientos de la ingeniería de sistemas como análisis de datos, modelos de predicción y aprendizaje automático para la construcción de una herramienta que contribuya a mejorar los procesos de planificación y administración, generando beneficios tanto para las organizaciones y los usuarios finales.

1.3.1 Impactos

Mejorar la disponibilidad de los productos mediante una mejor planificación de los inventarios, apoyar la toma de decisiones del personal encargado del abastecimiento, y fortalecer el uso de herramientas tecnológicas en el sector administrativo.

1.4 Propuesta de Solución

Con este proyecto se propone el desarrollo de un software que ayude a predecir la demanda de productos en diferentes dominios y contextos de gestión de inventarios. Este sistema se realizará tras analizar datos como historial de consumo, esto para poder estimar exactamente las cantidades necesarias de cada producto en periodos mensuales.

La solución implementará modelos de predicción de demanda que procesarán la información histórica para identificar patrones y en base a ellos, generar estimaciones futuras. A partir de estas predicciones, el sistema podrá proporcionar información relevante para la toma de decisiones, tales como cantidades recomendadas de compra, puntos de reposición y posibles alertas de desabastecimiento.

Se utilizarán datos reales del año 2021 del sector salud como caso de estudio, que serán complementados con datasets con datos públicos de diferentes dominios esto para poder evaluar el comportamiento del sistema en diferentes contextos.

Adicionalmente, el sistema tendrá en cuenta variables como tiempo de entrega de proveedores (lead time) y cantidad de productos disponible, con el propósito de lograr que el sistema sea lo más preciso posible.

Esta información será presentada en una interfaz que permitirá visualizar los resultados de manera comprensible, esto para facilitar su uso por parte del personal encargado de la gestión de inventarios.

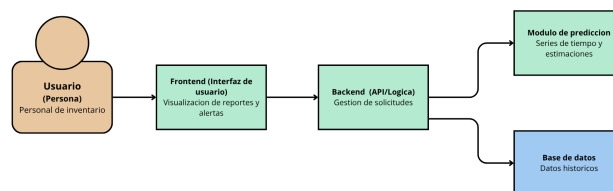


Fig. 2. Esquema detallado de la propuesta de solución.

1.5 Pregunta de Investigación

¿Cómo se puede diseñar e implementar un sistema de apoyo a la decisión que, mediante modelos de predicción de demanda, permita estimar las cantidades necesarias de productos en un sistema de gestión de inventarios?

1.5.1 Preguntas Derivadas

¿Qué variables y datos históricos son necesarios para estimar la demanda mensual de productos?

¿Qué modelos de predicción de demanda son más adecuados para estimar el consumo o venta de productos en diferentes dominios?

¿Cómo debe estructurarse el proceso de análisis y procesamiento de datos para generar predicciones confiables?

¿Qué tipo de salidas debe generar el sistema (por ejemplo, cantidades estimadas, punto de reposición, alertas) para apoyar la toma de decisiones?

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un sistema que le permita a los usuarios poder predecir cual va a ser la demanda mensual de productos mediante el análisis de datos históricos, para garantizar su correcta planificación y abastecimiento

1.6.2 Objetivos Específicos

Identificar las fuentes de datos relevantes, incluyendo el historial de consumo y las variables del inventario necesarias para la predicción de la demanda de productos.

Seleccionar modelos de predicción de demanda adecuados para estimar el consumo o venta mensual de productos en diferentes contextos de los inventarios.

Incorporar variables como el tiempo de entrega de proveedores y el punto de reposición dentro del sistema para complementar las estimaciones de demanda.

Diseñar un módulo de visualización que presente las predicciones de manera clara y comprensible para apoyar la toma de decisiones del usuario.

Evaluar el desempeño del sistema desarrollado mediante la realización de pruebas con datos históricos, con el fin de analizar la precisión de las predicciones generadas y verificar su utilidad en el apoyo a la toma de decisiones en la gestión del inventario.

1.7 Restricciones

El proyecto tiene como principal alcance el desarrollo de un sistema de apoyo a la decisión orientado a la predicción de la demanda de medicamentos críticos en una farmacia hospitalaria, esto utilizando los datos históricos de consumo como principal fuente de información. El sistema permitirá estimar la cantidad mensual requerida de medicamentos mediante la implementación de modelos de predicción de demanda, integrando funcionalidades como visualización de resultados, generación de reportes y apoyo en la toma de decisiones relacionadas con la planificación del abastecimiento. Asimismo, el sistema proporcionará al usuario cálculos como el punto de reposición, con el fin de contribuir a una mejor organización y control de los recursos disponibles en los entornos hospitalario.

El desarrollo del proyecto presenta algunas restricciones que deben ser consideradas. En primer lugar, el sistema se basará en un conjunto de datos históricos reales de la Clínica Nuestra señora de los remedios limitado al año 2021, lo que puede afectar la capacidad del modelo para capturar comportamientos actuales o cambios en la demanda a lo largo del tiempo. Además, no se contará con integración en tiempo real con sistemas hospitalarios, por lo que las predicciones se realizarán sobre datos estáticos.

1.8 Recursos y Viabilidad Técnica

1.8.1 Identificación de Recursos

El desarrollo del proyecto cuenta con los siguientes recursos:

Recursos de software. El proyecto se apoya en herramientas de software de código abierto, por lo que no se requiere de la adquisición de alguna licencia de pago. Entre las herramientas utilizadas se encuentran:

- Lenguaje de programación Python, junto con las librerías Flask (framework web), pandas, scikit-learn, XGBoost, statsmodels y SQLAlchemy, entre otras, para el desarrollo del sistema y del pipeline de predicción.
- MySQL como motor de base de datos relacional, y MySQL Workbench como herramienta de administración y consulta.
- Git y GitHub para el control de versiones del código fuente, junto con GitHub Actions para

la ejecución automatizada de pruebas (integración continua).

- *pytest* para la implementación de las pruebas automatizadas del sistema.
- Visual Studio Code como editor de código para el desarrollo del proyecto.

Recursos humanos. En cuanto a recursos humanos, el proyecto fue desarrollado individualmente por una única persona que tomó los roles de investigación, analista de datos y desarrolladora del sistema, esto junto a la orientación del asesor del proyecto semanalmente.

Recursos de hardware. Tanto el desarrollo del sistema como las pruebas se realizaron en un computador portátil con procesador Intel Core i5-6200U (2.30GHz), 16GB de memoria RAM y almacenamiento de 238GB en unidad de estado sólido (SSD), sin requerir hardware especializado (como unidades de procesamiento gráfico dedicadas) para el entrenamiento de los modelos de predicción, dado el volumen de datos manejado en el caso de estudio (856 productos, con historial mensual limitado al año 2021).

Infraestructura y despliegue. El despliegue del sistema se encuentra en la plataforma de hosting en la nube Railway, bajo un plan de uso gratuito, el cual es suficiente para el alcance del proyecto.

1.8.2 Análisis de Viabilidad

El análisis de viabilidad del proyecto se realiza teniendo en cuenta cuatro aspectos: técnica, económica, operativa y legal. A continuación se describen cada uno de ellos:

Viabilidad técnica. El proyecto se considera técnicamente viable, dado que se apoya únicamente en tecnologías de código abierto que se encuentran ampliamente documentadas.

Viabilidad económica. Debido a los límites con los que cuenta el proyecto, este se desarrolló implementando herramientas de acceso libre, que sean gratuitas o al menos tengan un plan gratuito, de manera que el costo económico del sistema es mínimo.

Viabilidad operativa. El diseño del sistema cuenta con una aplicación web que es accesible desde un navegador, no es necesario que el usuario final tenga que instalar un software adicional, por lo que es fácil de usar y adoptar en la operación diaria. Además, la interfaz de usuario cuenta con un diseño intuitivo y fácil de usar para cualquier tipo de usuario.

Viabilidad legal. Los datos históricos reales que fueron usados para probar el sistema fueron

proporcionados con fines exclusivamente académicos. El sistema cuenta con mecanismos de seguridad que garantizan la confidencialidad de la información manejada.

2 Capítulo 2: Marco de Referencia

Para la construcción del estado del arte, se realizó una revisión de literatura en bases de datos académicas reconocidas, con el fin de identificar estudios relevantes relacionados con la predicción de demanda y la gestión de inventarios.

2.1 Criterios de Revisión de Literatura

2.1.1 Criterios de inclusión

Se consideraron los siguientes criterios para la selección de los estudios:

- Trabajos disponibles en español o inglés.
- Publicaciones académicas en bases de datos como Scopus, ScienceDirect, Google Scholar y demás bases de datos, incluyendo las que ofrece la universidad de San Buenaventura.
- Estudios enfocados en gestión de inventarios.
- Investigaciones que utilicen modelos de predicción de demanda.
- Estudios que describan el diseño de sistemas informáticos de apoyo a la decisión.

2.1.2 Criterios de exclusión

Se excluyeron los estudios que cumplieran con las siguientes características:

- Estudios con más de 10 años de antigüedad.
- Documentos incompletos o resúmenes sin acceso al texto completo.

2.2 Fundamentos Teóricos

2.2.1 Gestión de inventarios

La gestión de inventarios consiste en el proceso que se realiza sobre la mercancía para conocer la cantidad disponible, esto permitiendo analizar los niveles de existencia y la aproximación del consumo real.[1]

Los pasos básicos para realizar una correcta gestión de inventario consiste en: Compra del inventario, en el que los productos se entregan al almacén o punto de venta, Almacenamiento del inventario, en el que este se guarda hasta que se necesite y Aprovechamiento del inventario, en donde se controla la cantidad de productos que hay a la venta para finalmente enviarlo o venderlo a los clientes.[2]

2.2.2 Gestión de inventarios con demanda independiente

Cantidad Económica de Pedido (EOQ):

La Cantidad Económica de Pedido (*Economic Order Quantity*, EOQ) consiste en un modelo de gestión de inventario que permite determinar el tamaño que debería tener un pedido para minimizar el costo total que supondría mantener el inventario. Toma en cuenta dos factores: El costo de hacer un pedido, el cual tiende a aumentar si se hacen muchos pedidos pequeños, y el costo de realizar el mantenimiento del inventario, el cual aumenta si el inventario es muy grande.[3]. El modelo se expresa como:

$$EOQ = \sqrt{\frac{2DS}{H}} \quad (1)$$

Donde D es la demanda anual del producto, S es el costo de realizar un pedido, y H es el costo anual de mantener una unidad en inventario. El resultado es la cantidad de unidades que debe pedirse cada vez que se realiza una orden de compra, de forma que el costo total (ordenar + mantener) sea el mínimo posible.

Punto de reorden y stock de seguridad:

El punto de reorden (*Reorder Point*, ROP) consiste en la cantidad mínima de producto que debe haber en el inventario, cuando se llega a dicho nivel, se debe generar una nueva orden de compra, de manera que el nuevo pedido llegue antes de que el inventario actual se agote [3]. Este se calcula como:

$$ROP = d \cdot L + SS \quad (2)$$

Donde d es la demanda promedio por periodo, L es el tiempo de entrega del proveedor (*lead time*), y SS es el stock de seguridad.

El Stock de Seguridad se trata de una cantidad adicional de cada producto que se mantiene en el inventario como medida de seguridad frente a la variabilidad de la demanda o del tiempo de entrega de los proveedores, protegiendo a la organización de quedarse sin existencias ante una demanda mayor a la esperada durante el periodo de espera del pedido.

2.2.3 Clasificación de inventarios

Clasificación ABC:

La clasificación ABC es una técnica que divide el catálogo de productos en tres categorías según su importancia dentro de la demanda o el valor total acumulado. Esta clasificación permite que se prioricen los productos de mayor impacto en vez de tratar a todo el catálogo con los mismos estándares [4].

- La categoría A agrupa a los productos de mayor importancia (típicamente aquellos que en conjunto representan la mayor parte de la demanda o el valor).
- La categoría B a los de importancia intermedia.
- La categoría C agrupa al resto de productos que representen un menor impacto individual.

Coefficiente de variación y clasificación de la demanda: Otro concepto importante a la hora de clasificar productos es poder clasificar la variabilidad de la demanda, ya que es probable que un producto que cuente con consumo estable se beneficie más de métodos de predicción más simples, mientras que uno con consumo errático puede requerir de un modelo más complejo.

Syntetos, Boylan y Croston [5] proponen un esquema de clasificación basado en el coeficiente de variación al cuadrado (CV^2) y el intervalo promedio entre demandas, estableciendo umbrales que permiten distinguir entre patrones de demanda suave, errática, intermitente y discontinua (*lumpy*), cada uno de los cuales se beneficia de un método de pronóstico distinto.

En cuanto al sistema, en Thary se retoma el concepto general de esta clasificación, pero de forma simplificada: se calcula el coeficiente de variación (CV, sin elevar al cuadrado), definido como la desviación estándar de la demanda dividida entre su media, y se clasifica la demanda de cada producto en tres niveles (baja, media o alta variabilidad), sin considerar el intervalo entre demandas como parte del análisis.

2.2.4 Predicción de demanda

La predicción de la demanda es un proceso que permite mejorar la gestión de los niveles de inventario mediante la utilización de datos históricos. Una correcta predicción de la demanda le permite a las organizaciones el poder tomar decisiones más inteligentes a la hora de manejar sus provisiones, evitando la falta o exceso de productos, de esta manera, aumentando tanto la satisfacción del cliente como la creación de estrategias de negocio más efectivas a corto y largo plazo [6].

En los últimos años, los mercados han evolucionado y se han vuelto cada vez más exigentes en cuanto a la calidad y precisión de los servicios, es por eso que muchas organizaciones se han visto inclinadas a incluir el uso de nuevas herramientas y tecnologías como Inteligencia Artificial, Machine Learning o análisis predictivo, para automatizar y perfeccionar esta tarea, ya que los métodos actuales estaticos que son la norma en muchas empresas y organizaciones han empezado a quedar obsoletos para satisfacer las necesidades del mercado actual [6], [7].

La principal debilidad de los modelos de inventario actuales es que al ser poco robustos, no toman en contemplacion diferentes factores externos de manera inteligente [7].

2.2.5 Series de tiempo

Las series temporales son usadas para analizar datos históricos recogidos en intervalos regulares, como ventas diarias o mensuales, para identificar patrones, tendencias estacionales y ciclos. Estos patrones se identifican mediante técnicas como gráficos ACF y PACF, también estos patrones sirven para ajustar modelos estadísticos o de aprendizaje automático (como ARIMA, SARIMAX o Prophet) que permiten proyectar valores futuros [8].

Se utilizan estos modelos para hacer predicciones, los datos se agrupan en intervalos (mensualmente) para reducir variabilidad y facilitar el análisis. Tras el análisis de datos, se ajustan los modelos y se evalúan mediante métricas como el RMSE, para que las predicciones sirvan en la toma de decisiones de gestión y planificación de recursos.

2.2.6 Modelos de predicción de demanda

Media Móvil:

La media móvil consiste un método de pronóstico que se utiliza para estimar la demanda futura de un producto a partir de promediar la cantidad de demanda de los periodos más recientes, en lugar de utilizar todo el historial disponible. Este es uno de los métodos más simples que existen para

realizar pronósticos de demanda, ya que no requiere ajustar parámetros complejos ni contar con un historial extenso para poder aplicarse [9].

Debido a esta simplicidad, la media móvil suele utilizarse como una línea base (*baseline*) contra la cual se comparan modelos más sofisticados, con el fin de determinar si estos últimos realmente aportan una mejora que justifique su mayor complejidad. Es también un método que se comporta bien cuando el producto tiene un consumo relativamente estable, ya que al no depender de patrones complejos, no se ve tan afectado por variaciones puntuales del historial como si le podría ocurrir a un modelo más complejo entrenado con pocos datos.

Redes Neuronales Artificiales (RNA): Las redes neuronales artificiales (RNA) son una técnica que se ha aplicado con frecuencia a la predicción de demanda [10]. A diferencia de un modelo como la regresión lineal, que solo capta relaciones directas entre las variables, las RNA pueden captar relaciones más complejas y no lineales entre los datos, incluso cuando esa relación no se conoce de antemano [10]. Es por esto que resultan útiles cuando la demanda no responde de forma proporcional a sus variables de entrada, algo difícil de conseguir con métodos más simples.

Long Short-Term Memory (LSTM) y Gated Recurrent Unit (GRU): LSTM y GRU son variantes de redes neuronales recurrentes, diseñadas específicamente para el procesamiento de datos secuenciales, a diferencia de las RNA de tipo *feed-forward*. Ambas incorporan mecanismos de memoria interna (compuertas) que les permiten capturar dependencias temporales de largo plazo en los datos, siendo GRU una variante más simple y computacionalmente menos costosa que LSTM [11], [12]. Estas dos técnicas fueron consideradas durante la revisión de literatura, pero finalmente no se incorporaron como modelos candidatos en Thary, principalmente porque requieren un volumen de historial considerablemente mayor al disponible en el caso de estudio para superar a técnicas más simples.

2.2.7 Modelos estadísticos y de Machine Learning tradicionales

Regresión lineal:

Los modelos de regresión son utilizados para resolver problemas de predicción; el objetivo de este modelo puede ser explicativo, donde se busca, por ejemplo, evaluar cómo ciertas variables independientes afectan a una variable dependiente, o predictivo, en el que se busca estimar el valor de la variable dependiente en función de las independientes [13].

Un modelo de regresión se expresa como:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon \quad (3)$$

donde y es la variable dependiente, $x_1, \dots, x_n$ son las variables independientes, β_0 es el intercepto, $\beta_1, \dots, \beta_n$ son los coeficientes de regresión y ϵ es el error aleatorio, es decir, la parte que no se puede explicar a partir de las variables incluidas en el modelo [13]. Los coeficientes se estiman mediante el método de mínimos cuadrados, que consiste en calcular la suma de las distancias al cuadrado entre los puntos reales y los definidos por la recta estimada, de forma que la mejor estimación es la que minimiza estas distancias [13].

Existen distintas opciones para estimar un modelo de regresión, siendo las más comunes la regresión lineal y la regresión logística. La elección depende del tipo de variable dependiente: si es una variable continua se utiliza regresión lineal, mientras que si se trata de una variable dicotómica se utiliza regresión logística [13].

Selección de variables en modelos de regresión: No todas las variables disponibles aportan información útil a un modelo de regresión, y construirlo con un exceso de ellas lo vuelve más complejo y menos confiable. Para decidir cuáles conservar existen distintas técnicas: la selección hacia adelante (*forward*), que introduce variables en el modelo mientras cumplan una condición; la selección hacia atrás (*backward*), que parte de un modelo con todas las variables y va suprimiendo las que cumplen una condición definida a priori hasta que no se pueda eliminar ninguna más; y la selección por pasos (*stepwise*), que combina ambas [13]. Cuando el objetivo del modelo es predictivo, se busca el que proporcione predicciones más fiables y, entre los que lo logran, el más sencillo posible [13].

Es necesario poder identificar cuales son las variables que realmente le aportan información al modelo de regresión, ya que de no hacerlo, el modelo podría volverse más complejo innecesariamente y menos confiable. Para esto, existen distintas técnicas de selección de variables:

- la selección hacia adelante (*forward*), que introduce variables en el modelo mientras cumplan una condición
- la selección hacia atrás (*backward*), que parte de un modelo con todas las variables y va suprimiendo las que cumplen una condición definida a priori hasta que no se pueda eliminar ninguna más
- la selección por pasos (*stepwise*), que combina ambas [13]

En general, se busca el modelo que de las predicciones fiables, y que al mismo tiempo, lo haga de la manera más sencilla posible [13].

Eliminación Recursiva de Características (RFE) La eliminación recursiva de características fue propuesta por Guyon et al. [14], y se usa para seleccionar algunas variables para crear un grupo más pequeño de variables. El procedimiento consiste en entrenar un modelo lineal en el que primero se ordenan las variables según el peso que cada una tiene en el modelo, después se elimina la menos importante y se repite el proceso con las variables restantes hasta quedarse con el número deseado [14]. Como el orden de importancia depende de la magnitud de los coeficientes, las variables deben estar en una escala comparable antes de aplicar el método. En Thary se aplica esta misma lógica de eliminación sobre el modelo de regresión lineal.

Significancia estadística (p-valor) Cuando se arma el modelo, para cada una de las variables se tiene que determinar si esta realmente tiene relación con la demanda o solo da la ilusión de tenerla por coincidencia. Para responder esto inicialmente se asume que la variable no tiene relación real con la demanda y se revisa si los datos respaldan o no esta suposición. Esta revisión se realiza mediante el cálculo del p-valor, si este es menor que el nivel de significancia α , se rechaza dicha hipótesis y se considera que el coeficiente es estadísticamente significativo [13]. Un nivel de significancia común es $\alpha = 0.05$ (95% de confianza), que es el que emplea Thary.

Multicolinealidad y Factor de Inflación de Varianza (VIF) Se habla de colinealidad cuando dos o más variables tienen prácticamente la misma información, por lo que si el modelo recibe dichos datos no sabrá a cual de las dos variables atribuirle el efecto, por lo que sus coeficientes se vuelven inestables y poco confiables [13].

El VIF o Factor de Inflación de Varianza, es el número que mide cuanto se repite una variable en las demás variables del modelo [15]. Se calcula como:

$$VIF_i = \frac{1}{1 - R_i^2} \quad (4)$$

donde R_i^2 es la proporción de la varianza de la i-ésima variable independiente que se asocia con las demás variables independientes del modelo [15]. El VIF indica cuántas veces aumenta la varianza del coeficiente estimado de esa variable respecto a la que tendría si no compartiera varianza con las demás ($R_i^2 = 0$) [15]. Cuanto más se parece una variable a una combinación de las otras, más

se acerca R_i^2 a 1 y más crece el VIF; en el caso extremo de dependencia lineal exacta, la tolerancia $(1 - R_i^2)$ es cero, el VIF tiende a infinito y la solución del modelo deja de ser única [15].

Para decidir cuándo un VIF es demasiado alto se usan reglas prácticas, como los valores de 4, 5 o 10 [15]. Thary usa el valor de 5, que corresponde a $R_i^2 = 0.80$: una variable se descarta cuando más del 80 % de su información ya está contenida en las demás. O'Brien advierte que estos valores no deben aplicarse de forma automática, ya que un VIF alto no siempre es un problema por sí solo, y que eliminar variables solo por su VIF, o seleccionarlás por pasos, puede dejar un modelo sin una base teórica sólida [15]. Como Thary busca predecir y no interpretar cada coeficiente, usa este límite como un criterio práctico para evitar modelos inestables, aceptando esta limitación.

2.2.8 Modelos basados en árboles de decisión

XGBoost:

XGBoost (*Extreme Gradient Boosting*) es un sistema de aprendizaje automático basado en el método de *tree boosting*, que se ha utilizado ampliamente para obtener resultados de vanguardia (*state-of-the-art*) en tareas relacionadas con datos y predicción [16].

Su funcionamiento consiste en un sistema de aprendizaje automatizado basado en el método de Tree Boosting, el cual que incluye un algoritmo de aprendizaje de arboles o por conjuntos: (*ensemble learning*). Este se basa en la idea de que en lugar de depender de un solo modelo, se combinen muchos árboles sencillos para obtener uno más potente. Cada uno de estos árboles es un árbol de regresión, que divide las observaciones según los valores de sus variables hasta llegar a una hoja, y a cada hoja se le asigna un peso, que es el valor que aporta a la predicción [16]. La predicción final para una observación es la suma de lo que aporta cada árbol, y se escribe de la siguiente manera [16]:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), f_k \in F \quad (5)$$

Donde:

$$F = \{f(x) = w_{q(x)}\}(q : R^m \rightarrow T, w \in R^T) \quad (6)$$

es el espacio de los árboles de regresión. En estas ecuaciones, K es la cantidad de árboles, f_k es el k -ésimo árbol, q es la estructura del árbol, que asigna cada observación a una hoja, T es el

número de hojas y w son los pesos de las hojas [16].

Los árboles no se entrenan todos a la vez, sino que se entrena uno por uno, en cada paso se agrega el árbol que más mejora el modelo construido hasta ese momento [16]. El problema de este proceso es que, si no se controla, los árboles pueden volverse demasiado complejos y memorizar los datos de entrenamiento en vez de aprender el patrón general, lo que se conoce como sobreajuste. Es por esto que XGBoost agrega al error de predicción un término de regularización que penaliza la complejidad de cada árbol [16]:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (7)$$

donde γ penaliza cada hoja adicional y λ penaliza los pesos grandes en las hojas. De esta manera, el modelo tiende a preferir árboles simples que sigan siendo predictivos. La librería incluye además una penalización de tipo L1 sobre los pesos (`alpha`), y en ambos casos, aumentar su valor hace que el modelo sea más conservador [17].

Para reducir aún más el sobreajuste se utilizan dos técnicas adicionales. La primera es la tasa de aprendizaje (*shrinkage*), que escala por un factor η el peso de cada árbol que se agrega, de modo que se reduce la influencia de cada árbol individual y se deja espacio para que los árboles siguientes mejoren el modelo [16]. La segunda es el submuestreo: en cada iteración se puede usar solo una fracción aleatoria de las observaciones de entrenamiento [17], y solo una fracción de las variables al construir cada árbol [16], [17]. Además, la profundidad máxima limita cuántas divisiones puede hacer cada árbol, y con ello su complejidad.

Todos estos son mecanismos que se incluyen en la configuración del modelo XGBoost en Thary.

2.2.9 Transformaciones de variables

Transformación logarítmica:

La demanda de un producto normalmente tiene muchos meses con valores bajos y pocos meses con valores muy altos, por lo que si el modelo se entrena directamente con esos valores, se enfocaría demasiado en los casos extremos y predeciría peor la mayoría de los casos normales. Para reducir este problema se aplica una transformación logarítmica, que hace que los valores muy grandes dejen de pesar tanto frente a los pequeños, y de esta manera se estabiliza la varianza de la serie, algo que se considera una de las formas más directas de lograrlo [18].

Un problema del logaritmo es que no está definido para el cero ni para valores negativos [18], y

como la demanda mensual de un producto puede ser cero, se utiliza una versión que le suma 1 a la demanda antes de aplicar el logaritmo, conocida como $\log1p$ [18]:

$$w_t = \log(y_t + 1) \quad (8)$$

Con esto, un mes con demanda cero sigue siendo cero después de la transformación, ya que $\log(0 + 1) = 0$. Cuando el modelo hace su predicción en esta escala transformada, se tiene que devolver a la escala original con la operación inversa, que es $y_t = e^{w_t} - 1$, porque lo que interesa es la demanda en unidades reales.

Aun así, hay que tener en cuenta que esta transformación no siempre mejora el pronóstico, ya que su beneficio depende de que realmente logre estabilizar la varianza de la serie: si lo logra, puede mejorar la precisión, pero si no lo logra, puede perjudicarla [19]. En Thary se aplica a la demanda antes de entrenar XGBoost, y las predicciones se devuelven a la escala original antes de compararlas con las de los demás modelos.

Codificación cíclica de variables periódicas:

Las variables de calendario como el mes se repiten cada cierto tiempo, es decir, después de diciembre vuelve a empezar enero. El problema es que si el mes se representa como un número del 1 al 12, el modelo va a ver a diciembre (12) y a enero (1) como dos meses muy lejanos entre sí, cuando en realidad son meses consecutivos. Para evitar esto se utiliza la codificación cíclica, que representa el mes con dos valores calculados con seno y coseno, de manera que la periodicidad quede representada de una forma que los modelos puedan interpretar [20]:

$$mes_sin = \sin\left(mes \times \frac{2\pi}{12}\right), \quad mes_cos = \cos\left(mes \times \frac{2\pi}{12}\right) \quad (9)$$

Estos dos valores ubican a cada mes como un punto en un círculo, y así diciembre y enero quedan tan cerca entre sí como cualquier otro par de meses consecutivos. Por ejemplo, diciembre queda en el punto $(0, 1)$, enero en $(0.50, 0.87)$ y febrero en $(0.87, 0.50)$, por lo que la distancia entre diciembre y enero es de aproximadamente 0.52, igual que la de enero a febrero. En cambio, con números enteros, la diferencia entre 12 y 1 sería de 11 unidades, mientras que la de enero a febrero sería de 1.

Se necesitan los dos valores y no solo uno, ya que con el seno por sí solo, meses distintos pueden dar el mismo resultado, por ejemplo, el mes 1 y el mes 5 tienen ambos un seno de 0.5, y es

la combinación de seno y coseno la que permite identificar cada mes de forma única. En Thary, el mes se representa de esta manera (`mes_sin` y `mes_cos`), como se explica en la Etapa 3.

2.2.10 Combinación de pronósticos

Cuando se cuenta con varios modelos de predicción para un mismo producto, lo habitual es quedarse con el que se equivoca menos, pero de esta manera se descarta por completo la información que podrían aportar los demás. La combinación de pronósticos plantea otra alternativa: en vez de escoger uno solo, combinar varios pronósticos en uno, con el objetivo de mejorar la precisión aprovechando la información de distintas fuentes y de no depender de un único "mejor" pronóstico [21]. Esta idea fue introducida por Bates y Granger [22].

La forma más común de hacerlo es con un promedio ponderado. Si se tienen dos pronósticos, $\hat{y}_1$ y $\hat{y}_2$, la combinación se calcula como [22]:

$$\hat{y}_c = w \cdot \hat{y}_1 + (1 - w) \cdot \hat{y}_2 \quad (10)$$

donde w es el peso que se le da al primer pronóstico y $(1 - w)$ el que se le da al segundo, por lo que los dos pesos siempre suman 1. Por ejemplo, si un modelo predice una demanda de 100 y el otro de 60, con $w = 0.5$ la combinación da 80, y con $w = 0.75$ da 90. Mientras w se mantenga entre 0 y 1, la combinación siempre queda entre las dos predicciones.

La pregunta es cuánto peso darle a cada modelo. Bates y Granger propusieron que el peso dependa de los errores que cada pronóstico tuvo en el pasado, de forma que el que se equivocó menos recibe un peso mayor [22]. Después, Granger y Ramanathan propusieron calcular los pesos con una regresión por mínimos cuadrados, en la que la demanda real es la variable que se quiere explicar y los pronósticos pasados de cada modelo son las variables explicativas [21], [23]. Su propuesta tiene tres variantes según las restricciones que se le pongan a la regresión, y la que obliga a que los pesos sumen 1 y no incluye constante da los mismos pesos "óptimos" de Bates y Granger [21]. Granger y Ramanathan señalaron que la variante sin restricciones y con constante es superior, aunque otros autores advierten que al usarla hay que tener en cuenta la estacionariedad de la serie, la correlación entre los errores y la multicolinealidad [21].

En Thary se utiliza la variante con pesos que suman 1 y sin constante, ya que de esta manera la combinación es un promedio ponderado fácil de interpretar. Con esta variante, el peso w que mejor se ajusta a la demanda real se obtiene minimizando la suma de los errores al cuadrado de la combinación:

$$\min_w \sum_i (y_i - (w \cdot \hat{y}_{1,i} + (1 - w) \cdot \hat{y}_{2,i}))^2 \quad (11)$$

donde y_i es la demanda real. Reordenando, esto equivale a una regresión por mínimos cuadrados sin constante de $(y_i - \hat{y}_{2,i})$ sobre $(\hat{y}_{1,i} - \hat{y}_{2,i})$, que tiene una solución directa:

$$w = \frac{\sum_i (y_i - \hat{y}_{2,i})(\hat{y}_{1,i} - \hat{y}_{2,i})}{\sum_i (\hat{y}_{1,i} - \hat{y}_{2,i})^2} \quad (12)$$

Siguiendo el ejemplo anterior, si un modelo predijo 100, el otro 60 y la demanda real fue 90, el peso que mejor se ajusta es $w = 0.75$, ya que $0.75 \cdot 100 + 0.25 \cdot 60 = 90$.

El peso no es una cantidad de unidades, sino una proporción que indica qué tan cerca quedó la demanda real de cada modelo. Siguiendo el ejemplo anterior, a la predicción de 60 le faltaron 30 unidades para llegar a la demanda real, y entre las dos predicciones hay 40 unidades de diferencia, por lo que el peso que mejor se ajusta es $w = 30/40 = 0.75$, ya que $0.75 \cdot 100 + 0.25 \cdot 60 = 90$. Cuando se tienen muchos meses y productos, el peso es el valor que mejor se ajusta a todos los casos a la vez. En Thary este cálculo se hace entre XGBoost y Media Móvil, y el peso se limita entre 0 y 1 para que siempre sea un promedio ponderado, ya que si la demanda real queda fuera del rango entre las dos predicciones, el cálculo daría un peso mayor a 1 o menor a 0. La combinación se evalúa únicamente como un análisis complementario en el panel técnico, sin participar en la selección del modelo campeón.

En Thary este cálculo se hace entre XGBoost y Media Móvil, el peso se limita entre 0 y 1 para que siempre sea un promedio ponderado, y la combinación se evalúa únicamente como un análisis complementario en el panel técnico, sin participar en la selección del modelo campeón.

2.2.11 Métricas de evaluación de pronósticos

Para saber qué tan bien predice un modelo, se compara la predicción con lo que realmente ocurrió, y la diferencia entre ambos valores es el error del pronóstico [24]:

$$e_t = y_t - \hat{y}_t \quad (13)$$

donde y_t es la demanda real y $\hat{y}_t$ es la predicción para el mes t . Como se tiene un error por cada producto y cada mes, se necesita una forma de resumirlos en un solo número, y a cada forma de

hacerlo se le llama métrica. El problema es que no existe una métrica que funcione bien en todas las situaciones, ya que cada una tiene casos en los que falla [24], por lo que es recomendable evaluar los mismos pronósticos con varias métricas a la vez, para comprobar que se comportan bien bajo distintos criterios [24]. Es por esto que Thary calcula seis métricas, que se explican a continuación con un mismo ejemplo de cuatro meses (Tabla I). En todas ellas, menos en el R^2 , un valor más bajo significa un mejor modelo.

TABLA I
Ejemplo de cuatro meses utilizado para explicar las métricas.

Mes	Demanda real	Predicción	Error absoluto
1	10	12	2
2	20	15	5
3	0	2	2
4	30	36	6

Error Absoluto Medio (MAE):

El MAE es el promedio de los errores, sin importar si la predicción quedó por encima o por debajo de la demanda real [24]:

$$MAE = \frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t| \quad (14)$$

Se expresa en las mismas unidades de la demanda, por lo que es fácil de interpretar. En el ejemplo, el MAE es $(2 + 5 + 2 + 6)/4 = 3.75$, es decir, en promedio la predicción se equivoca por 3.75 unidades. Un aspecto a tener en cuenta es que, cuando se calcula con todos los productos juntos, los productos de mayor demanda dominan el resultado, ya que sus errores en unidades son más grandes [24].

Raíz del Error Cuadrático Medio (RMSE):

En cuanto al RMSE, este es parecido al MAE, pero los errores se elevan al cuadrado antes de promediarlos y después se saca la raíz, para que el resultado vuelva a quedar en las unidades de la demanda [24]:

$$RMSE = \sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2} \quad (15)$$

Como los errores se elevan al cuadrado, los errores grandes pesan mucho más que los pequeños, por lo que el RMSE es más sensible a los valores atípicos que el MAE [24]. En el ejemplo, el RMSE es $\sqrt{(4 + 25 + 4 + 36)/4} = 4.15$. La diferencia se ve mejor con otro caso: si un producto tuviera un error de 97 unidades y otros tres de 1 unidad, el MAE sería 25, pero el RMSE sería 48.5. Es por esto que conviene mirar los dos, ya que cuando el RMSE es mucho mayor que el MAE, significa que hay unos pocos errores muy grandes.

Mediana del Error:

La mediana del error es la mediana de los errores absolutos, es decir, el valor que queda en la mitad cuando los errores se ordenan de menor a mayor [24]. A diferencia del MAE, que es un promedio, la mediana no se deja arrastrar por unos pocos casos con error muy grande, ya que la mediana se ve menos afectada por los valores atípicos que el promedio [24]. En el ejemplo, los errores ordenados son 2, 2, 5 y 6, por lo que la mediana es $(2 + 5)/2 = 3.5$. En el caso anterior del error de 97 unidades y tres de 1 unidad, el MAE era 25, pero la mediana es solo 1. En Thary se calcula junto al MAE para revisar si el MAE de un modelo está siendo inflado por pocos productos, o si de verdad representa el error típico del catálogo.

Coefficiente de Determinación (R^2):

El R^2 indica qué proporción de la variabilidad de la demanda real logra explicar el modelo, y multiplicado por 100 se interpreta como el porcentaje de la varianza que queda explicada [13]. Se calcula como:

$$R^2 = 1 - \frac{\sum_{t=1}^n (y_t - \hat{y}_t)^2}{\sum_{t=1}^n (y_t - \bar{y})^2} \quad (16)$$

donde $\bar{y}$ es el promedio de la demanda real. Con esta fórmula, un R^2 de 1 significa predicciones perfectas, un R^2 de 0 significa que el modelo no es mejor que predecir siempre el promedio de la demanda, y un valor negativo significa que es peor que eso. En el ejemplo, el promedio de la demanda real es 15, por lo que el denominador es $25 + 25 + 225 + 225 = 500$, y el R^2 es $1 - 69/500 = 0.862$, es decir, el modelo explica el 86.2 % de la variabilidad de la demanda.

Error Porcentual Absoluto Medio (MAPE):

Otra métrica que se utiliza es el MAPE, que indica qué porcentaje de la demanda real se falló en promedio. Para calcularlo, primero se divide el error de cada mes entre la demanda real de ese mes, para expresarlo como porcentaje, y después se promedian esos porcentajes [24]:

$$MAPE = \frac{1}{n} \sum_{t=1}^n \left| \frac{100 (y_t - \hat{y}_t)}{y_t} \right| \quad (17)$$

Al ser un porcentaje, es fácil de entender y de comparar entre productos con distintos volúmenes. Sin embargo, tiene dos problemas. El primero es que cuando la demanda real es cero no se puede calcular, ya que se divide entre cero, y cuando es cercana a cero da valores muy grandes [25]. El segundo es que cuando la demanda es pequeña, el porcentaje exagera el error [24]: por ejemplo, si un producto vende 1 unidad y se predicen 2, el error es de 100 %, aunque solo se hayan equivocado por una unidad. En Thary, por el primer problema, el MAPE se calcula únicamente con los meses en los que la demanda real es mayor que cero. En el ejemplo, se excluye el mes 3 y el MAPE es $(20 \% + 25 \% + 20 \%) / 3 = 21.67 \%$.

Error Porcentual Absoluto Ponderado (WAPE):

El WAPE, en vez de calcular un porcentaje para cada mes y promediarlo, divide el error total entre la demanda total [24]:

$$WAPE = \frac{\sum_{t=1}^n |y_t - \hat{y}_t|}{\sum_{t=1}^n |y_t|} \times 100 \quad (18)$$

Es decir, indica qué porcentaje de la demanda total se falló. Como el denominador es la demanda total y no la de cada mes, un producto con demanda baja pesa poco en el resultado, y un mes con demanda cero no impide calcularlo mientras la demanda total sea mayor que cero. De esta manera, el WAPE ayuda a evitar el problema de los porcentajes exagerados cuando la demanda real es pequeña, ya que la escala se calcula con valores agregados [24]. En el ejemplo, el WAPE es $15/60 = 25 \%$. Aun así, el WAPE tiene sus propias limitaciones, por ejemplo en series con tendencias fuertes, ya que una escala calculada con varios meses puede no representar bien a todos ellos [24].

En Thary, las seis métricas se calculan juntando las predicciones de todos los productos en el periodo de prueba, y el criterio de aceptación principal del sistema se define con el WAPE (Sección 3.4.3).

2.2.12 Validación de modelos de series de tiempo

Cuando se entrena un modelo de predicción no basta con que funcione bien con los datos con los que se entrenó, sino que hay que comprobar que también funciona con datos que nunca vio. En las series de tiempo esto tiene una dificultad adicional, ya que los datos tienen un orden, por lo que

la forma de separar los datos de entrenamiento y de prueba, y la forma de decidir si un modelo es realmente mejor que otro, requieren técnicas propias [24]. A continuación se explican las tres que se utilizan en Thary.

Validación cruzada temporal (walk-forward):

En la validación cruzada tradicional (*k-fold*), los datos se mezclan al azar y se dividen en partes, y cada parte se usa por turnos para evaluar el modelo [24]. En una serie de tiempo esto es un problema, ya que el modelo termina entrenándose con meses posteriores a los que después tiene que predecir, es decir, con información del futuro que en la realidad no tendría [24]. A esto se le conoce como fuga de datos (*data leakage*), que consiste en usar durante el entrenamiento datos de prueba, o datos que no estarían disponibles al momento de predecir [24].

Para evitarlo se utiliza el esquema de origen móvil (*rolling origin*), también llamado validación cruzada de series de tiempo, en el que siempre se respeta el orden cronológico: el modelo se entrena únicamente con los meses anteriores al periodo de prueba, y el punto de corte, llamado origen del pronóstico, se va desplazando hacia adelante en el tiempo, de manera que se generan varios periodos de prueba en vez de uno solo [24]. Cada vez que el origen avanza, los meses que antes eran de prueba pasan a formar parte del entrenamiento del siguiente paso [24]. Existen dos variantes: la ventana expansiva (*expanding window*), en la que el entrenamiento crece en cada paso, y la ventana móvil (*rolling window*), en la que el tamaño del entrenamiento se mantiene constante y se van descartando los meses más antiguos. La ventana expansiva es una buena opción cuando las series son cortas [24].

Por ejemplo, con 9 meses de historial y una ventana de prueba de 3 meses, una ventana expansiva genera 3 pruebas: la primera entrena con los meses 1 al 4 y evalúa del 5 al 7, la segunda entrena del 1 al 5 y evalúa del 6 al 8, y la tercera entrena del 1 al 6 y evalúa del 7 al 9. En este documento a este esquema se le llama validación cruzada temporal o *walk-forward*, y es el que utiliza Thary (Sección 4.2). Un problema de este esquema es que los primeros pasos pueden contar con pocos datos de entrenamiento [24], algo que ocurre en Thary, donde el primer fold se entrena con solo 4 meses.

Prueba de Diebold-Mariano:

La validación cruzada entrega los errores de cada modelo mes a mes, pero no responde una pregunta importante: si un modelo tiene un error promedio menor que otro, ¿de verdad es mejor, o solo tuvo suerte en esos meses? En una muestra concreta siempre uno de los dos tiene que ganar, aunque en realidad ambos sean igual de buenos [26]. La prueba de Diebold-Mariano [27] es una herramienta estadística para responder eso, ya que permite evaluar si una superioridad aparente es significativa [26].

Su funcionamiento se puede explicar en cuatro pasos:

1. Se define una pérdida $L(e_t)$, que es una forma de medir qué tan costoso es el error de cada mes. Por ejemplo, la pérdida cuadrática es $L(e_t) = e_t^2$ [26]. En Thary se utiliza el error absoluto, $L(e_t) = |e_t|$.
2. Se calcula, mes a mes, la diferencia entre la pérdida del modelo 1 y la del modelo 2 [26]:

$$d_t = L(e_{1t}) - L(e_{2t}) \quad (19)$$

3. Se plantea la hipótesis nula de que los dos modelos tienen la misma pérdida esperada, es decir, que el promedio de las diferencias es cero, $E(d_t) = 0$ [26].
4. Se calcula el estadístico [26]:

$$DM = \frac{\bar{d}}{\hat{\sigma}_{\bar{d}}}, \quad \bar{d} = \frac{1}{T} \sum_{t=1}^T d_t \quad (20)$$

donde $\bar{d}$ es el promedio de las diferencias, $\hat{\sigma}_{\bar{d}}$ es su error estándar y T es el número de meses. Este estadístico se aproxima a una distribución normal estándar [26].

Es decir, se trata de una prueba de que el promedio de una serie, en este caso la serie de diferencias, es igual a cero [26]. Si el p-valor es menor al nivel de significancia α (0.05 en Thary), se rechaza la hipótesis nula y se considera que la diferencia entre los modelos es real. Un aspecto importante es que las diferencias pueden estar correlacionadas entre meses, por lo que el error estándar debería calcularse de forma robusta [26].

Cabe aclarar que Diebold [26] señala que esta prueba fue diseñada para comparar pronósticos y no modelos, y que para comparar modelos existen procedimientos más sencillos que usan toda la muestra. Aun así, el mismo autor reconoce que el análisis fuera de muestra puede ser útil para conocer el desempeño predictivo comparado en un periodo histórico [26]. En Thary se usa con este último propósito, sobre los pronósticos del backtest de cada producto, y se reconoce como una limitación. Además, en Thary el error estándar se calcula de forma sencilla, como la desviación estándar de las diferencias dividida entre $\sqrt{T}$, sin el ajuste por autocorrelación, ya que el backtest solo cuenta con entre 3 y 6 meses por producto.

Corrección de muestra pequeña (Harvey-Leybourne-Newbold):

La prueba de Diebold-Mariano usa la distribución normal, que es una aproximación que funciona

bien cuando se tienen muchos datos. Con pocos datos esta aproximación puede ser muy mala, y la prueba puede rechazar la hipótesis nula más veces de las que debería, es decir, declarar como significativas diferencias que no lo son [27], [28]. Como Thary solo cuenta con entre 3 y 6 meses de backtest por producto, este problema es especialmente relevante.

Harvey, Leybourne y Newbold [28] propusieron dos ajustes: corregir el estadístico y compararlo contra una distribución t de Student con $T - 1$ grados de libertad, en vez de la normal. El estadístico corregido es:

$$DM^* = \sqrt{\frac{T + 1 - 2h + h(h - 1)/T}{T}} \cdot DM \quad (21)$$

donde h es el horizonte del pronóstico, es decir, cuántos meses hacia adelante se predice. Cuando se predice un mes hacia adelante ($h = 1$), la fórmula se simplifica a $DM^* = \sqrt{(T - 1)/T} \cdot DM$, que es la que utiliza Thary. La corrección hace dos cosas: reduce un poco el estadístico, y usa una distribución t que exige un valor más grande para declarar significativa una diferencia cuando hay pocos datos. En otras palabras, con pocos meses se necesita una ventaja más clara y sostenida para crearla.

La Tabla II muestra dos casos de un producto con 4 meses de backtest, en los que el mejor modelo tiene el mismo error, pero el segundo modelo cambia.

TABLA II

Ejemplo de la prueba de Diebold-Mariano con y sin la corrección de muestra pequeña ($T = 4$, $h = 1$).

	Caso A	Caso B
Error absoluto del mejor modelo (4 meses)	2, 3, 2, 3	2, 3, 2, 3
Error absoluto del segundo modelo	4, 3, 6, 9	7, 9, 6, 8
Diferencias d_t	-2, 0, -4, -6	-5, -6, -4, -5
Promedio $\bar{d}$	-3	-5
Desviación estándar de d_t	2.58	0.82
DM	-2.32	-12.25
p-valor sin corrección (normal)	0.020	≈ 0
DM^*	-2.01	-10.61
p-valor con corrección (t , 3 g.l.)	0.138	0.002
Conclusión ($\alpha = 0.05$)	No significativa	Significativa

En el caso A, el mejor modelo tiene un MAE de 2.5 frente a 5.5 del segundo, es decir, menos de la mitad del error, pero su ventaja no es sostenida: en el segundo mes ambos se equivocan igual. Sin la corrección parecería significativa (p-valor de 0.020), pero con la corrección deja de serlo (0.138), por lo que en Thary ese producto usaría la Media Móvil. En el caso B, en cambio, la ventaja es grande y se repite todos los meses, por lo que sigue siendo significativa aun con la corrección y el producto conserva su mejor modelo.

2.3 Antecedentes

2.3.1 Comparación de modelos de predicción de demanda

Debido a que existen diferentes modelos y técnicas para la predicción de la demanda, se realizó una revisión de literatura de diferentes estudios que emplean los modelos más utilizados para la predicción de la demanda, esto con el fin de realizar el siguiente análisis comparativo para determinar cual es el más adecuado para la predicción de demanda en el presente proyecto, teniendo en cuenta ventajas, limitaciones y características como: tipo de datos utilizados, dominio de aplicación, técnica de aplicación, métricas de evaluación y resultados obtenidos, dataset utilizado, entre otros.

TABLA III

Comparación de técnicas de predicción aplicadas a problemas de demanda y series de tiempo

Aspecto	Redes neuronales (ANN)	LSTM	Regresión lineal	GRU	XGBoost
Referencia	<i>Demand Forecasting Using Neural Network for Supply Chain Management</i> [10]	<i>LSTM Model Based Demand Forecasting in Supply Chain Using Efficient Optimizer</i> [29]	<i>Application of ABC Analysis and Linear Regression Model in Inventory Management</i> [30]	<i>StockNet-GRU Based Stock Index Prediction</i> [31]	<i>XGBoost-Based Demand Forecasting in Supply Chain Management Using Machine Learning Algorithm</i> [32]
Datos utilizados	Datos históricos de ventas organizados como series de tiempo. Se emplean ventas mensuales de un filtro de combustible entre 2011 y 2013 para predecir la demanda de 2014.	Datos históricos de ventas organizados como series de tiempo. Incluyen fecha, unidades vendidas, ganancias y ubicación, seleccionando las variables relevantes para el entrenamiento.	Datos históricos de inventario y ventas. Incluyen información de productos, inventario, órdenes de compra y venta, cantidad y precio de venta, fecha, marcas, clasificación e impuestos.	Series de tiempo financieras correspondientes a precios históricos de apertura del índice bursátil CNX Nifty y de las 10 acciones con mayor correlación. Los datos se normalizan y organizan mediante ventanas deslizantes de 20 días.	Datos logísticos y de cadena de suministro relacionados con la demanda, incluyendo variables asociadas con proveedores y logística. Los datos se normalizan mediante Z-score y posteriormente se seleccionan las variables relevantes.
Dominio	Gestión de la cadena de suministro, control de inventarios, manufactura y distribución. También presenta aplicaciones en turismo, consumo eléctrico, supermercados y ventas.	Gestión de la cadena de suministro y predicción de demanda.	Gestión de inventarios, cadena de suministro y análisis de ventas.	Predicción de índices bursátiles, mercados financieros y análisis de series de tiempo.	Gestión de la cadena de suministro, predicción de demanda, logística y optimización de inventarios. El enfoque puede aplicarse a diferentes sectores que requieran pronósticos de demanda.
Técnica	Red neuronal <i>feed-forward</i> entrenada mediante <i>Backpropagation</i> . Se comparan Gradient Descent, Conjugate Gradient y Levenberg-Marquardt, siendo este último el de mejor desempeño.	Red neuronal recurrente LSTM con tres capas, regularización mediante Dropout y optimizadores Adam y SGD. Se experimenta con épocas, tasa de aprendizaje, tamaño de lote y longitud de secuencia.	Modelo de regresión lineal para identificar tendencias en cantidad y precio de venta. Utiliza Gradient Descent para minimizar el error cuadrático, después de realizar limpieza de datos y selección de variables.	Red neuronal recurrente GRU integrada en la arquitectura StockNet. Utiliza ventanas deslizantes de 20 días, optimizador Adam, activación ReLU y datos normalizados entre 0 y 1.	El proceso incluye normalización Z-score, selección de características mediante un Algoritmo Genético, predicción mediante XGBoost y optimización de hiperparámetros mediante Particle Swarm Optimization (PSO).
Métricas	Error absoluto de pronóstico (<i>Forecasting Error</i>) y porcentaje de error, comparando los valores pronosticados con los valores reales.	RMSE para medir la diferencia entre valores reales y predichos. También se compara el tiempo promedio por época para evaluar la eficiencia computacional.	Coefficiente de determinación (R^2). El estudio reporta un valor de 70,57 % para la predicción de la cantidad vendida y cercano al 80 % para el precio de venta.	RMSE, MAE y MAPE. También se considera el tiempo de ejecución y se utiliza la prueba estadística de Wilcoxon para validar diferencias entre modelos.	MAE, RMSE y coeficiente de determinación. Se compara el desempeño de XGBoost con SARIMA, Decisión Tree (DT) y Random Forest (RF).
Dataset	Datos reales de consumo mensual de un filtro de combustible perteneciente a una empresa manufacturera. Contiene registros mensuales entre 2011 y 2013 para entrenamiento y validación.	<i>Sales Dataset</i> con información histórica de ventas utilizada para entrenar y evaluar el modelo.	Dataset de inventario de una tienda de comestibles (<i>grocery store</i>) con información de productos, inventario y ventas. Después del preprocesamiento se utilizan aproximadamente 60 observaciones correspondientes a dos meses.	Índice bursátil CNX Nifty y las 10 acciones más correlacionadas del National Stock Exchange (NSE) de India, obtenidos desde Yahoo Finance. Contiene 5.904 días de negociación entre abril de 1996 y enero de 2020.	Dataset de demanda de cadena de suministro con información relacionada con logística y proveedores. Se utiliza para entrenar y evaluar el modelo de predicción.
Comentario / reseña	Las ANN permiten modelar relaciones no lineales y presentan resultados favorables frente a métodos tradicionales de series de tiempo. Son útiles para predicciones de corto plazo, aunque requieren suficiente información y ajuste de parámetros.	LSTM constituye una alternativa eficaz para la predicción de demanda debido a su capacidad para capturar dependencias temporales de largo plazo. Puede contribuir a la optimización de inventarios y a la toma de decisiones en la cadena de suministro.	La regresión lineal permite identificar y proyectar tendencias generales de ventas e inventario y presenta una menor complejidad. Sin embargo, tiene limitaciones para representar relaciones complejas y eventos extraordinarios.	GRU permite capturar dependencias temporales y presenta resultados favorables en la predicción de series de tiempo. Sin embargo, el estudio analizado se desarrolla en el dominio financiero, por lo que su aplicación a inventarios requiere evaluación adicional.	XGBoost presenta capacidad para modelar relaciones complejas y puede utilizarse para apoyar la planificación de inventarios, reducir costos y optimizar operaciones en la cadena de suministro.

Fuente: Elaboración propia a partir de los estudios revisados.

2.3.2 Implementaciones de referencia consultadas

Además de los estudios académicos revisados, se consultaron cuatro implementaciones prácticas de código abierto, disponibles en Kaggle y GitHub, que sirvieron como base y punto de partida para la construcción del pipeline de predicción de Thary. La Tabla IV resume cada una de estas implementaciones, el identificador que se usara para referirse a ellas a lo largo del documento, y el aporte que realizaron al diseño del sistema.

TABLA IV
Implementaciones de referencia consultadas para el desarrollo de Thary.

ID	Autor	Título	Aporte a Thary
Ejemplo 1 (SKU Demand Forecasting)	Dhruvi Shah [33]	<i>Demand Forecasting & Inventory Optimization for Consumer Products</i>	Base para el flujo general de predicción y cálculo de reposición
Ejemplo 2 (Media Móvil)	Samir Saci [34]	<i>Machine Learning for Retail Demand Forecasting: Comparative Study of Demand Forecasting Methods for a Retail Store (XGBoost Model vs. Rolling Mean)</i>	Base para el uso de la media móvil como modelo candidato de comparación
Ejemplo 3 (XGBoost)	ksmooi [35]	<i>Forecasting Inventory Demand Using XGBoost</i>	Base para el entrenamiento y ajuste del modelo XGBoost
Ejemplo 4 (BoomBikes)	Tushar Verma [36]	<i>Linear Regression: Demand Forecasting</i>	Base para la metodología de selección de variables en el modelo de Regresión Lineal (RFE, eliminación por p-valor y VIF)

A lo largo del Capítulo 3 se hace referencia a estas cuatro implementaciones mediante el identificador presentado en la Tabla IV (Ejemplo 1, Ejemplo 2, Ejemplo 3 o Ejemplo 4), indicando en cada caso que partes específicas de Thary se basaron en ellas y cuales corresponden a extensiones propias desarrolladas durante el proyecto.

2.3.3 Brecha identificada

Una vez habiendo analizado algunas de las implementaciones ya existentes para la resolución de una problemática similar a la planteada en este proyecto, se identificó que la mayoría de estas implementaciones no contemplan un enfoque de selección de modelos por producto, sino que utilizan un único modelo (ANN, LSTM, Regresión Lineal, GRU o XGBoost) para todo el catálogo.

El único ejemplo que hace algo parecido a comparar varios modelos es el Ejemplo 2, que compara XGBoost frente a Media Móvil, pero no incluye un mecanismo de selección automática por producto.

Esto puede generar problemas de predicción, ya que no todos los productos tienen un comportamiento similar y por lo tanto, no todos los modelos son adecuados para todos los productos.

Además, la mayoría de estas implementaciones no cuentan con un mecanismo de validación estadística que permita confirmar que las diferencias observadas entre modelos sean reales y no producto del azar. Los estudios reportan únicamente el modelo con mejor métrica de error (MAE, RMSE, MAPE o R^2) como superior, sin verificar si dicha diferencia es estadísticamente significativa o podría explicarse por casualidad.

Por último, la mayoría de estas implementaciones solo se centran en la etapa de predicción de la demanda, sin traducir los resultados a un análisis que pueda ayudar a los usuarios a tomar decisiones operativas de reposición como punto de reorden, cantidad económica de pedido o costo de ruptura ante un posible desabastecimiento. Solo uno de los estudios académicos revisados, el de Lin [30] (Regresión Lineal, Tabla III), incorpora un análisis de clasificación ABC, pero de forma independiente al modelo predictivo, no como parte de un mismo sistema integrado.

En resumen, la diferencia entre la propuesta de Thary en comparación a los antecedentes revisados es que Thary busca integrar, dentro de un mismo sistema, la comparación de múltiples modelos candidatos, un mecanismo formal de selección respaldado estadísticamente, y su traducción directa en recomendaciones operativas de inventario, aplicado y validado sobre un caso de estudio real, en este caso, del dominio de salud.

2.4 Estándares de Ingeniería Aplicables

El desarrollo del sistema Thary se realizó en base a un conjunto de estándares y normas de la ingeniería de sistemas, los cuales fueron aplicados de forma continua a lo largo de las distintas etapas del proyecto, y su implementación y resultados son detallados en el Capítulo 3 y 4 respectivamente.

Se aplicaron estándares formales publicados por organismos oficiales como: ISO/IEC/IEEE 12207, IEEE 29148 e ISO/IEC 25010. También se aplicaron las metodologías creadas por autores independientes: Modelo C4 y modelado de amenazas STRIDE.

ISO/IEC/IEEE 12207 — Procesos del ciclo de vida del software: La norma ISO/IEC/IEEE 12207 del año 2026 [37] define un marco de procesos para todo el ciclo de vida del software, desde el inicio del proyecto hasta el final.

En Thary, esta norma orienta la organización de los procesos técnicos, los cuales incluyen: identificación de stakeholders, elicitación y especificación de requisitos, diseño de la arquitectura, diseño detallado, análisis de amenazas y el diseño seguro.

ISO/IEC/IEEE 29148 — Ingeniería de requisitos: Por su parte, la norma ISO/IEC/IEEE 29148 [38] se encarga de establecer una serie de lineamientos específicos para los procesos relacionados con la parte de ingeniería de requisitos, dichos procesos incluyen la elaboración de especificaciones de requisitos de software (SRS) mediante casos de uso, por lo que esta norma fue la que se tomó en cuenta a la hora de realizar la construcción de dicha especificación de requisitos. Se estructuró en: requisitos funcionales, requisitos no funcionales, y su trazabilidad hacia los casos de uso, la cual fue documentada en detalle en el Anexo A.

ISO/IEC 25010 — Calidad del producto de software: Se utilizó la norma ISO/IEC 25010 [39] para definir el modelo de características de calidad del software, el cual fue utilizado como marco de referencia para los escenarios arquitecturales posteriormente definidos.

Entre los escenarios definidos que se relacionan directamente con las características de calidad se encuentran:

- El escenario de confiabilidad ante datos escasos el cual corresponde a la característica de *fiabilidad*;
- El escenario de desempeño ante catálogos extensos corresponde a *eficiencia de desempeño*;

- El escenario de confidencialidad de la información corresponde a *seguridad*;
- El escenario de adaptabilidad del formato de datos corresponde a *portabilidad*.

Modelo C4: El Modelo C4 se utilizó para representar la arquitectura de Thary en distintos niveles (Contexto, Contenedores, Componentes), permitiendo tener tanto una vista general del sistema como el detalle interno del subsistema de predicción.

Modelado de amenazas STRIDE: Como parte del análisis de seguridad del sistema, se aplicó el modelo STRIDE (*Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege*), un modelo utilizado para la identificación de amenazas durante el diseño de software. Este análisis permitió identificar y documentar las amenazas que podrían afectar el sistema dividiéndolas en seis categorías, y las mitigaciones que se aplicarían a cada una de ellas.

3 Capítulo 3: Diseño y Desarrollo

3.1 Enfoque Metodológico

El enfoque metodológico de este proyecto se basa en dos componentes principales. El primero se enfoca en determinar de manera empírica cual o cuales modelos de predicción de demanda resultan más adecuados para el caso de estudio. El segundo componente se centra en la estructuración de dicho modelo dentro de una herramienta funcional, siguiendo procesos técnicos alineados con la norma ISO/IEC/IEEE 12207.

Para la evaluación de los modelos de predicción, el enfoque adoptado es empírico y por producto, ya que en lugar de seleccionar un único modelo ganador para la totalidad del catálogo de productos, se entrenan y comparan distintos modelos candidatos, y para cada producto, se selecciona de forma individual aquel que presente el menor error de pronóstico durante el periodo de prueba. Esta decisión responde a que no existe un modelo que universalmente pueda adaptarse a todos los patrones de demanda como: productos de consumo estable, esporádico o altamente variable, cada uno de los patrones puede favorecerse de distintas técnicas. Por otra parte, en cuanto a la selección de variables del modelo de Regresión Lineal, este se llevó a cabo siguiendo la misma metodología reportada en la literatura revisada en el Capítulo 2, es decir, teniendo en cuenta factores como la eliminación recursiva de características seguida de eliminación hacia atrás por significancia estadística y factor de inflación de varianza.

Debido a que los datos que fueron empleados corresponden a series de tiempo (registros de consumo mensual), se respeta el orden cronológico de la información. La separación entre los datos de entrenamiento y prueba se realiza por fecha, de manera que, se entrenan con los periodos más antiguos y se evalúa sobre los más recientes. Adicionalmente, se incorpora un procedimiento de validación cruzada que repite el proceso de entrenamiento y evaluación sobre distintas ventanas de tiempo sucesivas, con el fin de confirmar que el desempeño del sistema no dependa de las particularidades de un solo periodo de prueba.

Finalmente, el enfoque también incluye la evaluación mediante el uso de un caso de estudio real, ya que se usaron los datos históricos de consumo del año 2021 de la clínica Nuestra Señora de los Remedios, que constituye la validación del sistema en el dominio de la salud.

3.1.1 Gestión del Proyecto (Planeación)

La planeación del proyecto se estructuró en cuatro fases secuenciales, cada una con un conjunto de entregables y actividades que se detallan a continuación:

- Fase 1 (Fase de fundamentación): En esta fase se plantean los objetivos del proyecto y se realiza la revisión de literatura y construcción del marco de referencia, se definen los criterios de inclusión y exclusión de los estudios consultados y se compara el desempeño reportado por los distintos modelos de predicción de demanda.
- Fase 2 (Fase de análisis y diseño): Se definen los requisitos del sistema, la arquitectura y el modelo de datos
- Fase 3 (Fase de construcción): Se implementa el sistema de predicción, con el módulo de cálculo de puntos de reposición y la interfaz de usuario, en esta fase se desarrolla el sistema identificando limitaciones en los datos o en los modelos, así como limitaciones propias de la arquitectura inicial del sistema, las cuales se fueron corrigiendo a medida que avanzaba el desarrollo y surgían nuevas necesidades y propuestas para satisfacerlas.
- Fase 4 (Fase de evaluación): En la fase final se le aplica al sistema un plan de verificación y validación, que incluye la evaluación de los modelos de predicción de demanda y la validación del sistema en el dominio de la salud, mediante el uso de un caso de estudio real.

3.1.2 Identificación y mitigación de riesgos

Durante la planeación del proyecto se identificaron los siguientes riesgos, junto con las medidas de mitigación adoptadas para cada uno:

TABLA V
Identificación y mitigación de riesgos del proyecto.

ID	Riesgo	Mitigación	Probabilidad	Impacto
R1	Limitación en la calidad de los datos históricos (2021)	Realizar limpieza y validación de datos, aplicar técnicas de preprocesamiento	Media	Alto
R2	Baja precisión de los modelos de predicción	Probar diferentes modelos y ajustar parámetros	Media	Alto
R3	Retrasos en el cronograma del proyecto	Planificación por fases y seguimiento continuo	Baja	Medio
R4	Dependencia de un único conjunto de datos	Documentar limitaciones y considerar ampliación futura	Alta	Medio

Al finalizar el proyecto, los riesgos R1 y R2 se mitigaron mediante las etapas de carga y limpieza de datos (Sección 3.2.9) y la comparación empírica de los tres modelos candidatos (Sección 3.2.13), respectivamente. El riesgo R3 se controló mediante las cuatro fases de planeación descritas anteriormente en esta misma sección. El riesgo R4, por su parte, se materializó parcialmente: el sistema se validó únicamente sobre el dominio de salud, y la validación en otros dominios de inventario se documenta como una línea de trabajo futuro, tal como se había previsto en su mitigación original.

3.1.3 Ciclo de Vida del Software (Ejecución)

La ejecución del proyecto se realizó siguiendo un ciclo de vida iterativo-incremental, en el cual el sistema se construyó en sucesivas versiones del mismo en el que en cada una de estas se iba ampliando o corrigiendo detalles de la anterior a partir de la evaluación de los resultados obtenidos. La elección de esta estrategia se basa en la naturaleza del proyecto, en el que al estar trabajando en el desarrollo de un sistema de predicción, es esperable que decisiones como la forma de representar los meses como variable de entrada, o el conjunto de variables históricas utilizadas por el modelo, no podrían definirse por adelantado, sino que se irían determinando y ajustando a medida que se experimentara y observara el comportamiento del sistema durante sus fases de desarrollo.

Un ejemplo de una de estas decisiones fue el como se llevaría a cabo la persistencia de la información en el sistema. En una primera versión se llevó a cabo mediante archivos estructurados, pero posteriormente se migró hacia una base de datos relacional al requerir un mayor control y almacenamiento de la información, más específicamente, es necesario el poder mantener un historial de auditoria y de corridas anteriores, algo que no se había contemplado por completo desde el inicio

del proyecto.

Otro ejemplo es que durante las pruebas realizadas, se encontró un error en el que el sistema no podía reconocer la columna "precio" en español a pesar de que la interfaz la anunciaba como si hubiese sido aceptada.

De igual forma, la manera en la que iban a incorporarse las pruebas automatizadas en el proyecto también fue un aspecto que no se planeó desde el inicio, sino que fue parte de los detalles que se fueron incorporando al desarrollo del sistema a medida que este avanzaba.

En cuanto al diseño de la arquitectura del sistema, el diseño de la interfaz de usuario, y los mecanismos de control de acceso al sistema, se siguieron los procesos técnicos que indica la norma ISO/IEC/IEEE 12207, que establece un conjunto de procesos a tener en cuenta como la identificación de stakeholders, especificación de requisitos, diseño arquitectónico, los cuales fueron definidos para el sistema.

3.2 Procesos Técnicos y Diseño (ISO/IEC/IEEE 12207)

Los procesos técnicos que guiaron el diseño y desarrollo del sistema Thary se estructuraron siguiendo la ISO/IEC/IEEE 12207, una norma que establece un conjunto de procesos estándar para el ciclo de vida del software. Dichos procesos fueron aplicados al desarrollo del sistema de la siguiente manera:

3.2.1 Identificación de Stakeholders

A partir del planteamiento del sistema y tras un análisis de sus características y funcionalidades, se identificaron los siguientes stakeholders que estarían involucrados en el sistema, ya sea directa o indirectamente:

- Personal encargado de la gestión del inventario y el abastecimiento: Es el usuario final que interactúa con el sistema, es responsable de interpretar los resultados que Thary genera y tomar decisiones de reposición en base a ellas.
- Equipo de desarrollo: Desarrolladores y diseñadores responsables de la implementación del sistema Thary, asegurando que cumpla con los requisitos y estándares establecidos.
- Administradores del sistema: Es el perfil con permisos y privilegios, es responsable de

cargar los datos históricos de consumo, es decir, el archivo de entrada, ejecutar el proceso de predicción, además, puede crear nuevos usuarios para los empleados con menor rango.

- **Institucion usuaria del sistema:** Es la entidad interesada en utilizar Thary para poder reducir el riesgo de desabastecimiento de productos. El sistema se planeo para poder adaptarse en distintos dominios, por lo que cualquier tipo de empresa puede usarlo.
- **Proveedores de productos:** Aunque los proveedores no interactuan con Thary, indirectamente son una variable importante para el sistema, ya que factores externos como el tiempo de entrega o lead time, marcan una gran diferencia en los resultados del sistema.

TABLA VI
Análisis de interés e influencia de los stakeholders

Stakeholder	Interés	Influencia
Personal de inventario y abastecimiento	Alto	Media
Administrador del sistema	Alto	Alta
Institución usuaria	Alto	Alta
Proveedores de productos	Medio	Media
Equipo de desarrollo	Alto	Alta

3.2.2 Elicitación de Requisitos

La elicitación de requisitos consiste en poder identificar las necesidades del sistema a partir de los usuarios, en el caso de Thary, el proceso se llevó a cabo mediante dos fuentes principales de información, en primera instancia, la revisión de literatura permitió analizar que variables suelen ser relevantes para este tipo de proyectos, y que salidas le aportan mayor valor al usuario. En segundo lugar, se realizo un análisis a un caso de estudio real, los datos usados durante el desarrollo del sistema eran datos auténticos de la Clínica Nuestra señora de los Remedios de la ciudad de Cali, lo cual permitió observar como se comportaría el sistema en un escenario real como la existencia de variaciones en la demanda o datos históricos no tan limpios como historiales incompletos o datos con errores, y que ajustes serían necesarios para mejorar su rendimiento.

Es en base a todo esto, que se definieron los siguientes requisitos funcionales:

- RF-01: El sistema debe permitir que un usuario registrado como admin pueda realizar la carga de un archivo CSV con datos como el historial de consumo mensual de los productos, validando que contenga los datos mínimos requeridos: fecha, id del producto y cantidad demandada.
- RF-02: El sistema debe generar una estimación de la demanda mensual, esperada para cada producto a partir de la información compartida en el historial cargado
- RF-03: El sistema debe de calcular para cada producto datos como el punto de reposición y la cantidad recomendada de compra, considerando factores externos como el tiempo de entrega del proveedor o lead time, presupuestos y el inventario actual disponible
- RF-04: El sistema debe de contar con alertas visuales claras para los productos cuyo inventario actual se encuentre en o por debajo de su punto de reposición.
- RF-05: El sistema debe presentar los resultados, demanda pronosticada, estado del inventario, etc, de una manera clara y sencilla de entender en una interfaz intuitiva para el usuario
- RF-06: El sistema debe permitirle al personal del inventario registrar un pedido, guardando información importante de este como proveedor elegido, fecha del pedido y el usuario responsable de la realización del pedido.
- RF-07: El sistema debe restringir el acceso a la carga de los archivos y la ejecución del proceso únicamente al perfil administrador.
- RF-08: El sistema debe permitirle al perfil de administrador el poder crear nuevos usuarios con menos privilegios para sus empleados.
- RF-09: El sistema debe permitir el registro e inicio de sesión de los usuarios.
- RF-10: El sistema debe permitir deshacer un pedido previamente confirmado.
- RF-11: El sistema debe registrar y permitirle al administrador consultar una bitácora de las acciones relevantes realizadas por los usuarios.
- RF-12: El sistema debe permitirle a los usuarios ver un calendario con las fechas en las que tienen que realizarse los pedidos de cada producto
- RF-13: El sistema debe permitirle a los usuarios acceder a su perfil.

En cuanto a los requisitos no funcionales de Thary, se identificaron los siguientes:

- RNF-01: Si el sistema es utilizado por más de una organización, los datos de una no deben de ser accesibles para otra
- RNF-02: El sistema debe de poder procesar archivos de distintos dominios de inventario (salud, alimentos, ferretería, etc) siempre y cuando contengan los datos mínimos requeridos.
- RNF-03: El sistema debe poder generar una estimación incluso si el historial de producto presenta datos faltantes.

3.2.3 Especificación de Requisitos de Software (SRS)

A partir de los requisitos especificados, se elaboró la especificación de requisitos funcionales (SRP), la cual fue estructurada mediante casos de uso y tomando como referencia los lineamientos de la norma IEEE 29148 para la elaboración de los mismos. El sistema contempla dos actores principales, el administrados y el personal de inventario. El administrador es el tipo de usuario que más privilegios tiene, es responsable de cargar el historial de consumo e iniciar el procesamiento y gestionar las cuentas de los usuarios. Mientras que el personal de inventario solo usa el sistema para consultar las predicciones y, entre otras actividades, gestionar los pedidos sugeridos. La figura debajo representa los casos de uso correspondientes al Administrador:

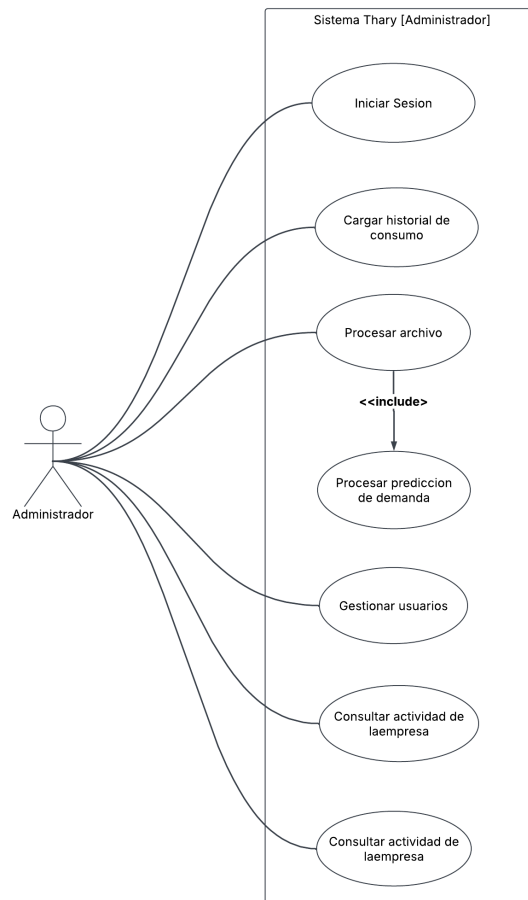


Fig. 3. Casos de uso del Administrador

Por su parte, la siguiente figura muestra los casos de uso correspondientes al Personal de Inventario:

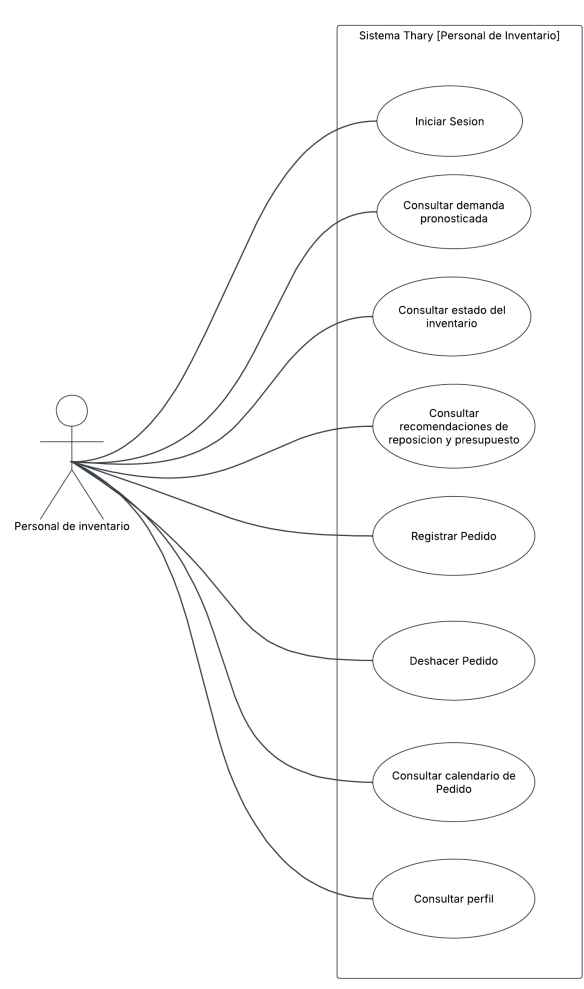


Fig. 4. Casos de uso del Personal de Inventario

Como se puede ver en los diagramas, los casos de uso identificados se relacionan con los requisitos funcionales de la siguiente manera:

TABLA VII
Trazabilidad entre casos de uso y requisitos funcionales

ID	Caso de uso	Actor principal	Requisitos relacionados
CU-01	Iniciar sesión	Usuario registrado	RF-09
CU-02	Cargar historial de consumo	Administrador	RF-01, RF-07
CU-03	Procesar archivo	Administrador	RF-02, RF-07
CU-04	Consultar demanda pronosticada	Personal de Inventario	RF-02, RF-05
CU-05	Consultar estado del inventario	Personal de Inventario	RF-04, RF-05
CU-06	Consultar recomendaciones de reposición y presupuesto	Personal de Inventario	RF-03, RF-05
CU-07	Registrar pedido	Personal de Inventario	RF-06
CU-08	Gestionar usuarios	Administrador	RF-08
CU-09	Deshacer pedido	Personal de Inventario	RF-10
CU-10	Consultar actividad de la empresa	Administrador	RF-11
CU-11	Consultar calendario de pedidos	Personal de Inventario	RF-12
CU-12	Consultar perfil	Usuario registrado	RF-13

- Caso de uso CU-01 (Iniciar sesión (Usuario registrado)): Este caso de uso corresponde al RF-09, y le permite a cualquier usuario registrado ingresar al sistema mediante sus credenciales para acceder a las funcionalidades según su rol.
- Caso de uso CU-02 (Cargar historial de consumo (administrador)): Este caso de uso corresponde al RF-01, y permite al administrador seleccionar y cargar el archivo csv con el historial de consumo.
- Caso de uso CU-03 (Iniciar procesamiento(Administrador)): Corresponde al requerimiento RF-02, que hace referencia a que una vez cargado el archivo, el administrador debe de seleccionar la opción "Procesar archivo" para que el sistema inicialice el procesamiento de los datos.
- Caso de uso CU-04 (Consultar demanda pronosticada (Personal de inventario)): Este caso de uso corresponde al RF-02 y al RF-05, y le permite al usuario consultar la demanda mensual estimada para cada producto una vez el archivo ya ha sido procesado.

- Caso de uso CU-05 (Consultar estado del inventario (Personal de inventario)): Este caso de uso corresponde a RF-04, que permite al usuario visualizar tanto el inventario disponible como los productos que presentan condiciones de alerta.
- Caso de uso CU-06 (Consultar las recomendaciones de reposición y presupuesto (Personal del inventario)): Corresponde al requisito RF-03, y le permite al usuario consultar las recomendaciones generadas por el sistema y la información asociada al presupuesto disponible
- Caso de uso CU-07 (Registrar pedido (Personal de inventario)): Este se relaciona directamente con el RF-06, y permite marcar como completado las recomendaciones de pedido generadas por Thary. Los pedidos confirmados quedan registrados con el nombre de la persona que los completa.
- Caso de uso CU-08 (Gestionar cuentas de usuario(administrador)): Le permite al administrador el poder crear nuevas cuentas al personal del inventario.
- Caso de uso CU-09 (Deshacer pedido (Personal de inventario)): Corresponde al RF-10, y le permite al personal de inventario revertir un pedido previamente confirmado, en caso de haberse equivocado o de que la confirmación ya no sea necesaria.
- Caso de uso CU-10 (Consultar actividad de la empresa (Administrador)): Este caso de uso corresponde al RF-11, y le permite al administrador consultar la bitacora completa de acciones realizadas por todos los usuarios de su empresa: carga de archivos, confirmación o reversión de pedidos, creación de usuarios, con fines de auditoria.
- Caso de uso CU-11 (Consultar calendario de pedidos (Personal de inventario)): Se relaciona con el RF-12, y le permite al usuario visualizar en un calendario las fechas estimadas en las que debe realizarse el pedido de cada producto, según las recomendaciones generadas por el sistema.
- Caso de uso CU-12 (Consultar perfil (Usuario registrado)): Este caso de uso corresponde al RF-13, y le permite a cualquier usuario, sin importar su rol, poder consultar los datos de su propia cuenta y el historial de las acciones que el mismo ha realizado en el sistema.

Todos los casos de uso se documentan individualmente en la especificación SRS presentada en el Anexo A, la cual contempla detalles como objetivo, actor principal, precondiciones y postcondiciones, flujo principales y alternativos.

En cuanto a elementos como las precondiciones, estos se establecen antes de iniciar una funcionalidad, mientras que las postcondiciones describen cual es el estado esperado del sistema al finalizar el proceso, es de esta manera que realizar una correcta especificación de casos permite establecer una base a la hora de continuar con el proceso de diseño de arquitectura e implementación de servicios.

3.2.4 Diseño de la Arquitectura

El sistema Thary fue diseñado basandose en una organización pipeline para el componente de predicción y un estilo arquitectónico en capas de las cuales se distinguen tres principales:

- **Capa de Presentación:** Esta capa es la responsable de funcionalidades como carga de archivos, visualización de resultados, registro de pedidos y demas funcionalidades relacionadas con toda la interacción que el usuario final va a tener con el sistema.
- **Capa de Control:** Esta capa es la encargada de recibir todas las solicitudes que el usuario le haga a la aplicación, y devolver los resultados esperados a la capa de presentación
- **Capa de servicios de dominio:** En esta capa se encuentra el componente de predicción organizado como un pipeline secuencial compuesto por las diferentes etapas del sistema, en general, en esta capa se concentra toda la lógica de Thary.

En cuanto a porque se decidió llevar la organización mediante un pipeline, esta decisión se tomó debido a que el proceso de predicción de demanda, por su naturaleza, corresponde a un flujo secuencial de transformación de los datos, por lo que el uso de esta estructura facilita que se pueda separar responsabilidades de una manera más sencilla y la trazabilidad de los resultados obtenidos.

Por otro lado, a la hora de elegir como se manejaría la persistencia de la información, se optó por usar un modelo de base de datos relacional (MySQL). Esta decisión se tomó considerando la necesidad de poder contar con un historial con las corridas anteriores del sistema. Adicional a esto, la arquitectura de Thary contempla el aislamiento de los datos para cada organización, de manera que todas cuentan con un espacio independiente evitando que puedan acceder a información de otro usuario.

3.2.5 Escenarios Arquitecturales

Para garantizar que Thary cuenta con una arquitectura que es capaz de responder a los atributos de calidad necesarios para un proyecto con sus características, se plantearon los siguientes

escenarios:

- Escenario 1 (Confiabilidad ante datos escasos): Este escenario hace referencia a que cuando un usuario cargue el historial de un producto con menos de cuatro meses de registros, Thary no debe fallar ni excluir el producto, en su lugar, debe utilizar un método de estimación de respaldo, como la media móvil. Esto le aporta valor al sistema cuando no exista suficiente información histórica para usar un modelo más complejo.
- Escenario 2 (Desempeño ante catálogos extensos): Cuando se procesa un archivo con un número alto de productos diferentes, los cálculos de variables como puntos de reposición y cantidades sugeridas deben poder completarse de manera eficiente, sin que el tiempo de procesamiento se degrade de forma desproporcionada al tamaño del catálogo.
- Escenario 3 (Confidencialidad de la información): En este escenario se tiene en cuenta que Thary debe garantizar que los datos relacionados con el inventario y el historial de consumo de la organización, no se transmitan a ningún servicio externo del que Thary se encuentra alojado.
- Escenario 4 (Aislamiento entre organizaciones): Al trabajar con tantos usuarios/administradores, lo más probable es que en algún momento, al menos dos organizaciones distintas usen el sistema de manera simultánea, Thary debe asegurarse que ningún tipo de dato quede expuesto ni pueda ser modificado por otra organización.
- Escenario 5 (Adaptabilidad del formato de datos): Este escenario se refiere a que cuando se cargue un archivo que pertenezca a un dominio diferente al caso de estudio principal, el cual fue salud, Thary tiene que ser capaz de reconocer y normalizar las columnas sin tener que modificar el código.

Estos escenarios permiten poder verificar que las decisiones arquitectónicas tomadas respondan a las necesidades que tiene el sistema, facilitando que posteriormente se pueda comprobar el funcionamiento de Thary.

3.2.6 Análisis de Alternativas y Decisiones de Diseño (ADRs)

Las decisiones de diseño que fueron adoptadas durante todo el desarrollo del sistema Thary y las alternativas consideradas.

- Decisión 1 (Persistencia basada en base de datos relacional): Debido a que el sistema va a trabajar con información relacionada con los usuarios y los historiales de consumo que

compartan con el sistema, se tuvo que seleccionar la manera más adecuada de almacenar dicha información. En una primera versión de la producción del sistema, se decidió que se utilizaran archivos estructurados independientes para cada la información de cada organización. Sin embargo, una vez se quiso mantener un registro de eventos como historial de corridas o de actividades, se migró hacia un motor de base de datos relacional MySQL. La elección de esta alternativa además de ofrecer dichas posibilidades, permite una mayor integridad de los datos y mejora la capacidad de consulta a cambio de una mayor complejidad.

- Decisión 2 (Selección del modelo de predicción por producto): Existen distintos productos del inventario que pueden presentar patrones de demanda diferentes entre si, teniendo esto en cuenta, al inicio del desarrollo se propusieron dos alternativas, la primera era generalizar para todo el sistema el uso de un mismo modelo, y la segunda implementar tres modelos distintos. Se optó que la mejor opción es entrenar diferentes modelos candidatos (XGBoost, regresión lineal, y media móvil), comparar los resultados de los 3 y dependiendo de si el consumo es estable, esporádico o muy variable, seleccionar el que presente el menor error de predicción, esto basandose en el cálculo del MAE (Error absoluto medio). La justificación del porqué se eligió esta métrica es que el MAE es una medida que se expresa en las mismas unidades que la demanda, lo que facilita que se pueda interpretar más fácilmente.
- Decisión 3 (Validación temporal y validación cruzada): A la hora de validar los resultados de Thary, los cuales son conseguidos a partir de series de tiempo (historial de consumo mensual), se pensó en dos propuestas, la primera se basaba en hacer una validación cruzada aleatoria, que supone hacer una partición sin considerar el orden cronológico, mientras que la segunda se basaba en una validación cruzada temporal de origen móvil, en el que se generan varias particiones de entrenamiento y pruebas consecutivas. Se decidió optar por la segunda opción, ya que la primera terminaría mezclando datos del futuro, lo cual se quiere evitar ya que la idea es representar un escenario real de uso del sistema, en el que no se tendrían estos datos del futuro. Es por esto que a pesar de que repetir el entrenamiento sobre varias particiones en lugar de una sola representa un mayor costo computacional, se eligió porque resulta más representativa al desempeño real que se espera que tenga el sistema.

3.2.7 Representación de la Arquitectura (Modelo C4)

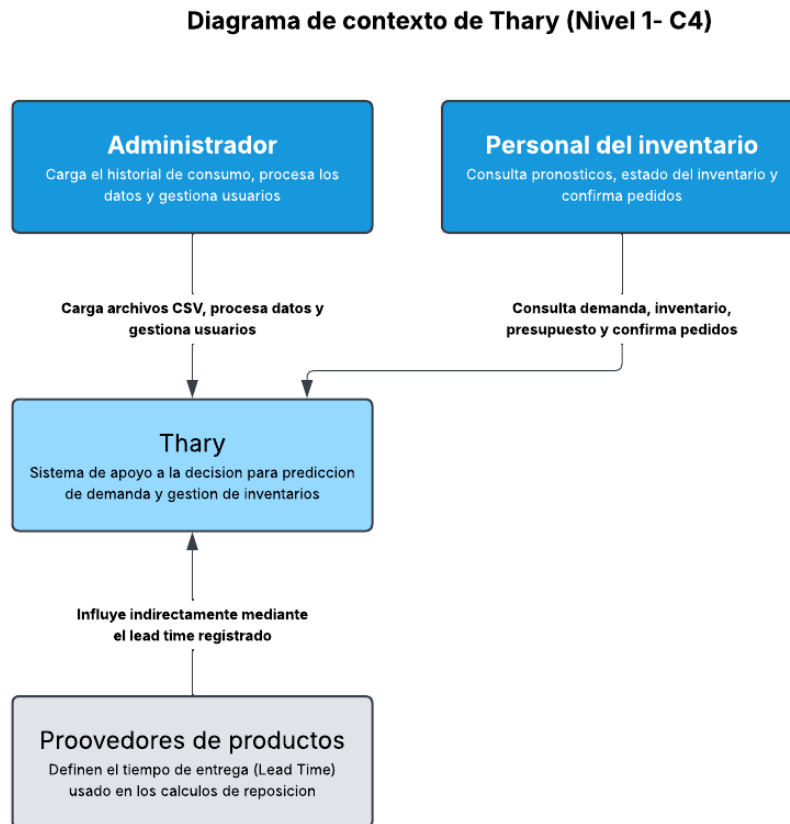


Fig. 5. Diagrama de contexto del sistema (nivel 1 del modelo C4).

Como se observa en la Fig. 5, el sistema interactúa con los actores identificados en la sección 3.2.1.

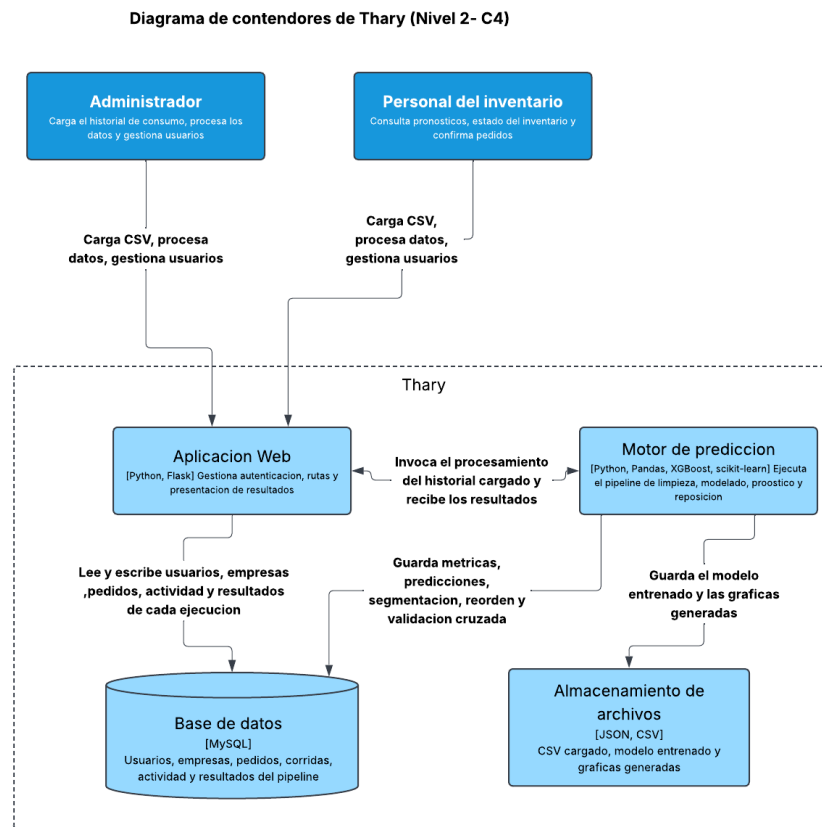


Fig. 6. Diagrama de contenedores del sistema (nivel 2 del modelo C4).

En la figura 6 se puede observar como se organizaron los contenedores del sistema Thary, factores como la aplicación web, el motor de predicción y la base de datos son contenedores independientes que se comunican entre si para el correcto funcionamiento del sistema.

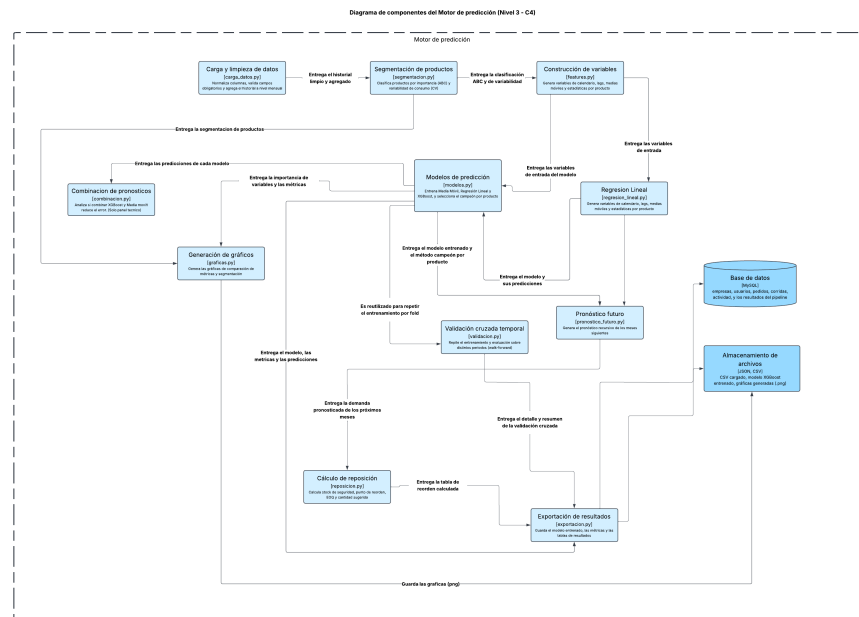


Fig. 7. Diagrama de componentes del sistema (nivel 3 del modelo C4).

Finalmente, el diagrama de componentes detalla a profundidad el funcionamiento interno del motor de predicciones.

3.2.8 Diseño Detallado

A continuación, se describe el diseño detallado del sistema Thary, el cual consiste en las siguientes etapas del pipeline:

3.2.9 Etapa 1: Carga y limpieza de datos

La primera etapa del pipeline (`app/services/prediccion/carga_datos.py`) recibe el archivo CSV cargado por el administrador y lo normaliza hacia un esquema interno único:

Código 1. Función cargar_y_limpiar y etiquetas de las columnas obligatorias.

```

ETIQUETAS_COLUMNS_OBLIGATORIAS = {
    "fecha": "Fecha (o Date, Sale_Date)",
    "producto_id": "Código (o Code, Id, Product_ID)",
    "demanda": "Cantidad (o Quantity, Demanda, Demand)",
}

SINONIMOS_COLUMNS = {
    "fecha": ["Fecha", "Date", "Sale_Date"],
    "producto_id": ["Product_ID", "Codigo", "Code", "Id"],
    "demanda": ["Demanda", "Demand", "Quantity_Sold", "Sales", "Cantidad"],
    # ... variantes adicionales para precio, inventario, categoria, pr
}

NOMBRES_PRODUCTO_AMBIGUOS = ["Producto", "Product"]

def _normalizar_encabezado(nombre):
    """Minúsculas, sin tildes, con espacios/guiones como "_"."""
    texto = unicodedata.normalize("NFKD", str(nombre)).encode("ascii",
    return re.sub(r"[\s\~]+", "_", texto.strip().lower())

def _renombrar_columnas(df):
    """Renombra las columnas del archivo a los nombres internos. Si el
    archivo trae "Producto"/"Product" y ademas ya tiene una columna de
    codigo clara (Codigo, Code, Id, Product_ID), esa columna ambigua s
    toma como el NOMBRE del producto en vez de su identificador."""
    # ... logica de resolucion por prioridad y ambigüedad (ver código)

def cargar_y_limpiar(ruta_archivo):
    try:
        df = pd.read_csv(ruta_archivo)
    except pd.errors.EmptyDataError:
        raise ValueError("El archivo está vacío. Sube un CSV con al men

```

```

except pd.errors.ParserError:
    raise ValueError(
        "No se pudo leer el archivo como CSV. Verifica que sea un a
        "separado por comas (por ejemplo, no un Excel renombrado a

    )
except UnicodeDecodeError:
    raise ValueError(
        "El archivo tiene una codificación de caracteres que el sis
        "Guárdalo como CSV con codificación UTF-8 e intenta de nuev

    )

if df.empty:
    raise ValueError("El archivo no tiene filas de datos, solo encab

df = _renombrar_columnas(df)

columnas_faltantes = [c for c in ETIQUETAS_COLUMNAS_OBLIGATORIAS i
if columnas_faltantes:
    faltan = ", ".join(ETIQUETAS_COLUMNAS_OBLIGATORIAS[c] for c in
    raise ValueError(
        f"Al archivo le falta al menos una columna obligatoria: {f
        "Agrégala al CSV (con ese nombre exacto u otro de los acep

    )

```

En esta sección del archivo (`app/services/prediccion/carga_datos.py`), se observa la etapa de carga y limpieza de los datos. Para garantizar que el sistema pueda adaptarse y funcionar para todo tipo de dominio de inventarios, se normalizan los nombres de las columnas de los archivos CSV con el fin de mantener un esquema único interno, de manera que el sistema pueda aceptar archivos de distintos dominios sin tener la necesidad de modificar el resto del pipeline.

Para facilitarle al usuario el proceso de cargar un archivo, se capturan los errores más comunes para el sistema a la hora de leer un CSV y se muestran a manera de mensaje de error en un lenguaje sencillo y fácil de entender.

Otra práctica que fue aplicada es el manejo de errores con mensajes dirigidos al usuario final, de esta manera, se evita que se propaguen los errores internos del sistema y el usuario sabe qué es lo que está ocurriendo y qué es lo que se debe hacer para resolverlo.

3.2.10 Etapa 2: Segmentación de productos

La segunda etapa del pipeline (`app/services/prediccion/segmentacion.py`) se basa en poder clasificar cada producto según dos criterios principales:

- **Clasificación ABC:** Esta clasificación permite poder identificar cual es la importancia relativa de cada producto dentro de la demanda total. Se calcula ordenando los productos de mayor a menor en base a su demanda total y se guarda su porcentaje.

Los productos que en conjunto representan hasta el 80% de la demanda, se clasifican como categoría A, es decir, los más importantes. Los que se encuentran entre el 80% y el 95% se clasifican como categoría B. El resto de productos se clasifican como categoría C.

Esta clasificación permite que en el dashbpard, el usuario pueda entender que tan importante y que tan errático es el consumo de cada producto

- **Variabilidad:** La variabilidad del consumo de los productos permite poder identificar cuales son los productos que presentan un comportamiento más estable y cuales son los que presentan un comportamiento más irregular, se calcula en base al coeficiente de variación (la desviación estándar dividida entre la media de la demanda), clasificando a cada producto como de variabilidad Baja (menor a 0.5), Media (entre 0.5 y 1) o Alta (mayor o igual a 1).

Código 2. Función `calcular_comportamiento_producto`: variabilidad y clasificación ABC de cada producto.

```
def calcular_comportamiento_producto(df, df_mensual):
    producto_comportamiento = (
        df_mensual.groupby("producto_id")["demanda"]
        .agg(demanda_total="sum", media="mean", desviacion="std")
        .reset_index()
    )
    producto_comportamiento["cv"] = (
        producto_comportamiento["desviacion"] / producto_comportamiento["media"]
    ).replace([np.inf, -np.inf], np.nan)

def clasificar_variabilidad(cv):
    if pd.isna(cv):
        return "Sin datos"
    if cv < 0.5:
        return "Baja"
```

```

        elif cv < 1:
            return "Media"
        else:
            return "Alta"

producto_comportamiento["variabilidad"] = producto_comportamiento["variabilidad"]

producto_comportamiento = producto_comportamiento.sort_values("demanda_total")
total_demanda = producto_comportamiento["demanda_total"].sum()
if total_demanda > 0:
    producto_comportamiento["demanda_acumulada_pct"] = producto_comportamiento["demanda_acumulada_pct"]
else:
    producto_comportamiento["demanda_acumulada_pct"] = 0

def clasificar_abc(pct):
    if pct <= 0.80:
        return "A"
    elif pct <= 0.95:
        return "B"
    else:
        return "C"

producto_comportamiento["categoria_abc"] = producto_comportamiento["categoria_abc"]

# ...
return producto_comportamiento

```

Es en esta etapa en la que se calculan estadísticas históricas como (media_producto y desviacion_producto).

3.2.11 Etapa 3: Construcción de variables

En esta etapa, (app/services/prediccion/features.py), se adapta el historial de demanda a un formato que contenga las variables necesarias que los modelos usan como entrada (*features*):

- Componentes de calendario
- Rezagos de la demanda

- Estadísticas móviles
- Estadísticas históricas por producto

La implementación de esta etapa empieza con un merge inicial, en este merge se trae las categorías abc y variabilidad a la tabla. Aunque no se usan como variables de entrada (`feature_cols`), se incluyen porque posteriormente se necesitaran para mostrarlas en el dashboard y tabla de predicciones:

Código 3. Unión de las categorías ABC y de variabilidad a la tabla mensual.

```
def construir_features(df_mensual, producto_comportamiento):
    # ...
    df_mensual = df_mensual.merge(
        producto_comportamiento[["producto_id", "categoria_abc", "variabilidad"],
                                on="producto_id", how="left"
    )
```

Posteriormente, se extraen tanto el año como el mes de cada ejemplo, esto adaptado del repositorio *Demand Forecasting & Inventory Optimization for Consumer Products* [33], al que en este trabajo se le referirá como *Ejemplo 1 (SKU Demand Forecasting)*. Con la diferencia de que en Thary, estos datos se incluyeron con el fin de en la implementación que se tomó como guía, el mes "month", queda como un feature final para el modelo, mientras que en Thary, este dato posteriormente sea transformado en `mes_sin` y `mes_cos`, proceso que se explicará más adelante:

Código 4. Extracción del año y del mes de cada registro.

```
df_mensual["año"] = df_mensual["fecha"].dt.year
df_mensual["mes"] = df_mensual["fecha"].dt.month
```

Un rezago (lag), se usa en Machine Learning y hace referencia al valor anterior a una variable, en este caso, representa el valor de la demanda del mes anterior, el cual es usado como dato de entrada para predecir el mes actual. Los lags junto con las medias y desviaciones móviles se encargan de capturar el comportamiento reciente de la demanda de cada producto. En Thary, para cada producto se tiene:

- lag_1: demanda del mes anterior
- lag_2: demanda de hace dos meses

- lag_3: demanda de hace tres meses

El modelo los usa como features porque la demanda pasada ayuda a poder determinar cuanto se va a vender el proximo mes. Por eso es que un producto necesita mínimo 3 meses de historial antes de los primeros 3 meses del dataset para poder calcular lag1, lag2 y lag3. El problema era que el sistema solo conoce los datos que vienen dentro del mismo archivo CSV que el usuario sube, por lo que no tiene forma de saber que paso antes del primer mes que aparece en el archivo.

Si un dataset empieza en enero de 2021, el lag1 de enero necesitaría la demanda de diciembre de 2020, dato que no esta en el historial dado por el usuario, solo seria a partir de abril 2021 que el sistema contaría con los 3 lags comparados correctamente. Es debido a que no hay información suficiente para calcular los lags correctamente, que el sistema descarta los primeros meses a la hora de entrenar el modelo mediante (`dropna()`). Como esos primeros 3 meses se descartan, el valor de mes de esos meses nunca aparece como algo que el modelo tuvo que predecir durante el entrenamiento, por lo que estaría el riesgo de que, si más adelante se le pide al modelo predecir justo ese mismo mes del año siguiente (por ejemplo, enero 2022), el modelo no tenga ninguna referencia previa de que número de mes es ese, y termine adivinando mal, generando predicciones muy bajas o poco confiables para ese mes en particular

Para evitar este problema, se decidió que durante esta etapa de features, se representaría el mes utilizando seno y coseno en vez de usar directamente el número del mes (1-12). Esto permite que el modelo interprete el mes como un punto continuo cercano a los meses vecinos que si fueron entrenados, por ejemplo, diciembre y enero quedan numericamente uno cerca del otro, lo cual tiene mucho más sentido que hacer que el sistema los vea como el número 12 y 1, valores mucho más lejanos de lo que deberían ser para dos meses vecinos.

Una vez que el modelo entienda la cercanía real de los meses, es capaz de dejar de interpretarlos como categorías independientes, y puede generar una predicción razonable de un mes que nunca halla visto antes (enero, por ejemplo), en base la información de un mes cercano (diciembre).

A la hora de armar la función para construir los features se tomaron en cuenta diferentes implementaciones previas de distintos estudios con diferentes propuestas para un sistema de predicción de demanda, se compararon y se seleccionaron aquellas que se ajustaban mejor a las necesidades del sistema y le aportaban valor al mismo, junto con la implementación de nuevas ideas y mejoras que se fueron propuestas e implementadas en Thary. Una de estas propuestas nuevas fue la codificación ciclica del mes, que como ya se explicó antes, consiste en representar los meses mediante dos valores calculados con seno y coseno, como se puede ver a continuación:

Código 5. Codificación cíclica del mes mediante seno y coseno.

```
# ...
df_mensual["mes_sin"] = np.sin(2 * np.pi * df_mensual["mes"] / 12)
df_mensual["mes_cos"] = np.cos(2 * np.pi * df_mensual["mes"] / 12)
```

Por otra parte, la metodología de implementar el uso de lags, también se tomó del *Ejemplo 1 (SKU Demand Forecasting)* [33] y se adaptó a las necesidades del sistema Thary, agregando la codificación cíclica de los meses y la inclusión de features adicionales como el inventario y el precio.

Código 6. Cálculo de los rezagos lag_1, lag_2, lag_3 y lag_6 por producto.

```
# ...
# (Ejemplo 1 - SKU Demand Forecasting)
df_mensual["lag_1"] = df_mensual.groupby("producto_id")["demanda"]
df_mensual["lag_2"] = df_mensual.groupby("producto_id")["demanda"]
df_mensual["lag_3"] = df_mensual.groupby("producto_id")["demanda"]
df_mensual["lag_6"] = df_mensual.groupby("producto_id")["demanda"]
```

Otra estrategia que se tomó del *Ejemplo 1 (SKU Demand Forecasting)* [33], fue el uso de las estadísticas móviles. Estas aportan al sistema ya que consisten en un resumen de varios meses juntos. Mientras que los rezagos solo representan cuanto se vendió exactamente en un mes, las estadísticas móviles son un resumen de varios meses juntos.

- `media_movil_3` y `media_movil_6`: Estas variables representan cual es el nivel reciente de la demanda de los productos calculando el promedio de los últimos 3 y 6 meses. Esto ayuda que el modelo pueda distinguir un producto que vende de manera estable de uno irregular, ya que de ser ventas regulares, la media móvil de 3 meses será muy similar a la de 6 meses, mientras que si es inestable, ambas medias serán muy diferentes.
- `desviacion_movil_3`: Representa que tan errática fue la demanda reciente, es decir, de los últimos 3 meses. Los productos que cuentan con demanda estable tendrán una desviación baja mientras que los productos con ventas inestable tendrán una desviación alta.

En general, el modelo puede ver el comportamiento general reciente de los productos más allá de solo los lags.

En cuanto a la implementación, se incluyó en la función de (`construir_features`), en el que cada línea calcula por producto (`groupby("producto_id")`) una estadística sobre una ventana de meses anteriores (`.shift(1)`) para no incluir el mes actual, evitando fuga de datos.

Código 7. Cálculo de las estadísticas móviles `media_movil_3`, `media_movil_6` y `desviacion_movil_3`.

```
# ...
# (Ejemplo 1 - SKU Demand Forecasting)

df_mensual["media_movil_3"] = df_mensual.groupby("producto_id")["d
    lambda x: x.shift(1).rolling(3).mean()
)
df_mensual["media_movil_6"] = df_mensual.groupby("producto_id")["d
    lambda x: x.shift(1).rolling(6).mean()
)
df_mensual["desviacion_movil_3"] = df_mensual.groupby("producto_id
    lambda x: x.shift(1).rolling(3).std()
)
```

Por otra parte, para la construcción de la etapa de Features también se tomó en cuenta el repositorio en Kaggle *Forecasting Inventory Demand Using XGBoost* [35] o *Ejemplo 3 (XGBoost)* [35]. De este repositorio, se tomó el uso de el promedio del producto, pero esta vez, calculados en base a todo el historial completo y no solo en base a los últimos 3 o 6 meses.

Como extensión propuesta por el proyecto, junto al promedio también se incluyó la desviación del producto, esto para poder seguir una misma estructura en todo el sistema, ya que si se tiene la (`media_movil_3`) y la (`desviacion_movil_3`), es decir, el nivel y la variabilidad de los últimos 3 meses, es conveniente tener la media y desviación de todo el historial del producto.

Estos valores ya fueron calculados en la etapa de segmentación, y simplemente son agregados en los features para que el modelo también los use como variables de entrada.

Código 8. Incorporación de la media y la desviación históricas de cada producto.

```
# ...
# (Forecasting Inventory Demand Using XGBoost)
stats_cols = ["demanda_total", "media", "desviacion"]
df_mensual = df_mensual.merge(
    producto_comportamiento[["producto_id"] + stats_cols].rename(
        columns={"media": "media_producto", "desviacion": "desviac
    ),
    on="producto_id", how="left"
)
```

Después, se calcula cuanto stock había el mes anterior. La condición (`if "inventario" in df_mensual.columns:`) determina si el csv que fue subido por el usuario contiene la columna "inventario", esta es una columna opcional que simplemente representa cuantas unidades de ese producto (stock disponible) había físicamente en el inventario en ese mes, Thary funciona sin ella, pero si el usuario la incluye, el modelo la usará como una variable de entrada más para poder brindarle un análisis mucho más detallado.

(`inventario_lag_1`) es el inventario del mes anterior, con (`.groupby("producto_id")`) se asegura que este se calcule por separado para cada producto, esto con la intención de evitar que el inventario de un producto se vaya a mezclar con otro, esto usando (`shift(1)`) para poder obtener el inventario del mes anterior. Si no se hace esto, el modelo podría aprender que el inventario de un producto es igual al del mes actual, lo cual no es correcto y generaría predicciones erróneas.

Código 9. Cálculo del inventario del mes anterior (`inventario_lag_1`).

```
# ...
# (Forecasting Inventory Demand Using XGBoost))
if "inventario" in df_mensual.columns:
    df_mensual["inventario_lag_1"] = df_mensual.groupby("producto_id")
```

En esta parte, se implementó otra idea del *Ejemplo 1 (SKU Demand Forecasting)* [33] para poder descartar las filas sin historial suficiente para tener los lags. Para esto, se define una lista con nueve nombres de las columnas obligatorias para poder entrenar, con (`dropna(subset=feature_history)`) se revisan las 4 columnas y elimina cualquier fila que tenga un espacio vacío, posteriormente el resultado se guarda en (`df_model`), que es la tabla final que se usará para entrenar el modelo.

Código 10. Descarte de las filas sin historial suficiente para calcular los rezagos.

```
# ...
# (Inventory Demand Forecast (XGBoost Reg))
feature_history = ["lag_1", "lag_2", "lag_3", "media_movil_3"]
df_model = df_mensual.dropna(subset=feature_history).copy()

if len(df_model) == 0:
    raise ValueError(
        "Ningún producto tiene al menos 3 meses seguidos de historial
        para calcular tendencias. Sube un archivo con más meses de datos."
    )
```

Otro de los conceptos que fueron propuestos en Thary es el tratamiento que se decidió darle a la variable con los precios de venta de los productos. Aunque el CSV pueda incluir esa variable, el modelo solo usa su valor actual (el precio de ese mes), sin calcular cómo cambió respecto al mes anterior. Esto debido a que por naturaleza, el precio de un producto puede influir en la demanda de un producto, ya que cuando algo baja de precio, la gente tiende a comprar más, mientras que cuando este sube, las personas compran menos. El problema es que si esta regla se repite muy seguido, se corre el riesgo de que el modelo aprenda que el cambio de precio predice muy bien la demanda, entonces, aunque el precio es una variable que si se tiene en cuenta, se excluye por completo su variabilidad de un mes a otro.

Código 11. Lista de variables de entrada (`feature_cols`) y variables opcionales de precio e inventario.

```
# ...
feature_cols = [
    "año", "mes_sin", "mes_cos",
    "lag_1", "lag_2", "lag_3", "lag_6",
    "media_movil_3", "media_movil_6", "desviacion_movil_3",
    "media_producto", "desviacion_producto"
]
if "precio" in df_model.columns:
    feature_cols.append("precio")
if "inventario" in df_model.columns:
    feature_cols.append("inventario")
    if "inventario_lag_1" in df_model.columns:
        feature_cols.append("inventario_lag_1")
```

Finalmente, como extensión propia de Thary, se hace un relleno final. Para esto, se recorre cada una de las columnas que van a usar los modelos, si hay alguna fila vacía en la tabla, se reemplazan los vacíos de la columna con 0 mediante (`df_model[col] = df_model[col].fillna(0)`). Esto es importante porque si el modelo recibe un producto que solo aparece a partir de cierto mes, por ejemplo, abril 2021 en un dataset que empieza en enero de ese mismo año, el mes de abril pudo pasar el filtro de `dropna()` porque ya tiene (`lag_1`, `lag_2`, `lag_3` y `media_movil_3` completos), pero en este caso, el `lag_6` necesitaría la demanda de octubre de 2020, mes que no existe en el archivo porque el producto todavía no tiene 6 meses de historial.

La fila pasaría el filtro porque `lag_6` no está en `feature_history`, pero inevitablemente terminará en `feature_cols`, y ahí es donde se ejecuta este bloque final, si `lag_6` es vacío en esa fila se rellena con 0.

Código 12. Relleno final de los valores vacíos de las variables de entrada con 0.

```
# ...
for col in feature_cols:
    if df_model[col].isna().any():
        df_model[col] = df_model[col].fillna(0)
```

3.2.12 Etapa 4: Modelos de predicción

Esta etapa se reparte en los siguientes archivos:

- app/services/prediccion/modelos.py
- app/services/prediccion/regresion_lineal.py

Definición de variables y funciones básicas: Al inicio del archivo `modelos.py`, se define el nivel de significancia `NIVEL_SIGNIFICANCIA_DM` como 5%, esta variable se va a usar posteriormente a la hora de evaluar si un resultado es estadísticamente significativo:

Código 13. Nivel de significancia de la prueba de Diebold-Mariano.

```
NIVEL_SIGNIFICANCIA_DM = 0.05
```

Como parte de la propuesta de Thary, también se implementaron las siguientes cuatro variables:

Código 14. Constantes que definen el tamaño del periodo de prueba (*backtest*).

```
MESES_ARCHIVO_UMBRAL_TEST_GRANDE = 18
TEST_SIZE_MESES_DEFAULT = 3
TEST_SIZE_MESES_GRANDE = 6

MESES_PERDIDOS_POR_REZAGOS = 3
```

Por defecto, el periodo de prueba son 3 meses, pero cuando se cuenta con un periodo de prueba más grande, de 18 meses o más, este pasa de 3 meses a 6 meses.

Esta decisión se tomó después de identificar un problema con el que contaba el sistema. Como punto de partida, el sistema elige el mejor modelo para cada producto comparando el MAE de los 3 modelos sobre 3 meses del historial, el problema era que con solo 3 meses, el resultado podría no ser lo suficientemente confiable, esto debido a que con tan pocos datos, un solo mes que sea atípico puede terminar haciendo que un modelo "gane" sin que eso refleje una diferencia real, sobretodo porque el MAE es una medición que cuenta con mucha varianza. Entre los riesgos identificados

se encuentra: poca capacidad de distinguir modelos parecidos, inestabilidad del resultado y sesgo estacional.

Antes de llegar a la solución definitiva se planteó una primera propuesta, en la que si el archivo contaba con menos de 18 meses, se haría la comparación del MAE de los 3 modelos sobre 3 meses, pero si contaba con 18 meses o más, la comparación se haría sobre 6 meses. Pero en realidad, esta solución no resolvía el problema "de raíz", solo mitigaba el problema de que hubiera pocos meses de prueba, además de que la decisión de que 18 meses fuese el punto de corte no tenía respaldo alguno.

Por eso fue que se corrigió y re-interpretó el enfoque de la solución, fundamentándola en que el problema a solucionar no era el número de meses, sino el cálculo del error, es decir, qué tan confiable es la comparación de error entre los modelos.

Fue aquí donde se propuso la prueba Diebold-Mariano, la cual consiste en una prueba estadística que permite medir si la diferencia de error que hay entre dos modelos es significativa o es solo ruido. La prueba Diebold-Mariano requiere de un nivel de significancia, que se definió como 5%.

Por otro lado, `MESES_PERDIDOS_POR_REZAGOS` es el número fijo de meses que se pierden al calcular los rezagos (visto en la etapa anterior), es necesario para distribuir los meses utilizables del total de meses que subió el usuario en el archivo.

Una vez definidas las variables, la primera función implementada en `modelos.py` es `tamano_test_backtest`. Esta función principalmente se encarga de implementar la solución ya explicada, recibe como parámetro `n_meses_usables`, la cantidad de meses que quedaron disponibles después de que se hayan perdido los primeros por el cálculo de los lags, y devuelve cuántos meses de esos son los que se van a usar como período de prueba (backtest).

Código 15. Función `tamano_test_backtest`, que decide el tamaño del periodo de prueba.

```
def tamano_test_backtest(n_meses_usables):  
    n_meses_archivo_original = n_meses_usables + MESES_PERDIDOS_POR_REZAGOS  
    if n_meses_archivo_original >= MESES_ARCHIVO_UMBRAL_TEST_GRANDE:  
        return TEST_SIZE_MESES_GRANDE  
    return TEST_SIZE_MESES_DEFAULT
```

Como el umbral de decisión (`MESES_ARCHIVO_UMBRAL_TEST_GRANDE = 18`) toma en cuenta el archivo completo, y la función recibe la cantidad de meses disponibles ya con los 3 meses de lag descontados, con `n_meses_archivo_original = n_meses_usables + MESES_PERDIDOS_POR_REZAGOS` se le suma esos 3 meses de vuelta, para saber cuántos meses tenía el archivo que el usuario subió originalmente.

Luego se compara si el total de meses completo cuenta con 18 meses, de hacerlo devuelve el tamaño de test grande con 6 meses y de no hacerlo devuelve el tamaño de test por defecto de 3 meses.

Esta función se llama desde `sistema_prediccion.py`, para decidir el tamaño del split que se usa para elegir el campeón y generar los resultados, y desde `validacion.py`, para que los *folds* de la validación cruzada temporal usen ese mismo tamaño de test y conservar un criterio estandarizado en ambas partes.

Es parte de la solución que se propone en Thary, esta función no ninguna en alguna de las implementaciones que fueron revisadas ya que ninguna adapta el tamaño del periodo de prueba seun la cantidad de historia disponible.

Modelos de demanda: Las funciones que permiten entrenar y evaluar los tres modelos de demanda que se van a utilizar en el sistema: Media Móvil, Regresión Lineal y XGBoost. Estos fueron elegidos luego de realizar una revisión de literatura en la que se compararon 5 modelos para la predicción de la demanda, tal como se vio en el capítulo "Antecedentes", de los cuales se escogieron: XgBoost y regresión lineal al ser los que presentan mejor desempeño en este tipo de problemas, y adicionalmente se incluyó media móvil como un modelo más sencillo que puede ser útil para productos con demanda estable o que no cuenten con suficiente historial implementar para un modelo más complejo.

Implementación de XGBoost: La implementación de XGBoost se hizo combinando los hiperparámetros de *Ejemplo 1 (SKU Demand Forecasting)* [33] con la transformación del objetivo propuesta en *Ejemplo 3 (XGBoost)* [35].

Ejemplo 3 (XGBoost) [35] cuenta con una configuración muchísimo mas simple de la que necesitaría Thary, ya que en el caso del ejemplo, se estaba trabajando con aproximadamente 60 millones de filas, por lo que se priorizó la velocidad sobre el ajuste fino de hiperparámetros, por lo que no cuenta con factores como algún control de sobreajuste:

Código 16. Parámetros del modelo XGBoost del *Ejemplo 3*.

```
# Define the parameters for the XGBoost model
params = {
    'max_depth': 8,                # Maximum depth of a tree
    'eta': 0.5,                   # Learning rate
    'objective': 'reg:squarederror' # Objective function for regression
}
```

Por otra parte, el *Ejemplo 1 (SKU Demand Forecasting)* [33] si cuenta con una configuración mas completa que se podría reutilizar, en este se arma un `XGBRegressor` mas cuidado en el que se incluyen variables como `n_estimators=300`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, pero principalmente se escogió seguir la lógica de este ejemplo porque de los cuatro ejemplos revisados, es el único que consiste en un

sistema completo con una premisa bastante similar a la de Thary, ya que ofrece pronóstico de demanda, traducción de ese pronóstico en recomendaciones de reposición, punto de reorden, EOQ, y todo eso bajo restricciones de lead time y presupuesto.

Es por eso, que se adoptó su enfoque general completo:

- Variables
- modelos
- hiperparametros
- comparación de modelos
- lógica de reposición

Los otros ejemplos se revisaron posteriormente cuando se necesitó mejorar aspectos, agregar nuevas funcionalidades o en general, agregarle mas valor y robustez a Thary.

Los hiperparámetros utilizados parten de los mismos que se encuentran en el *Ejemplo 1 (SKU Demand Forecasting)* [33]. Como ya se explicó anteriormente, al ser el ejemplo más cercano a una implementación completa de un sistema con las características de Thary, se optó por mantener su lógica de entrenamiento y ajuste de hiperparámetros, con la diferencia de que `n_estimators` se aumentó de 300 a 500, y `objective` se fijó explícitamente en `reg:squarederror`, mientras que en el ejemplo original no se especificaba. Esto se cambió ya que se buscaba implementar el uso de mínimos cuadrados para poder estimar la bondad de los modelos.

Parámetros como `verbosity` fueron omitidos en Thary, pero se incluyó `n_jobs`, el cual permite utilizar todos los núcleos de procesamiento disponibles en la máquina donde se ejecuta el entrenamiento, en lugar de un único núcleo. Esto contribuye a reducir el tiempo de entrenamiento del modelo cuando se procesan catálogos con un número elevado de productos.

Código 17. Función `entrenar_xgboost`, con los hiperparámetros del modelo.

```
def entrenar_xgboost(X_train, y_train_log, X_test):  
    model = xgb.XGBRegressor(  
        objective="reg:squarederror", n_estimators=500,  
        learning_rate=0.05,  
        max_depth=4,  
        subsample=0.8,  
        colsample_bytree=0.8,  
        reg_alpha=0.1,  
        reg_lambda=1.0,
```

```
    random_state=42,  
    n_jobs=-1  
)  
model.fit(X_train, y_train_log)  
pred_xgb = model.predict(X_test)  
pred_xgb = np.maximum(np.expml(pred_xgb), 0)  
return model, pred_xgb
```

Cada uno de estos hiperparámetros tiene un impacto específico en cómo se entrena el modelo, y a continuación se explica qué efecto tiene el valor elegido para Thary:

- `n_estimators=500`: Es la cantidad de árboles que se entrenan uno tras otro. Con más árboles, el modelo tiene más oportunidades de corregir los errores que va dejando cada árbol anterior, pero también tarda más en entrenarse. El Ejemplo 1 usa `n_estimators=300`; se subió a 500 porque, al combinarse con una tasa de aprendizaje baja (`learning_rate=0.05`), se necesitan más árboles para que el modelo aprenda lo suficiente. Para confirmar que este cambio ayudaba, se probó con distintas cantidades de árboles (de 100 a 1200): con 300 (el valor original del Ejemplo 1) el error fue mayor que con 500, y subir más allá de 500 lo seguía bajando, pero muy poco (menos de 1% entre 500 y 1200), mientras que el tiempo de entrenamiento se triplicaba. Se eligió 500 por ser una mejora real sobre el valor original, sin pagar un tiempo de entrenamiento mucho mayor para una ganancia casi nula, algo relevante porque el modelo se vuelve a entrenar cada vez que un usuario sube un archivo.
- `learning_rate=0.05`: Es la tasa de aprendizaje (*shrinkage*), y controla cuánto peso se le da a cada árbol nuevo que se agrega (Sección 2.2.9). Con un valor bajo como 0.05, cada árbol individual aporta poco al resultado final, por lo que se necesitan muchos árboles (de ahí los 500) para que el modelo aprenda bien, pero a cambio, el modelo es menos propenso a memorizar el ruido de los datos de entrenamiento. Este valor es idéntico al del Ejemplo 1.
- `max_depth=4`: Limita cuántas divisiones puede hacer cada árbol (Sección 2.2.9). Con una profundidad de 4, cada árbol es relativamente simple, lo que evita que un solo árbol se vuelva demasiado complejo y termine memorizando patrones muy específicos de los datos de entrenamiento en vez de aprender el comportamiento general de la demanda. Este valor es idéntico al del Ejemplo 1.
- `subsample=0.8`: En cada árbol nuevo, solo se usa el 80% de las filas de entrenamiento, escogidas al azar (Sección 2.2.9). Esto hace que los árboles no vean siempre exactamente los

mismos datos, lo que ayuda a que el modelo generalice mejor en vez de ajustarse demasiado a casos particulares del historial. Este valor es idéntico al del Ejemplo 1.

- `colsample_bytree=0.8`: De manera similar al anterior, pero con las variables (columnas): cada árbol solo usa el 80% de las variables disponibles, también escogidas al azar. Esto evita que el modelo dependa siempre de las mismas variables para todos los árboles, lo que también ayuda a que generalice mejor. Este valor es idéntico al del Ejemplo 1.
- `reg_alpha=0.1` y `reg_lambda=1.0`: Son los términos de regularización L1 y L2 que penalizan la complejidad de cada árbol (Sección 2.2.9). Un valor más alto hace que el modelo sea más conservador, prefiriendo árboles más simples. Los dos valores se mantuvieron idénticos al Ejemplo 1. Al probar otros valores de `reg_alpha` sobre el archivo del caso de estudio, el resultado con 0.1 fue muy parecido al de no usar esta regularización, así que se conservó el valor del Ejemplo 1 sin necesidad de cambiarlo.
- `random_state=42`: No afecta el resultado del modelo en sí, sino que fija la semilla aleatoria que se usa en el `subsample` y el `colsample_bytree`, para que cada vez que se entrene el modelo con los mismos datos, se obtenga exactamente el mismo resultado. Este valor también es idéntico al del Ejemplo 1.
- `n_jobs=-1`: Le indica a XGBoost que use todos los núcleos de procesamiento disponibles en la máquina, en vez de uno solo, lo que reduce el tiempo de entrenamiento cuando se procesan catálogos con muchos productos. Este parámetro no viene del Ejemplo 1 (que no lo especifica); se agregó en Thary porque no afecta el resultado del modelo, solo la velocidad de entrenamiento.

En cuanto a la transformación logarítmica, se implementó siguiendo la lógica del *Ejemplo 3 (XGBoost)* [35], en el que el modelo no entrena directamente sobre `y_train`, que sería la demanda real, sino sobre `y_train_log`, que es la transformación logarítmica de `y_train`.

Esto porque en la vida real, la demanda suele tener una distribución en la que puede haber muchos productos con demanda baja y pocos con picos muy altos. Si se entrenara directamente sobre esos valores el modelo podría enfocarse demasiado en los pocos casos extremos, esto llevaría a que se prediciera peor la mayoría de los casos normales. El logaritmo lo que hace es parejar esa diferencia: hace que los valores muy grandes dejen de pesar tanto en comparación con los valores más pequeños, así el modelo le presta atención equilibrada a todos los productos y no solo a los que tienen demanda extrema. Se usa `log1p` porque la demanda puede valer 0, y el logaritmo de 0 no existe.

Implementación de Media Móvil: Media móvil hace referencia al método más simple de los tres. La técnica con la que se implementó también se basa fuertemente en el *Ejemplo 1 (SKU Demand*

Forecasting) [33], y consiste en simplemente reutilizar la columna de `media_movil_3` que ya se tiene desde la etapa anterior de features. No se tiene que hacer ningún entrenamiento.

Código 18. Función `prediccion_baseline_movil`, que implementa el modelo de media móvil.

```
def prediccion_baseline_movil(test_df):
    return np.maximum(test_df["media_movil_3"].values, 0)
```

La función recibe como parámetro el `test_df` y posteriormente, para todas las filas, devuelve el valor `media_movil_3` que ya se había calculado anteriormente.

El único parámetro relevante de este modelo es el tamaño de la ventana que se promedia, en este caso, 3 meses (`media_movil_3`). Una ventana más pequeña haría que el modelo reaccionara más rápido a cambios recientes en la demanda, pero también lo haría más sensible a meses atípicos. Una ventana más grande suavizaría más el resultado, pero tardaría más en reflejar cambios reales en el comportamiento de un producto. Se eligió 3 meses siguiendo la misma lógica del Ejemplo 1 (SKU Demand Forecasting) [33], que también usa una ventana de este tamaño.

Implementación de Regresión Lineal: Para implementar regresión lineal, se adaptó la lógica del *Ejemplo 4 (BoomBikes)* [36], al ser el modelo que más pasos requiere, se implementó de manera individual en el archivo `regresion_lineal.py`.

Código 19. Filtro de las columnas sin variación (función `_seleccionar_features_regresion`).

```
def _seleccionar_features_regresion(X_train, y_train, max_p_valor=
    candidatas = [c for c in X_train.columns if X_train[c].std() >
    if len(candidatas) < 2 or len(X_train) <= len(candidatas) + 1:
        return candidatas
```

Inicialmente, se filtran y excluyen las columnas que no varían demasiado, descartando las que tienen una desviación estándar de 0, ya que una desviación estándar con este valor indica que en esas columnas el valor es siempre el mismo número.

con `std() > 0`, esto se hace porque cuando se entrena regresión lineal, no siempre conviene usar todas las variables disponibles porque no todas tienen algo significativo que aportar. Es incluso perjudicial para el modelo, ya que una columna sin variación no le sirve de nada al modelo (no hay nada que "aprender" de un número que nunca cambia).

Posteriormente, se convierten todos los valores de cada columna a un rango entre 0 y 1, para mantener las proporciones relativas con `MinMaxScaler`. Se ajusta `fit_transform` usando solo los datos con los que se hace el entrenamiento, no se puede incluir a los de prueba porque eso haría que el sistema vea los datos que se supone debe predecir.

Código 20. Escalado de las variables con MinMaxScaler.

```

try:
    scaler = MinMaxScaler()
    X_train_esc = pd.DataFrame(
        scaler.fit_transform(X_train[candidatas]),
        columns=candidatas, index=X_train.index
    )

```

En esta parte se incluye Recursive Feature Elimination (RFE). Este consiste en un método que entrena varias veces un modelo de regresión lineal, y en cada una se elimina la variable que aporte menos al modelo hasta que al final, se quede con un número controlado de variables, en este caso, tienen que ser 2 variables como mínimo para posteriormente poder realizar el análisis, pero no pueden ser más de 8 variables porque no se pueden dejar demasiadas $n_rfe = \max(2, \min(8, \text{len}(\text{candidatas}) - 1))$.

El n_rfe define cuantas variables como máximo puede conservar el proceso de RFE (entre 2 y 8, según $n_rfe = \max(2, \min(8, \text{len}(\text{candidatas}) - 1))$). El límite inferior de 2 garantiza que quede al menos una variable además de la más importante, y el límite superior de 8 evita que el modelo termine con demasiadas variables, ya que aunque RFE ya descarta las menos importantes, un número muy alto de variables restantes haría más lento el resto del proceso de selección (VIF y p-valor), que se aplican después sobre las que sobrevivan a RFE.

Finalmente, seleccionadas es la lista de columnas que sobrevivieron esta primera etapa, aunque es posible que aún existan variables redundantes:

Código 21. Selección de variables mediante *Recursive Feature Elimination* (RFE).

```

n_rfe = max(2, min(8, len(candidatas) - 1))
rfe = RFE(LinearRegression(), n_features_to_select=n_rfe)
rfe.fit(X_train_esc, y_train)
seleccionadas = [c for c, usar in zip(candidatas, rfe.support_) if

```

La segunda etapa para seleccionar variables consiste en calcular el VIF o Factor de Inflación de Varianza para cada variable. Este se encarga de medir que tan redundante es una variable en comparación con las demás, el criterio para decidir esto es estimar si una variable se puede predecir con exactitud a partir de las otras, de ser este el caso, su VIF será muy alto y eso es un problema porque el modelo no podría distinguir cuál de las dos variables redundantes es la que está causando el efecto.

En la implementación de Thary, entre las variables que se considerarían redundantes se encuentran:

- `inventario` y `inventario_lag_1`: Escencialmente tienden a ser casi la misma variable, en muchos casos, es bastante probable que el inventario de un mes sea bastante parecido al del mes anterior.
- Los lags (`lag_1`, `lag_2`, `lag_3`, `lag_6`): También es comun que la demanda de un producto tienda a ser similar a la de meses anteriores, si un producto vendió bien un mes, es muy probable que también venda mes el siguiente mes, lo que a la larga hace que los lags sean variables muy parecidas entre si.
- (`media_movil_3` y `media_movil_6`) con los lags: Que todos los Lags sean similares llevaría a que, inevitablemente, la media móvil de 3 meses y la de 6 meses también lo sean debido a que se calculan a partir de promediar los lags.
- (`media_producto`) y las medias móviles: (`media_producto`) consiste en el promedio histórico general del producto, si los productos cuentan con un comportamiento estable, es probable que sea similar a (`media_movil_3` y `media_movil_6`), por lo que también se considera una variable redundante.

Teniendo esto en cuenta, las variables que no se considerarían redundantes son: `año`, `mes_cos`, `mes_sin`, `desviacion_movil_3` y `desviacion_producto`.

Código 22. Cálculo del VIF de las variables seleccionadas.

```
while len(seleccionadas) > 1:
    X_sm = sm.add_constant(X_train_esc[seleccionadas])
    with warnings.catch_warnings():
        warnings.simplefilter("ignore", category=SingularMatrixWarn
        warnings.simplefilter("ignore", category=UserWarning)
        vif = pd.DataFrame({
            "feature": seleccionadas,
            "VIF": [variance_inflation_factor(X_sm.values, i + 1) f
        })
```

Posteriormente, se quitan las variables con un VIF infinito que quisiera se puede calcular, ya que de ser este el caso, significaría que la variable es tan redundante con otra que rompe las matemáticas del modelo. Esto ocurriría porque a la hora de calcular los coeficientes de la regresión lineal, el algoritmo necesita calcular cuanto aporta cada variable a la predicción, y para eso, necesita poder distinguir el efecto individual de cada una. El problema es que si dos variables son casi idénticas entre sí, es imposible distinguir las.

Es por eso, que en esta etapa esas variables son eliminadas. En las siguientes líneas de código, se revisa cual variable tiene el VIF más alto si ya no hay variables con VIF infinito, si el valor del vif más alto supera el límite que se definió en `max_vif`, esa variable se elimina y finalmente se vuelve a repetir el proceso desde el principio hasta que se halla calculado el VIF de las variables restantes y todas estas cuenten con un VIF menor al límite establecido.

Código 23. Eliminación de variables con VIF infinito o superior al límite.

```
vif_problema = vif[~np.isfinite(vif["VIF"])]
if not vif_problema.empty:
    seleccionadas.remove(vif_problema.iloc[0]["feature"])
    continue

peor_vif_fila = vif.loc[vif["VIF"].idxmax()]
if peor_vif_fila["VIF"] > max_vif:
    seleccionadas.remove(peor_vif_fila["feature"])
    continue
```

Una vez completado el proceso de eliminación de las variables redundantes, se prosigue con la implementación del entrenamiento del modelo de regresión lineal, en el que se revisa el "p-valor" de cada variable.

El P-valor consiste en una medida estadística que indica que tan probable es que la variable que se esta revisando este relacionada con la demanda solo por casualidad y no porque de verdad dicha variable este relacionada de verdad con la demanda. En este caso, si se encuentra que la variable que cuenta con el peor p-valor supera el límite de `max_p_valor`, que es estandarizado como 0.05 al representar el 95% de confianza en estadística, se elimina la variable y se repite el proceso. El bucle se detiene cuando ya no haya ninguna variable con el VIF alto o con el P-valor alto, al cumplir con ambos criterios, solo se contara con variables limpias y estadísticamente significativas para el proceso.

De haber algún fallo durante esta etapa, simplemente se devuelve la lista completa de variables con el fin de garantizar de que el pipeline no se rompa y el sistema pueda continuar funcionando.

Código 24. Eliminación de variables según el p-valor del modelo OLS.

```

modelo_ols = sm.OLS(y_train, X_sm).fit()
p_valores = modelo_ols.pvalues.drop("const")
peor_p_nombre = p_valores.idxmax()
peor_p_valor = p_valores[peor_p_nombre]

if peor_p_valor > max_p_valor:
    seleccionadas.remove(peor_p_nombre)
else:
    break

return seleccionadas
except Exception:
    return candidatas

```

Por otro lado, en ese mismo archivo, se implementó la función `entrenar_regresion_lineal`, que se encarga de entrenar el modelo de regresión lineal y devolver las predicciones, también es a la que se llama desde el resto del sistema.

La función llama a `_seleccionar_features_regresion` para obtener la lista de variables que se van a usar en el entrenamiento del modelo, (si por algún motivo, dicha función le devuelve una lista que se encuentre vacía, se usan todas las variables originales.)

El modelo es entrenado con las variables limpias asegurándose que las predicciones no sean negativas al no ser algo que tenga sentido en este contexto.

Finalmente, devuelve el modelo entrenado, las predicciones y la lista de variables que se usaron para entrenar el modelo.

Código 25. Función `entrenar_regresion_lineal`, que entrena el modelo con las variables seleccionadas.

```

def entrenar_regresion_lineal(X_train, y_train, X_test, feature_cols):
    features_lr = _seleccionar_features_regresion(X_train, y_train)
    if not features_lr:
        features_lr = list(feature_cols)

    modelo_lr = LinearRegression()
    modelo_lr.fit(X_train[features_lr], y_train)
    pred_lr = modelo_lr.predict(X_test[features_lr])

```

```

pred_lr = np.maximum(pred_lr, 0)

return modelo_lr, pred_lr, features_lr

```

3.2.13 Selección del campeón por producto

El sistema de predicción Thary no usa el mismo método para todos los productos del catálogo. Para cada uno de los productos se selecciona cual de los tres modelos candidatos usar para su pronóstico. Esta decisión se tomó de manera independiente para el sistema Thary, ya que al revisar las implementaciones de los 4 sistemas de predicción en las que se basó el sistema, se evidenció que estos simplemente usaban el mejor modelo para todo el catálogo por igual.

En una primera versión del sistema, la selección del mejor modelo por producto se hacía en base a cual de los 3 obtuvo un menor MAE. El problema con esta estrategia es que el backtest generalmente solo cuenta con entre 3 y 6 meses de datos si es que se cuenta con un archivo más grande, son pocos meses, por lo que el hecho de que un modelo tenga el MAE más bajo no es prueba de que sea mejor, podría darse por diferentes factores o por casualidad.

Es por eso que para seleccionar el mejor modelo, lo primero que se hace es calcular el MAE de los 3 modelos, se identifican los dos modelos que obtengan el menor MAE para cada producto. Esto usando `np.argsort` sobre la tabla de errores:

Código 26. Identificación del mejor y el segundo mejor modelo de cada producto (función `elegir_campeon_por_producto`).

```

def elegir_campeon_por_producto(resultados_prediccion):
    errores_por_metodo = pd.DataFrame({
        metodo: (resultados_prediccion["demanda"] - resultados_prediccion[metodo])
        for metodo, columna in COLUMNA_PREDICCION_POR_METODO.items()
    })
    productos = resultados_prediccion["producto_id"]
    errores_por_producto = errores_por_metodo.groupby(productos).mean()

    metodos = errores_por_producto.columns.to_numpy()
    orden = np.argsort(errores_por_producto.to_numpy(), axis=1)
    mejor_metodo = pd.Series(metodos[orden[:, 0]], index=errores_por_producto.index)
    segundo_metodo = pd.Series(metodos[orden[:, 1]], index=errores_por_producto.index)

```

Se calcula la diferencia de error entre el mejor y el segundo mejor modelo para cada uno de los productos mes a mes, y se resume esa diferencia con su media, desviación estándar y cantidad de observaciones:

Código 27. Diferencia de error entre el mejor y el segundo mejor modelo.

```

mejor_por_fila = productos.map(mejor_metodo)
segundo_por_fila = productos.map(segundo_metodo)
error_mejor = np.select(
    [mejor_por_fila == metodo for metodo in errores_por_metodo.columns],
    [errores_por_metodo[metodo] for metodo in errores_por_metodo.columns],
    )
error_segundo = np.select(
    [segundo_por_fila == metodo for metodo in errores_por_metodo.columns],
    [errores_por_metodo[metodo] for metodo in errores_por_metodo.columns],
    )
diferencia = pd.Series(error_mejor - error_segundo, index=resultado.index)
stats_diferencia = diferencia.groupby(productos).agg(d_media="mean")

```

Se implementó la prueba de Diebold-Mariano [27]. El problema que resuelve es que cuando se comparan dos modelos para un producto, se quiere saber si el modelo ganador realmente es mejor o si por algún motivo tuvo suerte en esos pocos 3 meses, ya que un solo mes con un comportamiento atípico puede hacer que un modelo gane sin que eso signifique que sea mejor.

La prueba Diebold-Mariano permite determinar estadísticamente si la diferencia de error entre el mejor y el segundo mejor modelo es estadísticamente significativa, o si es solo ruido. La prueba consiste en observar mes a mes cuánto le gana un modelo al otro. Si esa diferencia es consistente (un modelo le gana al otro casi todos los meses, por una cantidad parecida), es una señal fuerte de que la ventaja es real. Si la diferencia varía mucho de un mes a otro (a veces gana uno, a veces el otro, por cantidades muy distintas), es señal de que lo que se vio como "ganador" fue más bien casualidad.

Debido a que la prueba clásica está pensada para muestras grandes, se adaptó para muestras pequeñas siguiendo la corrección de Harvey, Leybourne y Newbold, ampliamente discutida y recomendada por Diebold [26], la cual consistió en ajustar ($dm_{ajustado} = dm * \sqrt{(n-1)/n}$) y comparar el resultado contra una distribución t de Student con n-1 grados de libertad, en vez de la normal estándar de la prueba original. Cuantos más grados de libertad tiene una prueba estadística, más confiable es su capacidad de detectar diferencias reales, especialmente cuando se cuenta con pocos datos.

La implementación de la prueba consiste, inicialmente, en medir qué tan confiable es el promedio de las diferencias, dado que se calculó con pocos datos. Cuanto más dispersas son las diferencias (d_std) o menos meses tiene (n), más "ruidoso" es ese promedio.

Para comparar qué tan grande es la ventaja promedio frente a qué tan confiable es: si la ventaja

promedio es grande pero el error estándar (la inconsistencia) también es grande, `dm` resulta pequeño, lo cual es señal de que la ventaja no es confiable.

Código 28. Prueba de Diebold-Mariano con la corrección de Harvey, Leybourne y Newbold.

```
error_estandar = stats_diferencia["d_std"] / np.sqrt(stats_dif
dm = stats_diferencia["d_media"] / error_estandar
correccion_hln = np.sqrt((stats_diferencia["n"] - 1) / stats_d
dm_ajustado = dm * correccion_hln
grados_libertad = (stats_diferencia["n"] - 1).clip(lower=1)
p_valor = pd.Series(
    2 * (1 - stats.t.cdf(dm_ajustado.abs(), df=grados_libertad
    index=stats_diferencia.index,
)

suficientes_datos = stats_diferencia["n"] >= 2
sin_varianza = stats_diferencia["d_std"].fillna(0) == 0
significativo = suficientes_datos & (p_valor < NIVEL_SIGNIFICANCIA_D

campeon = mejor_metodo.copy()
campeon[~significativo] = "Media móvil"
return campeon
```

Se descarta el resultado del MAE y se usa el modelo de Media Móvil como respaldo si:

- La diferencia no resulta estadísticamente significativa, es decir, que el p-valor de la prueba Diebold-Mariano es mayor al nivel de significancia definido (5%).
- Los dos métodos se equivocaron exactamente igual (varianza cero, sin evidencia de que uno sea mejor)
- No hay al menos 2 meses para medir varianza

En estos casos se selecciona la Media Móvil en vez del modelo que ganó por una diferencia que no es significativa y que podría tratarse solo de ruido.

Al aplicar este mecanismo al sistema, se evidenció un cambio drástico en el resultado real, probado sobre el catálogo completo de la Clínica Nuestra Señora de los Remedios (856 productos). Antes de implementar la validación con Diebold-Mariano, seleccionando únicamente por el MAE más bajo, el método campeón se distribuía así:

- XGBoost ganaba en 430 productos (50.2%)
- Media Móvil en 227 (26.5%)
- Regresión Lineal en 199 (23.2%)

Al incorporar la prueba de Diebold-Mariano, esa distribución cambió radicalmente:

- XGBoost pasó a ganar en solo 15 productos (1.8%)
- Regresión Lineal en 12 (1.4%)
- Media Móvil pasó a ser el método campeón en 829 productos (96.8%)

De los 430 productos en los que originalmente ganaba XGBoost, 415 pasaron a usar Media Móvil y solo 15 se mantuvieron con XGBoost; de los 199 en los que ganaba Regresión Lineal, 187 pasaron a Media Móvil y solo 12 se mantuvieron. Los 227 productos que ya usaban Media Móvil se mantuvieron igual, ya que este es el método de respaldo cuando la diferencia no resulta significativa. Esto evidencia que, sin la validación estadística de Diebold-Mariano, el sistema estaba seleccionando modelos "ganadores" cuya ventaja, en la gran mayoría de los casos, no era estadísticamente distinguible del ruido.

En cuanto al tamaño del periodo de prueba, cuando se cuenta con un test fijo de 3 meses, la prueba de Diebold-Mariano siempre tendría solo 2 grados de libertad, sin importar cuánta historia tenga el archivo, nunca ganando más poder estadístico para detectar diferencias reales. Por eso, el tamaño del periodo de prueba se ajusta según el historial disponible:

Código 29. Función `tamano_test_backtest`, que ajusta el periodo de prueba al historial disponible.

```
def tamano_test_backtest(n_meses_usables):  
    n_meses_archivo_original = n_meses_usables + MESES_PERDIDOS_POR_RE  
    if n_meses_archivo_original >= MESES_ARCHIVO_UMBRAL_TEST_GRANDE:  
        return TEST_SIZE_MESES_GRANDE  
    return TEST_SIZE_MESES_DEFAULT
```

En caso de que se cuente con un archivo csv grande (es decir, un archivo con 18 meses o más), se usa un test de 6 meses en vez de 3, dándole más grados de libertad y más poder estadístico a la prueba de Diebold-Mariano.

El corte de 18 meses es práctica definida para este proyecto, se escogió por ser un valor redondo (año y medio). No es un valor tomado de una fuente externa, a diferencia del las demás prácticas aplicadas como Diebold-Mariano o nivel de significancia.

3.2.14 Combinación de pronósticos (análisis complementario)

Además de seleccionar un modelo por producto, se quiso evaluar si combinar dos modelos aporta algún valor en comparación con usar uno solo. Este análisis se agregó de manera independiente para el sistema Thary, ya que ninguna de las 4 implementaciones de referencia combina modelos. La idea viene de la literatura de combinación de pronósticos [22], [23].

Un aspecto importante es que esta combinación no participa en la predicción que usa Thary en producción ni en la selección del modelo campeón, sino que solo se muestra en el panel técnico del administrador como un análisis complementario.

Se combinan las predicciones de XGBoost y Media Móvil porque, de los tres modelos candidatos, son los que obtuvieron mejores resultados en la validación (Tabla XI). Lo primero es calcular el peso w que se le da a XGBoost, ya que a la Media Móvil le corresponde $1 - w$. Esto se hace con la fórmula de mínimos cuadrados, en la función `_peso_minimos_cuadrados`:

Código 30. Cálculo del peso de la combinación (función `_peso_minimos_cuadrados`).

```
def _peso_minimos_cuadrados(y, pred_a, pred_b):  
    diferencia = pred_a - pred_b  
    denominador = float(np.sum(diferencia ** 2))  
    if denominador == 0:  
        return 0.5  
    w = float(np.sum((y - pred_b) * diferencia) / denominador)  
    return min(max(w, 0.0), 1.0)
```

Aquí `y` es la demanda real, `pred_a` es la predicción de XGBoost y `pred_b` la de Media Móvil, y el cálculo del peso es la misma fórmula presentada anteriormente, aplicada sobre todas las filas del backtest. Hay dos casos especiales. El primero es que si las dos predicciones son idénticas, el denominador es cero, y como no hay nada que decidir entre un modelo y otro, el peso es 0.5, lo que además evita dividir entre cero. El segundo es que el resultado se recorta entre 0 y 1 con `min` y `max`, para que la combinación siempre sea un promedio ponderado, ya que si la demanda real queda fuera del rango entre las dos predicciones, la fórmula daría un peso mayor a 1 o menor a 0. Para saber si la combinación realmente mejora los resultados, se compara contra XGBoost solo y Media Móvil sola sobre las mismas filas del backtest. Un aspecto importante es que el peso de cada mes se calcula sin usar ese mes, sino solo con los demás meses del backtest:

Código 31. Cálculo del peso de cada mes sin usar ese mes (función `evaluar_combinacion`).

```
for mes in np.unique(meses):
    es_mes = meses == mes
    resto = ~es_mes
    if resto.any():
        w = _peso_minimos_cuadrados(y[resto], pred_xgb[resto], pred_mm[es_mes])
    else:
        w = 0.5

    pesos_por_mes[str(mes)] = round(w, 4)
    combinada[es_mes] = w * pred_xgb[es_mes] + (1 - w) * pred_mm[es_mes]
```

Esto se hace porque si el peso se calculara con los mismos datos contra los que después se mide el error, el peso ya habría "visto" las respuestas de antemano y la combinación saldría artificialmente buena, que es el mismo principio de no hacer trampa que se explicó en la validación del modelo. Además, como los productos se juntan en un solo cálculo, hay miles de filas para calcular cada peso aunque cada producto tenga pocos meses. Si solo existe un mes, no quedan datos para calcular el peso y se usa 0.5.

Como resultado, la función devuelve las métricas (MAE, RMSE, Mediana del Error, R^2 , MAPE y WAPE) de las tres opciones: Media Móvil sola, XGBoost solo y la combinación. También devuelve el peso final calculado con todos los meses, y cuánto mejora la combinación respecto al mejor de los dos modelos solos, en porcentaje de MAE y de RMSE. Un valor positivo significa que la combinación es mejor, y uno negativo que es peor. Esta función cuenta con 7 pruebas automatizadas.

3.2.15 Pronóstico futuro

El cálculo del pronóstico futuro es la funcionalidad principal del sistema, es decir, predecir la demanda de los productos en los meses posteriores al último mes del archivo CSV cargado por el usuario.

Después de que el usuario sube el archivo CSV con el historial de consumo, el sistema le permite configurar tanto el presupuesto disponible como el número de meses que desea pronosticar. Se implementó en `app/services/prediccion/pronostico_futuro.py`:

```
def generar_pronostico_futuro(
    df_mensual, producto_comportamiento, model, feature_cols, modelo_1,
    horizonte_meses=HORIZONTE_MESES_DEFAULT,
):
```

```

proximo_mes = df_mensual["fecha"].max() + pd.DateOffset(months=1)
meses_pronosticados = [
    (proximo_mes + pd.DateOffset(months=h)).strftime("%Y-%m")
    for h in range(horizonte_meses)
]
meses_pronosticados_legibles = [
    f"{NOMBRES_MES_ES[(proximo_mes + pd.DateOffset(months=h)).month-1]}"
    f"{(proximo_mes + pd.DateOffset(months=h)).year}"
    for h in range(horizonte_meses)
]

```

En donde, se toma el último mes con datos del archivo, se le suma un mes, y desde ahí se generan los `horizonte_meses` meses consecutivos que se van a pronosticar.

Ciclo recursivo: Para poder calcular el pronóstico futuro, se necesita del mes anterior para predecir el mes futuro, pero cuando se quiere predecir varios meses hacia adelante, el mes anterior a ese todavía no existe. Para resolverlo, Thary utiliza una estrategia recursiva [40], es decir, para predecir un mes se usa la predicción del mes anterior como si fuera un mes real. Es recursiva porque el resultado de un mes se vuelve la entrada para predecir el mes siguiente. Es una característica propia de Thary respecto a los ejemplos de referencia *Ejemplo 1 (SKU Demand Forecasting)* [33] y *Ejemplo 3 (XGBoost)* [35], ya que estos solo pueden pronosticar el mes siguiente al último dato. La desventaja es que, al usar los meses pronosticados como entrada, es probable que el error vaya acumulándose y creciendo a medida que se pronostican meses más lejanos.

```

for h in range(horizonte_meses):
    mes_objetivo = proximo_mes + pd.DateOffset(months=h)

    filas_features = []
    metadatos = []

    for producto in productos_validos:
        estado = estado_por_producto[producto]
        demandas = estado["demandas"]
        ultima_filas = estado["ultima_filas"]

        lag_1 = demandas[-1]
        lag_2 = demandas[-2]
        lag_3 = demandas[-3]

```

```

lag_6 = demandas[-6] if len(demandas) >= 6 else 0

media_movil_3 = np.mean(demandas[-3:])
media_movil_6 = np.mean(demandas[-6:]) if len(demandas) >= 6 else 0
desviacion_movil_3 = np.std(demandas[-3:])

fila_features = {
    "año": mes_objetivo.year,
    "mes_sin": np.sin(2 * np.pi * mes_objetivo.month / 12),
    "mes_cos": np.cos(2 * np.pi * mes_objetivo.month / 12),
    "lag_1": lag_1,
    "lag_2": lag_2,
    "lag_3": lag_3,
    "lag_6": lag_6,
    "media_movil_3": media_movil_3,
    "media_movil_6": media_movil_6,
    "desviacion_movil_3": desviacion_movil_3,
    "media_producto": ultima_fila.get("media_producto", 0),
    "desviacion_producto": ultima_fila.get("desviacion_producto", 0)
}

```

En esta parte se preparan los datos de entrada que el modelo necesita para cada uno de los meses que se quiere pronosticar. La estructura cuenta con dos ciclos: el primero recorre los meses a pronosticar, por lo que, de querer predecir los próximos 3 meses con un archivo de 856 productos, el ciclo se ejecutará 3 veces, y el segundo, en cada una de las ejecuciones del primero, recorre los productos que se quieren pronosticar (los que tienen al menos 3 meses de historial), en este caso 856.

- Los lags `lag_1 = demandas[-1]`, `lag_2 = demandas[-2]`, `lag_3 = demandas[-3]` y `lag_6 = demandas[-6]` contienen las unidades vendidas hace 1, 2, 3 y 6 meses, respectivamente (si hay menos de 6 meses, `lag_6` es 0).
- `media_movil_3 = np.mean(demandas[-3:])`: Consiste en el promedio de los últimos 3 meses.
- `media_movil_6 = np.mean(demandas[-6:])`: Promedio de los últimos 6 meses (si hay menos de 6, el de los meses disponibles).

- `desviacion_movil_3 = np.std(demandas[-3:])`: Consiste en la desviación estándar de los últimos 3 meses.
- `año, mes_sin, mes_cos`: Estas variables se calculan del mes que se está pronosticando.
- `media_producto` y `desviacion_producto`: Consisten en el promedio y la desviación estándar del producto. Se toman de la última fila del producto y no cambian entre vueltas; si esa columna no existe, se usa 0.

Estos datos se guardan en el diccionario `fila_features`, que contiene las variables de entrada con las que el modelo fue entrenado.

En el segundo ciclo, cada producto aporta una fila con sus variables de entrada. Al terminar de recorrerlos todos, se tienen las filas de todos los productos acumuladas en `filas_features`, listas para que el modelo las use. En esta parte todavía no se predice nada.

El proceso es recursivo porque la lista de demanda de cada producto no es fija, se va actualizando al terminar cada mes, ya que la predicción obtenida se agrega al final de la lista, de manera que en el próximo mes se utilicen esos datos para la predicción. Así, en el primer mes pronosticado el `lag_1` es la demanda real del último mes del archivo, mientras que en el segundo mes pronosticado es la predicción del primero.

Se arma una tabla con las variables de todos los productos y se llama a `predict()` una sola vez por modelo, para predecir los productos de todo el lote. De esta manera, se pueden predecir todos los productos de un mes en una sola llamada al modelo, lo que mejora el rendimiento del sistema, ya que se obtienen los mismos resultados que si se llamara al modelo producto por producto y es mucho más rápido. Para cada uno de los productos se obtienen las tres predicciones de los 3 modelos.

```
X_futuro_lote = pd.DataFrame(filas_features)[feature_cols]
preds_xgb_lote = np.maximum(np.expm1(model.predict(X_futuro_lote))
preds_lr_lote = np.maximum(modelo_lr.predict(X_futuro_lote[feature

for (producto, media_movil_3), pred_xgb_futuro, pred_lr_futuro in
    estado = estado_por_producto[producto]
    segmento = estado["segmento"]

    pred_xgb_futuro = float(pred_xgb_futuro)
    pred_lr_futuro = float(pred_lr_futuro)
    pred_baseline_futuro = float(max(media_movil_3, 0))
```

Una vez que se tienen las 3 predicciones de los modelos, se tiene que seleccionar cuál de los 3 se va a usar para cada producto, guardar dicha selección y tenerla disponible para el siguiente mes.

La selección del modelo ya se tiene lista desde la etapa anterior de "selección de modelo". Siempre se usa el resultado del mejor modelo, a no ser que ese resultado sea incoherente, para esos casos, se implementó la salvaguarda, en la que se usa la media móvil en su lugar.

Las condiciones para considerar un resultado incoherente son:

- El modelo tiene que ser XGBoost o Regresión Lineal
- Media móvil tiene que ser mayor de 0, es decir, el producto tiene que haber vendido algo en los últimos 3 meses
- La predicción del modelo ganador es menor que la mitad de la media móvil de los últimos 3 meses, es decir, el modelo ganador predice que se va a vender menos de la mitad de lo que se ha vendido en los últimos 3 meses.

Se tiene el diccionario `predicciones_por_metodo` que cuenta con las 3 predicciones, de aquí es donde se obtiene el método elegido y posteriormente, con `pronostico_futuro.append({...})` se guarda una fila con:

- Identificación del producto: `producto_id`, `nombre_producto` y `categoria`
- Mes pronosticado: `mes_pronosticado`
- Las 3 predicciones de los 3 modelos: `prediccion_xgboost`, `prediccion_regresion_lineal` y `prediccion_baseline`
- Predicción final: La que se le mostrara al usuario final (`prediccion_final`)
- Método usado: `metodo_usado`, este puede cambiar si al final se recurrió a usar la salvaguarda.

Por último, se agrega la predicción hecha al final de la lista de demandas del producto, por lo que en el mes siguiente, `demandas[-1]` (el `lag_1`) ya no representará la demanda real, sino la predicción recién hecha.

```
metodo_campeon = segmento["metodo_campeon"]
if (
    metodo_campeon != "Media móvil"
    and media_movil_3 > 0
    and predicciones_por_metodo[metodo_campeon] < media_movil_3 *
):
    metodo_usado = "Media móvil"
```

```

else:
    metodo_usado = metodo_campeon

pred_final_futuro = predicciones_por_metodo[metodo_usado]

pronostico_futuro.append({
    "producto_id": producto,
    "nombre_producto": segmento.get("nombre_producto"),
    "categoria": segmento.get("categoria"),
    "mes_pronosticado": mes_objetivo.strftime("%Y-%m"),
    "horizonte": h + 1,
    "prediccion_xgboost": round(pred_xgb_futuro, 2),
    "prediccion_regresion_lineal": round(pred_lr_futuro, 2),
    "prediccion_baseline": round(pred_baseline_futuro, 2),
    "prediccion_final": round(pred_final_futuro, 2),
    "metodo_usado": metodo_usado,
})

estado["demandas"].append(pred_final_futuro)

```

Para mostrarle los resultados al usuario, se arma una tabla en la que se resume la predicción de la demanda de los productos, mostrando su id y nombre, la predicción final de cada mes pronosticado (una columna por mes, en unidades enteras) y el total. Esta tabla se puede descargar en formato CSV para su posterior análisis.

3.2.16 Cálculo de reposición (EOQ/ROP)

El cálculo de la reposición responde a la necesidad de determinar cuánto inventario de seguridad es conveniente mantener para cada producto en un nuevo pedido, junto a otra información relevante como en qué nivel de inventario es más conveniente hacer el pedido, qué cantidad se debe pedir y en qué fecha debe hacerse el pedido.

Para esta parte, se utilizaron las fórmulas de gestión de inventario clásicas para stock de seguridad, punto de reorden y cantidad económica de pedido (EOQ), los cuales también se usaron en los ejemplos de referencia *Ejemplo 1 (SKU Demand Forecasting)* [33] y *Ejemplo 2 (Machine Learning for Retail Demand Forecasting: Comparative Study of Demand Forecasting Methods for a Retail Store (XGBoost Model vs. Rolling Mean))* [34]. En este aspecto, la propuesta de valor de Thary es que estas fórmulas se hacen en base al pronóstico futuro que fue generado en la etapa anterior, y no en base a los datos históricos.

A partir de las predicciones finales de los meses pronosticados se calculan el promedio mensual $\bar{d}$

y su desviación estándar σ_d .

```
def calcular_tabla_reorden(
    df_model, df_mensual, producto_comportamiento, pronostico_futuro_df,
    presupuesto_capital_trabajo=None
):
    #...
    stats_pronostico = pronostico_futuro_df.groupby("producto_id")["pr
        demanda_promedio="mean", desviacion_demanda=lambda x: np.std(x
    )
```

Lead time: El lead time (L) de cada producto representa el tiempo en el que los proveedores entregarían el producto después de hacer un pedido, en el sistema se expresa en meses para que sea consistente con la unidad de tiempo de las demandas. Para asegurar esto, si el archivo original contiene el lead time en semanas, el sistema convierte los semanas de lead time a meses, dividiéndolas entre 4.345. De no tener el lead time, se asume un valor por defecto de 0.5 meses.

```
lead_time_meses = (
    fila_params["lead_time_semanas"] / 4.345
    if "lead_time_semanas" in df_mensual.columns and not pd.isna(fila_pa
    else LEAD_TIME_DEFAULT_MESES
)
```

Stock de seguridad y punto de reorden: El stock de seguridad es esa cantidad de productos extra que se guarda en el inventario para cubrir la variabilidad de la demanda mientras llega un pedido.

$$SS = Z \cdot \sigma_d \cdot \sqrt{L}$$

```
stock_seguridad = SERVICE_LEVEL_Z * desviacion_demanda * np.sqrt(lead_
```

El SERVICE_LEVEL_Z corresponde a un nivel de servicio de 95% ($Z=1.65$). Este porcentaje hace referencia a que se espera que el stock de seguridad sea suficiente para cubrir la demanda durante el lead time en 95% de los casos.

El punto de reorden es el nivel de inventario en el que, cuando se llega a este, se debe de hacer un nuevo pedido.

$$ROP = \bar{d} \cdot L + SS$$

```
punto_reorden = demanda_promedio * lead_time_meses + stock_seguridad
```

En el que:

- $\bar{d}$ es la demanda promedio.
- L es el lead time.
- SS es el stock de seguridad.

Cantidad económica de pedido (EOQ): El EOQ consiste en la cantidad de cada pedido que minimiza los costos de inventario, considerando el costo de mantener inventario y el costo de hacer pedidos. La fórmula consiste en:

$$EOQ = \sqrt{\frac{2 \cdot D \cdot S}{H}}$$

En donde:

- $D = 12 \cdot \bar{d}$: Demanda anual estimada
- S es el costo de hacer un pedido
- H es el costo anual de mantener una unidad en inventario, calculado como el costo unitario multiplicado por el porcentaje de costo de mantenimiento.

En el sistema, esto se implementó teniendo en cuenta que como el costo de pedido y el porcentaje de mantenimiento son columnas opcionales del archivo csv, por lo que el EOQ solo se calcula si están disponibles en el archivo:

```
demanda_anual_estimada = demanda_promedio * 12
if not pd.isna(costo_unitario_producto) and not pd.isna(porcentaje_mant):
    costo_mantenimiento_unitario = costo_unitario_producto * porcentaje_mant
    eoq = np.sqrt((2 * demanda_anual_estimada * costo_pedido_producto) / costo_mantenimiento_unitario)
else:
    eoq = np.nan
```

Cantidad sugerida y presupuesto: La cantidad sugerida se calcula en base al EOQ, de no estarlo, pero se conoce el inventario actual, se sugiere la cantidad que falta para llegar al punto de reorden. Esto se propuso en el sistema Thary para poder hacer recomendaciones de pedidos en catálogos que no traigan costos de pedido o mantenimiento.

En cuanto al presupuesto, la cantidad que el usuario incluye se reparte por igual entre todos los productos y el límite corresponde a la cantidad de unidades que ese monto permite comprar. La cantidad final es el menor valor entre la cantidad calculada y el límite por presupuesto. El costo estimado del pedido se obtiene multiplicando esa cantidad por el costo unitario.

```
presupuesto_por_producto = working_capital_budget / n_productos_activos

if not pd.isna(costo_unitario_producto) and costo_unitario_producto > 0:
    limite_presupuesto = int(presupuesto_por_producto / costo_unitario_producto)
else:
    limite_presupuesto = np.nan

if not pd.isna(eoq):
    cantidad_sugerida = eoq
elif not pd.isna(inventario_actual):
    cantidad_sugerida = max(0, punto_reorden - inventario_actual)
else:
    cantidad_sugerida = np.nan

if not pd.isna(cantidad_sugerida) and not pd.isna(limite_presupuesto):
    cantidad_sugerida = min(cantidad_sugerida, limite_presupuesto)
```

Decisión de pedido y fecha estimada: Para poder organizar los pedidos de una manera eficiente y fácil de interpretar, los productos se marcan con un indicador de si es necesario hacer un pedido o no, (ordenar = "SI") cuando el inventario actual del producto es menor o igual al punto de reorden.

También se incluyó la fecha estimada en la que el inventario llegará a ese punto de reposición, asumiendo que el consumo del inventario se mantiene con el ritmo promedio ya pronosticado, la fecha de referencia es el último mes del que se tiene información en el archivo csv. Si el inventario está por debajo del punto de reorden, se usa la fecha del último mes con datos de archivo. Por otra parte, si el inventario está por encima, se calculan los días que faltan para llegar al punto de reorden, dividiendo el inventario que sobra entre la demanda diaria, que es la demanda mensual pronosticada dividida entre 30.44 (días promedio por mes). Esos días se suman a la fecha de referencia y el resultado es la fecha estimada en la que se debe hacer el pedido. Se aplica un tope de 36 500 días (100 años) para evitar valores absurdos en productos con demanda pronosticada casi nula. Si no hay inventario o no hay demanda pronosticada, no se puede estimar la fecha.

```
ya_hay_que_pedir = not pd.isna(inventario_actual) and inventario_actual < punto_reorden
if pd.isna(inventario_actual):
    fecha_estimada_pedido = None
    dias_para_pedido = np.nan
elif ya_hay_que_pedir:
```

```

    fecha_estimada_pedido = fecha_referencia
    dias_para_pedido = 0
elif demanda_promedio > 0:
    demanda_diaria = demanda_promedio / DIAS_POR_MES
    dias_para_pedido = min(
        (inventario_actual - punto_reorden) / demanda_diaria,
        DIAS_PARA_PEDIDO_TECHO,
    )
    fecha_estimada_pedido = fecha_referencia + pd.Timedelta(days=dias_
else:
    fecha_estimada_pedido = None

```

Como calculos adicionales, se incluyó que la tabla de reposición también calculara el valor del inventario actual y el margen de dinero que se dejaria de ganar si el producto se agota durante el lead time.

Como principales limitaciones es que σ_d sale de las predicciones y no del error de pronóstico ni sobre la demanda real, por lo que mide que tanto varían las predicciones y no la incertidumbre real.

3.2.17 Modelo de Datos

Conceptualmente, el modelo de datos del sistema Thary utiliza un modelo relacional, en el que la información se organiza en 12 tablas relacionadas entre si, definidas mediante claves foraneas: empresas, usuarios, pedidos, actividad (el historial de movimientos), y corridas. Esta última es la tabla principal que representa cada vez que un usuario (administrador) sube un CSV, junto con 7 tablas que vienen de cada una de las corridas: segmentacion, metricas_modelos, predicciones_backtest, pronostico_futuro, reorder, validacion_cruzada_folds, validacion_cruzada_resumen. Cada archivo que se sube crea una fila nueva en corridas, aunque no se borra el historial, la app solo muestra la corrida más reciente.

Lógica y Fisicamente, el modelo hace uso de MySQL (motor de base de datos relacional), el sistema se conecta a este usando SQLAlchemy. En la tabla empresas, se almacena el id y el nombre de cada organización que es registrada en el sistema, por otra parte, en la tabla usuarios se guarda la información del usuario, como su nombre, correo electronico, contraseña (cifrada), el rol, ya sea admin o empleado, y el id de la empresa a la que pertenece. La tabla pedidos también se almacena de forma independiente separada de los resultados del procesamiento, y es única para cada empresa, así se garantiza que cuando se cargue un nuevo historial, no se elimine el registro de los pedidos ya confirmados. Por otro lado, las tablas de segmentación, recomendación de reposición y comportamiento del producto son generadas después de que el sistema procese el archivo CSV

que fue cargado por el administrador. Estos resultados se guardan y quedan asociadas a la corrida que las origino mediante `corrida_id`.

No toda la información se queda en la base de datos, ya que los archivos CSV que el usuario sube, las gráficas generadas y el modelo XGBoost entrenado se siguen almacenando como archivos en disco, los cuales estan organizados en carpetas por empresa. Solo se traslada a MySQL la información que pueda estructurarse en filas y columnas.

TABLA VIII
Entidades principales del modelo de datos.

Entidad	Descripción	Cardinalidad
Empresa	Organización que utiliza el sistema, identificada de forma única.	1 : N
Usuario	Persona registrada en el sistema, con un rol, asociada a una empresa mediante clave foránea.	1 : N
Pedido	Proveedor, cantidad, costo y usuario responsable de un pedido confirmado, asociado a una empresa.	1 : N
Corrida	Registro de un procesamiento de archivo, con sus parámetros y métricas globales.	1 : N
Segmentación	Clasificación ABC, variabilidad y método campeón por producto, para una corrida dada.	N : 1
Métricas de modelos	MAE, RMSE, MAPE y WAPE de cada modelo candidato, para una corrida dada.	N : 1
Predicciones de backtest	Demanda real y predicha por producto-mes, para el periodo de prueba de una corrida.	N : 1
Pronóstico futuro	Demanda estimada para los próximos meses, por producto, para una corrida dada.	N : 1
Reorden (reposición)	Punto de reorden, EOQ y cantidad sugerida por producto, para una corrida dada.	N : 1
Validación cruzada (folds)	Métricas de error por modelo, calculadas en cada fold de la validación cruzada temporal.	N : 1
Validación cruzada (resumen)	Promedio y desviación de las métricas de cada modelo, a través de los folds.	N : 1
Actividad	Bitácora de acciones realizadas por los usuarios, asociada a una empresa.	1 : N

La Tabla VIII resume las entidades del modelo con el detalle de cada campo y su cardinalidad.

La clave foranea (FOREIGN KEY) de usuarios hacia empresas rechaza cualquier intento de crear un usuario sin una empresa asociada, garantizando el aislamiento de la información entre organizaciones. La relación entre corridas y segmentacion es de dependencia en la que no tiene ningún sentido tener una fila en segmentación si no sabemos a qué corrida pertenece, ON DELETE CASCADE implica que, si se elimina una corrida, todas sus filas de segmentación (junto con las métricas, predicciones, reposición y validación cruzada) se eliminan junto con ella:

Código 32. Definición de las tablas empresas, usuarios, corridas y segmentacion con sus claves foráneas.

```
CREATE TABLE empresas (  
    id VARCHAR(32) PRIMARY KEY  
    #....  
)  
  
CREATE TABLE usuarios (  
    id INT PRIMARY KEY,  
    empresa_id VARCHAR(32) NOT NULL,  
    FOREIGN KEY (empresa_id) REFERENCES empresas(id)  
)  
  
CREATE TABLE corridas (  
    id INT PRIMARY KEY,  
    empresa_id VARCHAR(32) NOT NULL,  
    FOREIGN KEY (empresa_id) REFERENCES empresas(id)  
)  
  
CREATE TABLE segmentacion (  
    id BIGINT PRIMARY KEY,  
    corrida_id INT NOT NULL,  
    FOREIGN KEY (corrida_id) REFERENCES corridas(id) ON DELETE CASCADE  
)
```

En el siguiente fragmento, extraído de `app/services/db.py`, se observa como Thary establece la conexión hacia el motor de MySQL:

Código 33. Configuración de la conexión a MySQL (app/services/db.py).

```
_engine = None
```

```
def _config():
    return {
        "host": os.environ.get("DB_HOST", "127.0.0.1"),
        "port": os.environ.get("DB_PORT", "3307"),
        "user": os.environ.get("DB_USER", "thary_app"),
        "password": os.environ.get("DB_PASSWORD", ""),
        "database": os.environ.get("DB_NAME", "sistema_inventario"),
    }

def get_engine():
    global _engine
    if _engine is None:
        cfg = _config()
        url = (
            f"mysql+pymysql://{cfg['user']}:{cfg['password']}@"
            f"{cfg['host']}:{cfg['port']}/{cfg['database']}?charset=u
        )
        _engine = create_engine(url, pool_pre_ping=True)
    return _engine
```

Como se puede ver, la configuración de(`_config()`) no se obtiene de valores fijos, sino de las variables de entorno. Mientras que el motor (`engine`) solo se crea una vez durante toda la ejecución de la aplicación en vez de abrir una conexión nueva con cada solicitud, esto ayuda a que se evite el costo de tener que establecer una conexión nueva a la base de datos por cada petición HTTP que sea recibida por Thary.

Una vez realizada esta conexión, se utilizó `mySQL Workbench` para verificar que la base de datos y las tablas fueron creadas correctamente:

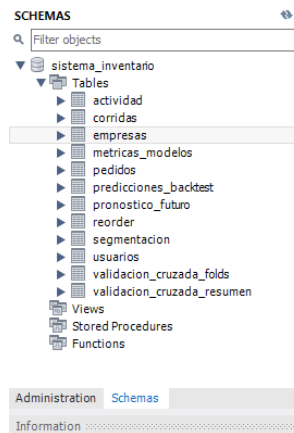


Fig. 8. Estructura de la tabla empresas vista desde MySQL Workbench, conectado a la base de datos local.

La tabla empresas existe en la base de datos con las columnas descritas anteriormente: un identificador de tipo `varchar (32)` como clave primaria, el nombre de la empresa, y la fecha de creación del registro.

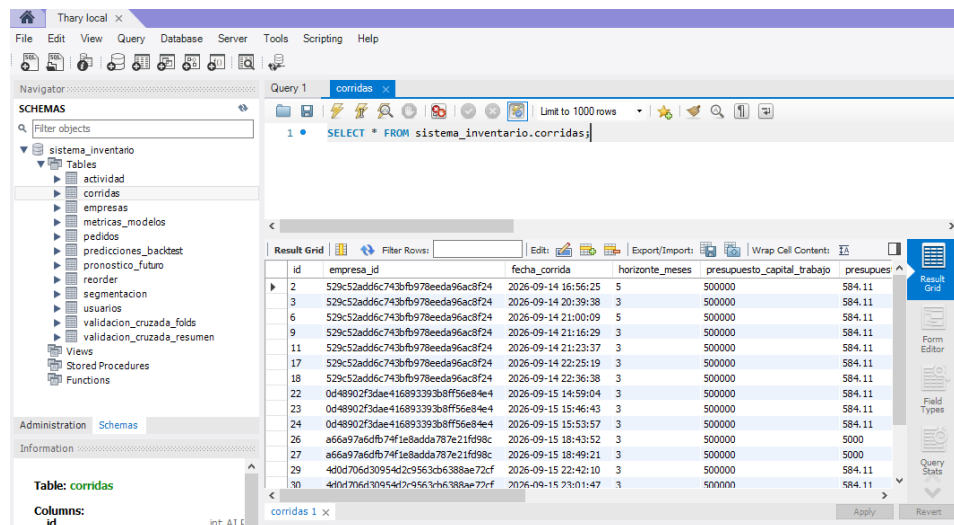


Fig. 9. Consulta sobre la tabla corridas en MySQL Workbench, mostrando el esquema completo de 12 tablas y el historial acumulado de corridas.

La Fig. 9 muestra la confirmación de que cada procesamiento de archivo genera una fila nueva en corridas sin eliminar las anteriores. Existen múltiples filas asociadas a una misma empresa,

correspondientes a distintas corridas realizadas en distintas fechas y horas.

3.2.18 Diseño UX/UI

El nombre del sistema "Thary" fue elegido en la etapa de diseño de identidad visual, es un nombre corto escogido por fonéticamente sonar similar a las últimas cuatro letras de la palabra "Inventory", (inventario en inglés). En cuanto a icono representativo, se eligió la figura de un buho, para transmitir la idea de orden y control, ya que los buhos son animales reconocidos por su capacidad de percepción y vigilancia, relacionado a como Thary se encarga de detectar señales de riesgo que podrían afectar el inventario de una empresa.



Fig. 10. Logotipo de Thary, compuesto por el nombre del sistema y el icono representativo del búho

Para realizar el diseño de interacción de Thary, se decidió separar las pantallas en dos: Una dirigida exclusivamente a los administradores y otra dirigida a los empleados del inventario. Al ingresar al sistema, un usuario con el rol de administrador sigue el siguiente flujo:

El usuario inicia sesión o se registra de no tener una cuenta.

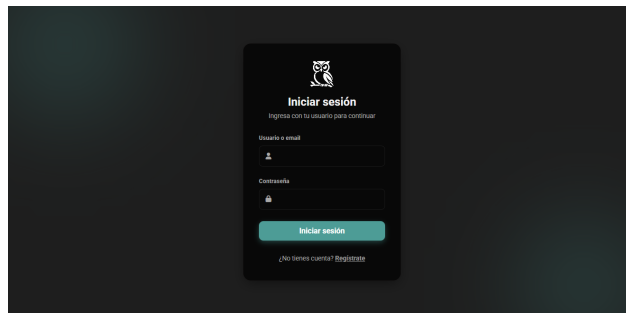


Fig. 11. Pantalla de inicio de sesión de Thary

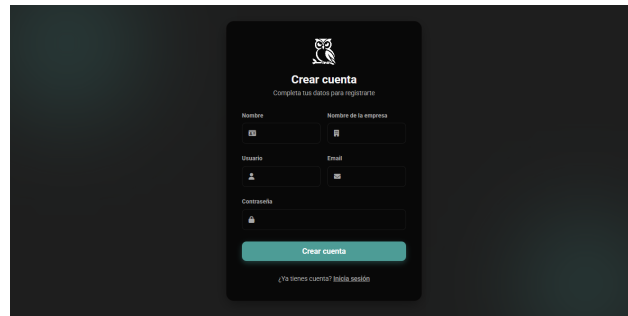


Fig. 12. Pantalla de registro de Thary

El administrador inicia en la pantalla de inicio del sistema, desde la cual puede acceder a la funcionalidad de carga de archivo (*index*), al haber cargado el archivo correctamente y que este sea válido, se le redirige automáticamente al dashboard, donde al seleccionar "Pronóstico de demanda", se muestran los resultados.

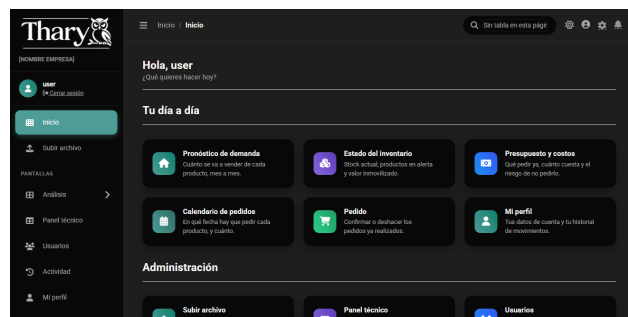


Fig. 13. Pantalla de dashboard de Thary

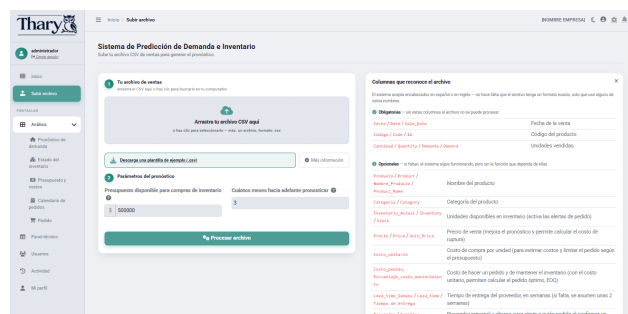


Fig. 14. Pantalla para subir archivos de Thary



Fig. 15. Pantalla de predicción de demanda de Thary

En el sidebar, además de encontrar demas opciones como estado del inventario, presupuesto y pedidos, el administrador puede navegar hacia la gestión de usuarios, donde puede crear cuentas nuevas para su personal de inventario.

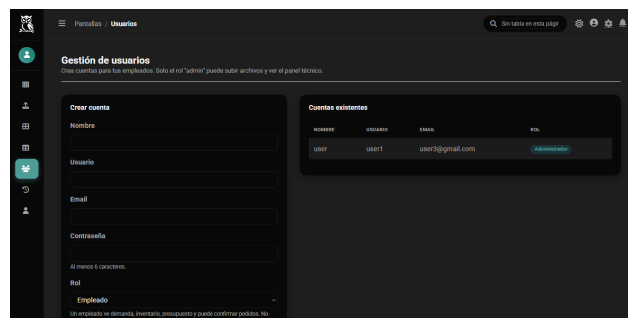
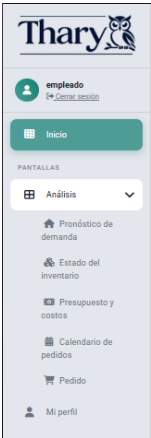
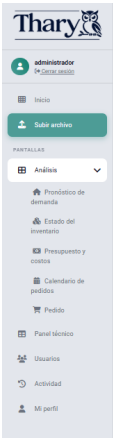


Fig. 16. Pantalla de gestión de usuarios de Thary

El flujo de los empleados es bastante similar, con la diferencia de que el perfil de un empleado cuenta con menos privilegios que el perfil de un administrador, por lo que al iniciar sesión, el empleado no puede subir ningún archivo, por lo que solo puede acceder al dashboard de la demanda pronosticada en base al csv que ya anteriormente fue cargado al sistema por el administrador que creó su cuenta, ya desde ahí, puede navegar a las mismas funcionalidades del análisis de los datos que el administrador: estado del inventario, presupuesto y pedidos.



(a) Sidebar de empleado



(b) Sidebar de administrador

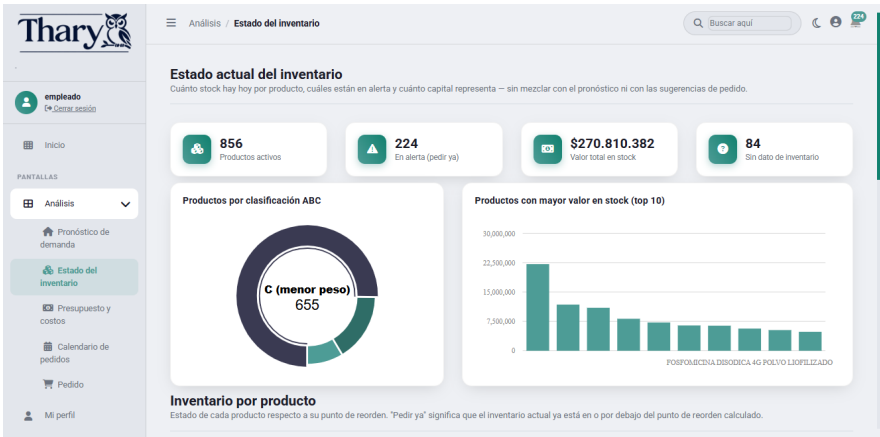
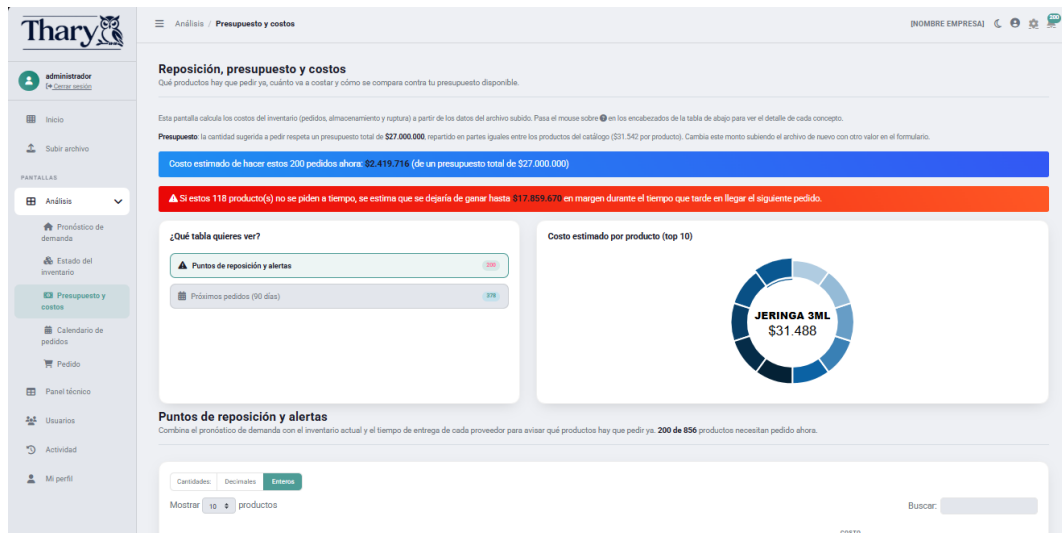


Fig. 18. Estado del inventario



(a) Presupuesto

Puntos de reposición y alertas
 Combina el pronóstico de demanda con el inventario actual y el tiempo de entrega de cada proveedor para avisar qué productos hay que pedir ya. **200 de 856** productos necesitan pedido ahora.

Características: Decimales: Enteros

Mostrar **10** productos

PRODUCTO	CATEGORÍA	PROVEEDOR	INVENTARIO ACTUAL	ESTADO DE REPOSICIÓN	PUNTO DE REPOSICIÓN	TIEMPO DE ENTREGA DEL PROVEEDOR	CANTIDAD SUGERIDA A PEDIR	COSTO ESTIMADO DEL PEDIDO	MARGEN SI NO SE PIDE	ACCIÓN
0002101 SOL PACHEL CAPSULA 30MG	MELOCARANTES	---	481	97	516	1.0 semana(s)	35	\$1.280	\$4.294	
120898 FRESENIUS HP ENERGY SOLUCION OVAL 1.500ML/1000ML ALIMENTO LACTEO	ALIMENTOS	FRESENIUS Alimento 01/01/2024	7	4	21	0.6 semana(s)	1	\$30.994	\$153.715	
120900 FRESENIUS HP ENERGY SOLUCION OVAL 1.500ML/1000ML ALIMENTO LACTEO	ALIMENTOS	FRESENIUS Alimento 01/01/2024	24	10	27	0.6 semana(s)	1	\$19.504	\$112.454	
145527 SOL PACHEL CAPSULA 30MG	MELOCARANTES	---	0	9	21	1.0 semana(s)	21	\$14.448	---	
162236 SOLINGA SOLUCION OVAL 1.500ML/1000ML ALIMENTO LACTEO	MELOCARANTES	SOLINGA Alimento 01/01/2024	1	0	2	1.7 semana(s)	0	\$0	\$22.345	
162278 SOLINGA SOLUCION OVAL 1.500ML/1000ML ALIMENTO LACTEO	MELOCARANTES	SOLINGA Alimento 01/01/2024	0	0	0	1.7 semana(s)	0	\$0	---	
200061 SOLINGA SOLUCION OVAL 1.500ML/1000ML ALIMENTO LACTEO	MELOCARANTES	SOLINGA Alimento 01/01/2024	0	0	0	1.7 semana(s)	0	\$0	---	
700245 SOLINGA SOLUCION OVAL 1.500ML/1000ML ALIMENTO LACTEO	MELOCARANTES	SOLINGA Alimento 01/01/2024	0	0	0	0.6 semana(s)	0	\$987	---	
700441 SOLINGA SOLUCION OVAL 1.500ML/1000ML ALIMENTO LACTEO	MELOCARANTES	SOLINGA Alimento 01/01/2024	7	1	14	1.7 semana(s)	4	\$30.072	\$22.094	
700605 SOLINGA SOLUCION OVAL 1.500ML/1000ML ALIMENTO LACTEO	MELOCARANTES	SOLINGA Alimento 01/01/2024	0	0	0	1.0 semana(s)	0	\$0	---	

Mostrando 1 a 10 de 200 productos

1 2 3 4 5 ... 20

(b) Alertas de reposición

La gestión de pedidos se puede ver en tres vistas: los pedidos sugeridos pendientes de confirmar, el historial de pedidos ya confirmados, y los próximos pedidos estimados según el calendario de reposición.

(a) Lista de Pedidos

(b) Pedidos confirmados

Próximos pedidos (siguientes 90 días)
Fecha estimada en la que cada producto va a llegar a su punto de reorden, calculada a partir del inventario actual y el ritmo de consumo pronosticado – todavía no hay que pedirlos, pero conviene tenerlos en el radar.

Mostrar 10 productos Buscar:

PRODUCTO	CATEGORÍA	FECHA ESTIMADA DE PEDIDO	DÍAS RESTANTES	INVENTARIO ACTUAL	PUNTO DE REORDEN
0002101 OMEPRAZOL CÁPSULA 20MG	MEDICAMENTOS	2022-06-02	1	481.0	413.93
0004721 OXACILINA POLVO LIOFILIZADO 10	MEDICAMENTOS	2022-07-18	47	21.0	6.0
0006422 OXIMETAZOLINA SOLUCIÓN NASAL 0,05%	MEDICAMENTOS	2022-07-10	39	2.0	0.54
103662 PALIVIZUMAB POLVO LIOFILIZADO 50 MG	MEDICAMENTOS	2022-06-24	23	2.0	0.52
118494 DEXAMETASONA SOLUCIÓN OFTÁLMICA 5ML	MEDICAMENTOS	2022-07-29	59	3.0	0.54
120804 CARBOXIMETILCELULOSA 0.5 % SOL. OFTÁLMICA	MEDICAMENTOS	2022-08-10	70	1.0	0.27
131059					

Fig. 21. Próximos pedidos

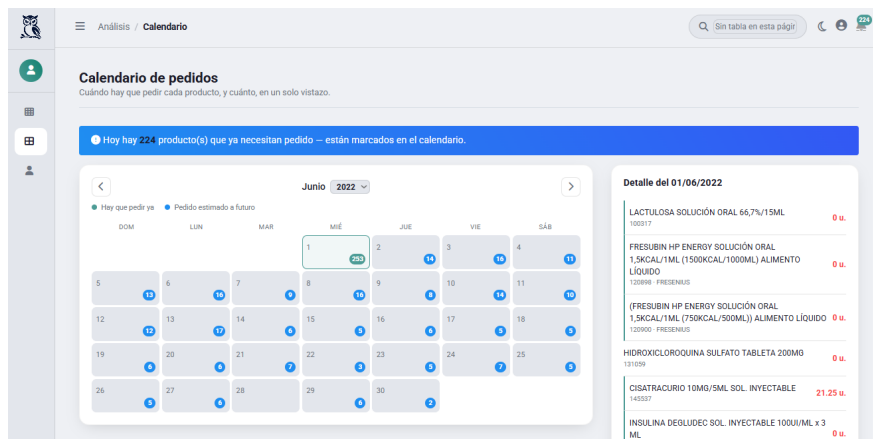


Fig. 22. Calendario de pedidos

Una vez finalizadas sus tareas, los usuarios pueden cerrar sesión en su cuenta.

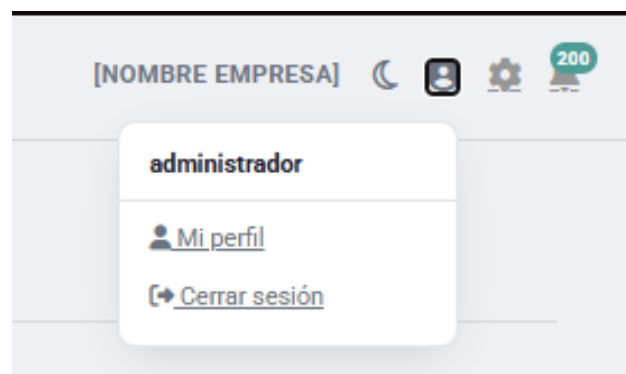


Fig. 23. Cerrar sesión

Los criterios de usabilidad que fueron tomados en cuenta fueron:

- Uso de alertas visuales codificadas por color para indicar fácilmente que productos se encuentran en o por debajo de su punto de reposición y requieren ser reabastecidos inmediatamente.

Inventario por producto
Estado de cada producto respecto a su punto de reposición. "Puede ya" significa que el inventario actual ya está en o por debajo del punto de reposición calculado.

Mostrar 10 productos Buscar:

PRODUCTO	CATEGORÍA	ABO	PROVEEDOR	INVENTARIO ACTUAL	PUNTO DE REPOSICIÓN	VALOR EN STOCK	ESTADO
0002101 OMEPRAZOL CÁPSULA 20MG	MEDICAMENTOS	A	—	481.0	413.93	\$17,797	OK
0004721 OXACOLINA POLVO LIOFILIZADO 10	MEDICAMENTOS	C	—	21.0	6.0	\$22,617	OK
0006422 OXIMETAZOLINA SOLUCIÓN NASAL 0.05%	MEDICAMENTOS	C	—	2.0	0.54	\$4,130	OK
1005917 LACTULOSA SOLUCIÓN ORAL 66.7%/15ML	MEDICAMENTOS	B	—	43.0	46.15	\$45,537	Puede ya

Fig. 24. Alertas visuales

- Cuando un usuario con el rol de empleado inicie sesión, pero su respectivo administrador no subió ningún archivo csv, en vez de ver la pantalla del dashboard, verá una pantalla con el mensaje de que una vez el admin cargue el archivo, podrá ver los análisis.

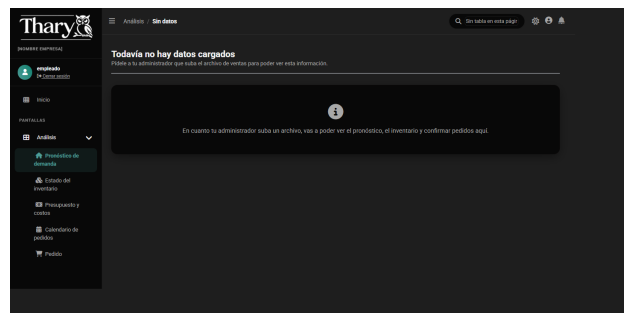


Fig. 25. Dashboard del empleado

- El sistema cuenta con un modo oscuro y modo claro para que el usuario pueda escoger según sus preferencias, mejorando la usabilidad ofreciendo ventajas como comodidad visual, menor fatiga en uso prolongado, contraste, etc.

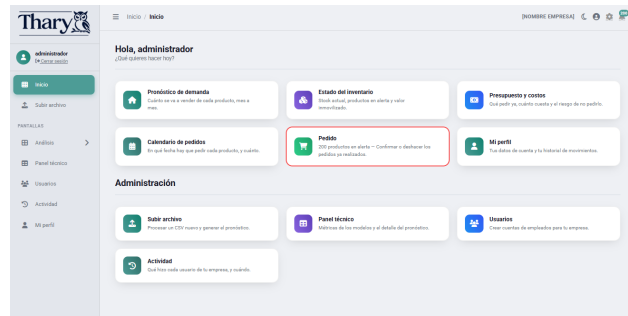


Fig. 26. Dashboard en modo claro

- Thary cuenta con una versión mobile, la cual le permite al usuario poder acceder a todas las funcionalidades de la versión de escritorio desde su teléfono móvil.

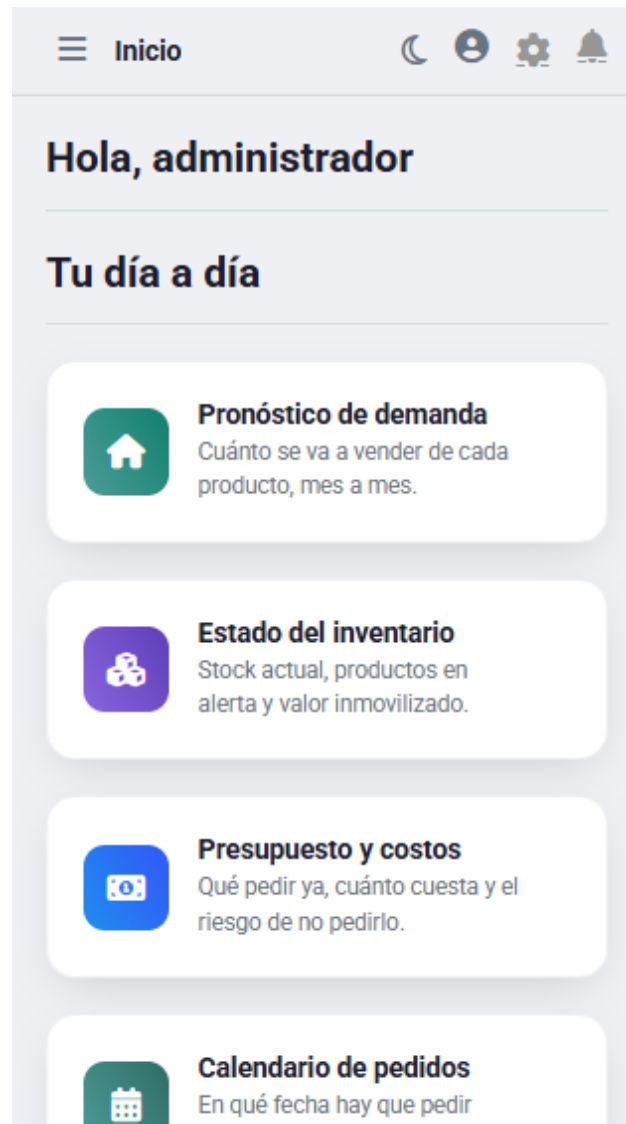


Fig. 27. Dashboard en modo mobile

En cuanto a criterios de accesibilidad, se tomaron en cuenta los criterios de usabilidad, pero no se tomaron en cuenta mecanismos adicionales para hacer más accesible el sistema como algún tipo de lector de pantalla para personas con discapacidad visual, o implementar un sistema de navegación completa por teclado para alguna persona que no pueda usar un mouse. Estas falencias se reconocen como una limitación del diseño actual, y se espera mejorarlas en el futuro.

3.2.19 Análisis de Amenazas y Diseño Seguro

Debido a su naturaleza, Thary es un sistema que maneja información que podría considerarse potencialmente sensible, como los datos de los usuarios, de consumo, entre otros. Teniendo esto en cuenta, se realizó un análisis en el que se identificaron las amenazas siguiendo el modelado de

amenazas STRIDE:

- **Suplantación de identidad:** Mitigada mediante autenticación basada en usuario y contraseña, almacenando esta última mediante un algoritmo de hash (*scrypt*, el método por defecto de `werkzeug.security`) y nunca en texto plano. Adicionalmente, la clave que usa Flask para firmar las cookies de sesión (`SECRET_KEY`) se lee desde una variable de entorno, en vez de estar escrita directamente en el código fuente, ya que si alguien tuviera acceso al código y esta clave estuviera ahí escrita, podría fabricar una sesión válida sin necesidad de conocer ninguna contraseña. Es por esto que, sin esta variable configurada, la aplicación no arranca.

Código 34. Función `crear_usuario`, que almacena la contraseña como hash.

```
def crear_usuario(nombre, username, email, password, rol="empl
#...
password_hash = generate_password_hash(password)
```

Código 35. Función `verificar_credenciales`, que compara la contraseña ingresada con su hash.

```
def verificar_credenciales(username_o_email, password):
#...
if not check_password_hash(usuario["password_hash"], password):
    return None
return usuario
```

- **Manipulación de datos:** Para evitar la manipulación de los datos, se garantiza que la cantidad y el costo de un pedido confirmado se tome siempre del análisis generado por el sistema, y no de valores que el usuario pueda modificar en el formulario de confirmación.

Código 36. Función `pedido_confirmar`, que registra los pedidos con la cantidad y el costo calculados por el sistema.

```
def pedido_confirmar():
#...
for producto_id in seleccionados:
    fila = productos_por_id.get(producto_id)
    if fila is None:
        continue
    proveedor = request.form.get(f"proveedor_{producto_id}")
    registrar_pedido(
```

```

        empresa_id=session["empresa_id"],
        producto_id=producto_id,
        proveedor=proveedor,
        cantidad=fila.get("cantidad_sugerida_pedido"),
        costo=fila.get("costo_estimado_pedido"),
        usuario=usuario,
    )

```

- **Repudio:** El sistema Thary cuenta con una tabla en la base de datos llamada "Actividad", en la cual se lleva un registro de todas las acciones relevantes que los usuarios realizan en el sistema, guardando información como el nombre del usuario, fecha y hora del evento, y la acción que fue realizada.

Código 37. Registro de la actividad del usuario al confirmar un pedido (función `pedido_confirmar`).

```

def pedido_confirmar():
    if confirmados_ahora:
        lista = ", ".join(seleccionados[:5]) + ("..." if len(seleccionados) > 5 else "")
        registrar_actividad(
            session["empresa_id"], session.get("username"), usuario,
            "pedido_confirmado", f"Confirmó {confirmados_ahora} pedidos"
        )

```

- **Divulgación de información:** Se evita que exista divulgación de información mediante el aislamiento de los datos por empresa, de forma que ninguna organización pueda acceder a los datos de otra.

Código 38. Función `carpeta_uploads`, que aísla los archivos cargados por cada empresa.

```

def carpeta_uploads(empresa_id):
    ruta = os.path.join(current_app.config["UPLOAD_FOLDER_BASE"], empresa_id)
    os.makedirs(ruta, exist_ok=True)
    return ruta

def carpeta_resultados(empresa_id):
    ruta = os.path.join(current_app.config["RESULTADOS_FOLDER_BASE"], empresa_id)
    os.makedirs(ruta, exist_ok=True)
    return ruta

```

- **Denegación de servicio:** Thary implementa varios controles para mitigar este riesgo. Se estableció un límite de tamaño para los archivos cargados (10 MB, mediante la configuración `MAX_CONTENT_LENGTH` de Flask). Adicionalmente, el servidor WSGI de producción (Gunicorn) se configuró con múltiples procesos de trabajo (*workers*) y un tiempo máximo de espera ampliado a 120 segundos, de forma que una solicitud de procesamiento pesada (por ejemplo, el entrenamiento de los modelos sobre un catálogo grande) ocupe únicamente uno de los procesos disponibles, sin bloquear el acceso al sistema para el resto de los usuarios. Finalmente, se incorporó un límite de tasa (*rate limiting*) mediante la librería `Flask-Limiter` sobre las rutas de inicio de sesión y registro, restringiendo a 10 solicitudes por minuto por dirección IP, mitigando intentos de fuerza bruta y saturación por solicitudes repetidas. Cabe señalar que, al usar Gunicorn con múltiples *workers* y un almacenamiento de límite de tasa en memoria del propio proceso, este límite se aplica de forma independiente por cada worker y no de forma global para todo el servicio.
- **Elevación de privilegios:** Para evitar este riesgo, se implementó un control de acceso basado en roles mediante el decorador `admin_required` (definido en `app/services/auth.py`), aplicado sobre las rutas de carga y procesamiento de archivos, el panel técnico de validación, la gestión de usuarios y la consulta de la actividad de toda la empresa, restringiendo estas funciones exclusivamente al perfil administrador.

3.3 Construcción, Integración y Despliegue

Thary es un sistema que se desarrollo mediante el uso de las siguientes tecnologías: Python, Flask como framework, librerías como scikit-learn, Pandas, y XGBoost.

A la hora de ejecutar el sistema en Flask, se tiene que ejecutar el comando `run.py` en un entorno virtual (*venv*), todas las dependencias necesarias para la correcta ejecución del proyecto se encuentran resumidas en el archivo `requirements.txt` para su fácil instalación.

Por su parte, el control de versiones del sistema se llevó a cabo mediante una seria de commits en GitHub en el que se documentaron los cambios más significativos del sistema durante todo el tiempo de desarrollo.

3.3.1 Construcción

La principal práctica de codificación que se siguió durante el desarrollo e implementación del sistema Thary es la separación de responsabilidades, práctica la cual se vio reflejada en la organización de archivos y carpetas del proyecto:

```
sistema_prediccion_inventario/  
  app/  
    routes/  
      auth.py
```

```

    main.py
services/
    auth.py
    db.py
    pedidos.py          # Registro de pedidos confirmados
    actividad.py        # Acciones de usuario
    prediccion/         # pipeline
        carga_datos.py  # 1. Carga y limpieza
        segmentacion.py # 2. Segmentacion ABC/variabilidad
        features.py     # 3. Construcción de variables
        modelos.py      # 4. Entrenamiento y selección
        regresion_lineal.py # 4.1. Modelo de Regresión Lineal
        validacion.py   # 5. Validación cruzada temporal
        pronostico_futuro.py # 6. Pronóstico recursivo
        reposicion.py   # 7. Puntos de reorden y EOQ
        graficas.py     # 8. Generación de gráficos
        exportacion.py  # 9. Exportación de resultados
templates/
tests/                  # pytest
    test_carga_datos.py
    test_modelos.py
    test_reposicion.py

```

Como se puede observar, la organización se divide en tres capas: `routes/` con el control del sistema, `services/` con la lógica de negocio y, dentro de esta última, `prediccion/`. Es en esa última capa, en la cual cada una de las 9 etapas del pipeline se encuentran alojadas en nueve módulos independientes: carga de datos, segmentación, construcción de variables, modelos, validación, pronóstico, reposición, gráficas y exportación. La principal ventaja de esta organización, es que permite que se pueda modificar una etapa específica del flujo sin afectar directamente a las demás partes, además de facilitar el mantenimiento y un entendimiento más fácil del código.

Como complemento a esta organización, se incorporaron pruebas automatizadas escritas con *pytest*, las cuales fueron agrupadas en tres archivos dentro de la carpeta `tests/`:

- `test_carga_datos.py`, en este archivo se verifican los mensajes de error ante archivos inválidos y el reconocimiento de columnas heterogéneas
- `test_modelos.py`, en el que se verifica la lógica de selección del modelo campeón por producto

- y por último, `test_reposicion.py`, en donde se verifican los calculos de stock de seguridad, punto de reorden y EOQ

(Los resultados obtenidos al ejecutar esta suite de pruebas se presentan en el Capítulo 4.)

3.3.2 Integración

Las rutas de la aplicación se organizan en el archivo `app/__init__.py`, mediante el mecanismo de Blueprints de Flask, el cual permite que se pueda agrupar un conjunto de vistas en un módulo independiente y registrarlo en la aplicación principal. Thary cuenta con dos blueprints: `main_bp` (dashboard, inventario, presupuesto, pedidos y panel técnico) y `auth_bp` (inicio de sesión, registro y gestión de usuarios), que se registran de la siguiente manera:

Código 39. Registro de los blueprints `main_bp` y `auth_bp` en la aplicación Flask.

```
app.register_blueprint(main_bp)
app.register_blueprint(auth_bp)
```

Esta organización permite que cada blueprint se desarrolle y se pruebe de forma independiente, y que agregar un nuevo conjunto de rutas al sistema no implique tener que modificar el código de los blueprints que ya existen.

Por otra parte, para integrar correctamente el modelo de predicción con la aplicación web, en el archivo `services/sistema_prediccion.py` se incluyó una función de entrada, `ejecutar_sistema()`, la cual se encarga de orquestar secuencialmente las nueve etapas del pipeline y devolver un resultado. Esta decisión de integración se tomó tomando en cuenta la separación de capas ya establecida para todo el proyecto. En este caso, la capa de servicios de dominio se mantiene independiente de la de presentación.

Código 40. Función `ejecutar_sistema`, que orquesta las nueve etapas del pipeline.

```
def ejecutar_sistema(
    ruta_archivo, carpeta_resultados=None, presupuesto_capital_trabajo
):
    # 1. Carga, limpieza y agregacion mensual
    df = cargar_y_limpiar(ruta_archivo)
    df_mensual = agregar_mensual(df)

    # 2. Segmentacion de productos
    producto_comportamiento = calcular_comportamiento_producto(df, df_

    # 3. Feature engineering
```

```

df_mensual, df_model, feature_cols = construir_features(df_mensual

# 4. Modelos
modelo_lr, pred_lr, features_lr = entrenar_regresion_lineal(X_train
model, pred_xgb = entrenar_xgboost(X_train, y_train_log, X_test)
pred_baseline_3 = prediccion_baseline_movil(test_df)

# 5. Validacion cruzada temporal
detalle_validacion_cruzada, resumen_validacion_cruzada = validacio

# 6. Pronostico recursivo de los proximos N meses
pronostico_futuro_df, ... = generar_pronostico_futuro(...)

# 7. Puntos de reorden y EOQ
reorder_df, ... = calcular_tabla_reorden(...)

# 8. Graficas y exportacion de artefactos
...

return {...}

```

Otro punto de integración relevante es que Thary tiene dos maneras de almacenar la información: la base de datos relacional (MySQL), para datos que sean estructurados como números o textos cortos y el sistema de archivos para archivos que por su naturaleza no se podría poner en una tabla, en el caso de Thary, estos archivos serían el archivo del modelo XGBoost ya entrenado, y las imágenes de las gráficas. Cada vez que se procesa un CSV, en MySQL se agrega una fila nueva, nunca se borra nada, pero los archivos, es decir, las gráficas, modelo entrenado, etc, van siempre a la misma carpeta de la empresa a la que pertenezcan, y cada vez que se procesa un nuevo archivo, se sobrescriben los anteriores. Así que, si se hace una serie de corridas, en MySQL habrá varias filas completas, pero en la carpeta, solo existirán los archivos de la última corrida.

app/routes/main.py:

Código 41. Función `carpeta_resultados`, que define la carpeta de resultados de cada empresa.

```
def carpeta_resultados(empresa_id):  
    """Carpeta donde ejecutar_sistema escribe el pronostico, los  
    graficos y demas archivos de resultado de una empresa puntual."""  
    ruta = os.path.join(current_app.config["RESULTADOS_FOLDER_BASE"], e  
    os.makedirs(ruta, exist_ok=True)  
    return ruta
```

La ruta de la carpeta depende del id de la empresa (`empresa_id`), y no del id de la corrida (`corrida_id`), por lo que, la carpeta siempre es la misma para una misma empresa, sin importar cuantas veces se haya procesado un archivo.

Por otra parte, tal como se explicó en capítulos anteriores, se definió un control de acceso basado en roles, esto con el objetivo de poder diferenciar a que funcionalidades debe tener acceso un usuario dependiendo de su rol. El administrador (`admin`) debe ser el único usuario que cuente con todos los privilegios de acceso a la información sensible de la organización, mientras que para otros usuarios con menor rango, (`empleado`), convendría restringir este acceso completo a solo aquellas funcionalidades menos invasivas que sean necesarias para el desempeño de su trabajo, como todas aquellas que describen el análisis de los datos y la lista de pedidos:

`app/services/auth.py`:

Código 42. Decorador `admin_required`, que restringe el acceso de una ruta al perfil administrador.

```
def admin_required(vista):  
    @wraps(vista)  
    def envoltura(*args, **kwargs):  
        if "username" not in session:  
            flash("Inicia sesión para continuar.")  
            return redirect(url_for("auth.login"))  
        if session.get("rol") != "admin":  
            flash("Esa sección es solo para el administrador.")  
            return redirect(url_for("main.dashboard"))  
        return vista(*args, **kwargs)  
    return envoltura
```

Integración continua (CI): El proyecto utilizó GitHub Actions para la automatización de pruebas unitarias (`.github/workflows/tests.yml`). Este se ejecuta automaticamente cada vez

que se haga un *push* o *pull request* en el repositorio. El archivo cuenta con una configuración que permite instalar tanto Python como todas las dependencias necesarias, y ejecuta las pruebas unitarias con `pytest`:

Código 43. Flujo de trabajo de GitHub Actions para ejecutar las pruebas automatizadas (`tests.yml`).

```
name: Tests

on:
  push:
  pull_request:

jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: "pip"

      - run: pip install -r requirements.txt

      - run: pytest tests/ -q
```

El flujo solo cubre la parte de CI (Integración Continua), en cuanto a CD (Despliegue Continuo), se decidió hacerlo de manera manual, principalmente porque al ser un proyecto desarrollado por una sola persona, es decir, sin múltiples entornos ni un equipo que dependa de constantes despliegues, el beneficio de un pipeline de CD completo es menor frente a la complejidad adicional que implicaría configurarlo.

3.3.3 Despliegue

El despliegue del sistema Thary (además del entorno de desarrollo local), se encuentra en la nube mediante la plataforma Railway.

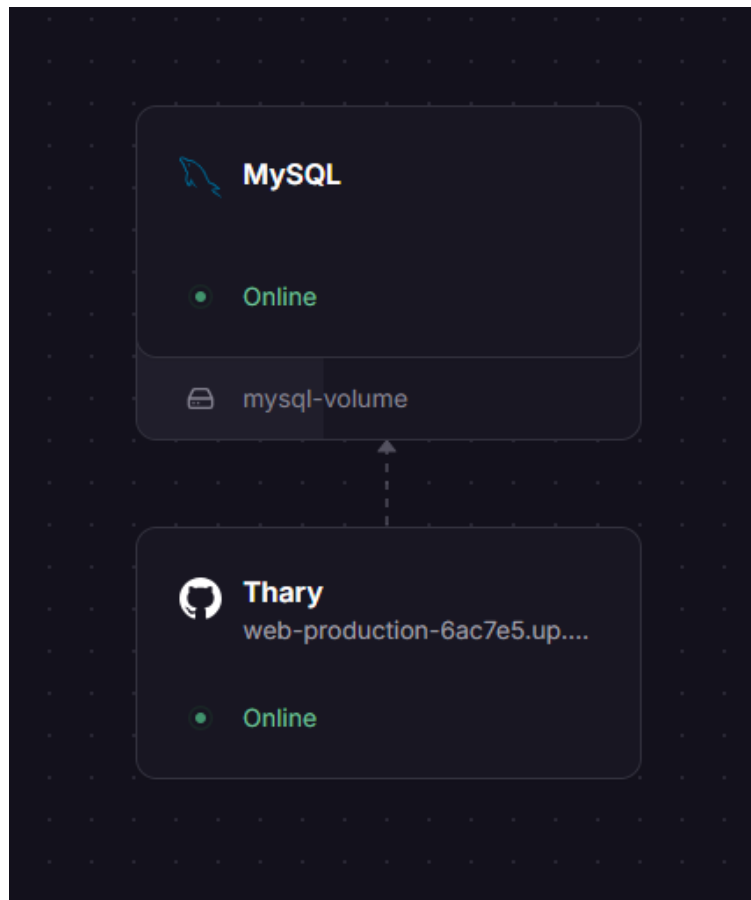


Fig. 28. Servicios desplegados en Railway: aplicación web (Thary) y base de datos MySQL.

Como se ve en la figura, Thary y la base de datos, se encuentran corriendo en Railway como servicios independientes, los cuales se comunican entre si para compartir información y funcionalidades. La aplicación se ejecuta mediante un servidor WSGI de producción (Gunicorn), configurado a través del archivo `Procfile`:

Código 44. Contenido del archivo `Procfile` para el servidor Gunicorn.

```
web: gunicorn run:app --bind 0.0.0.0:$PORT --workers 2 --timeout 120
```

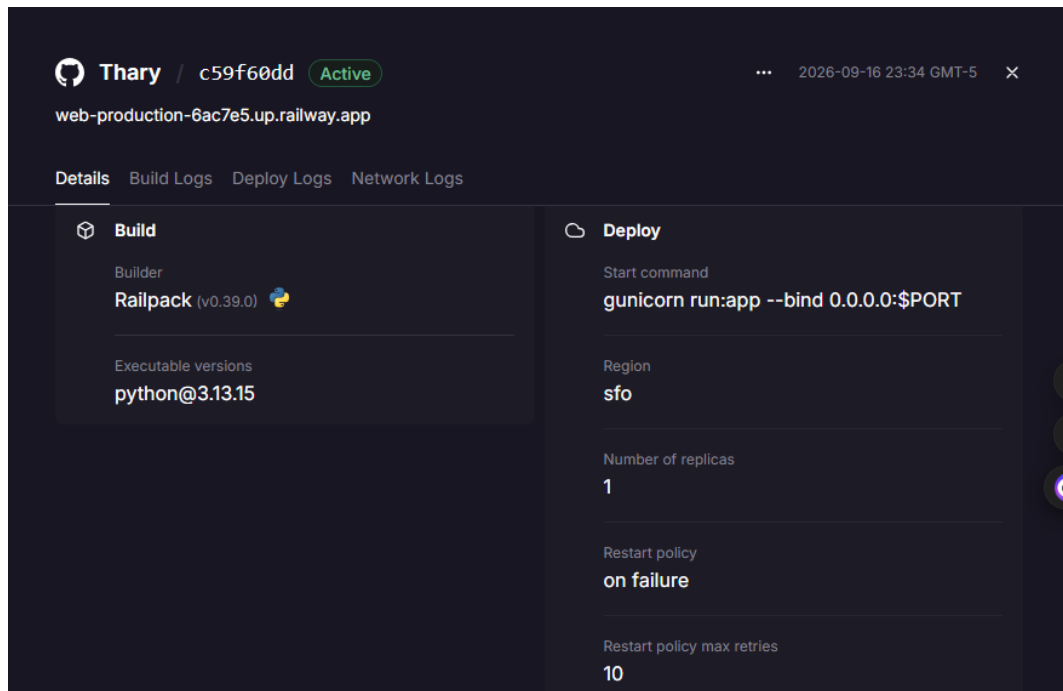


Fig. 29. Detalles de construcción y despliegue del servicio Thary en Railway.

Railway detecta y se encarga de ejecutar este comando *start command* cuando se realiza un despliegue. Automáticamente identifica la versión de python con la que se está trabajando, en este caso, `.python-version (3.13)`. El proyecto fija todas sus dependencias en `requirements.txt`, incluyendo las versiones exactas de las librerías de predicción (pandas, scikit-learn, XGBoost, statsmodels) y las necesarias para la conexión a la base de datos (SQLAlchemy, PyMySQL) y el servidor de producción (gunicorn).

En cuanto a la persistencia de los datos, esta se manejó mediante MySQL. La configuración de conexión a la base de datos solo se obtiene de las variables de entorno, tal como se detalló en la Sección 3.2.17 (función `_config()`).

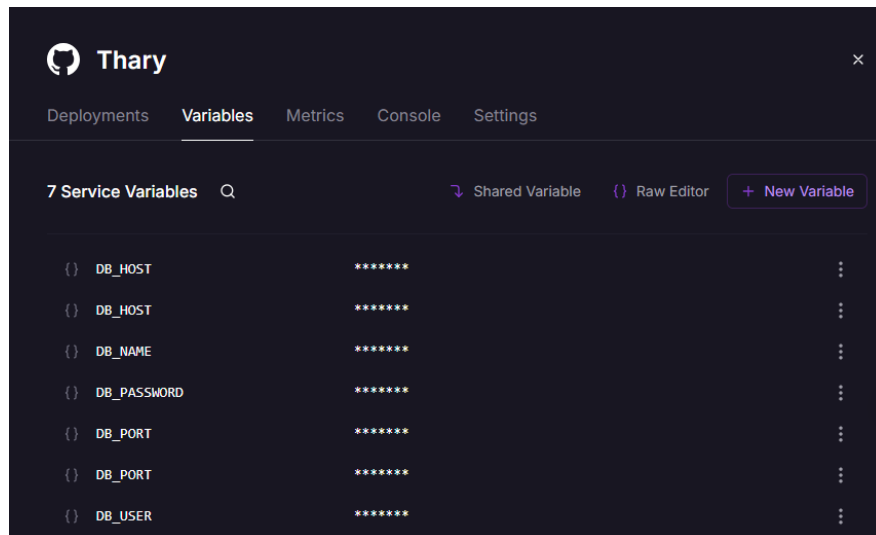


Fig. 30. Variables de entorno del servicio Thary en Railway, con los valores ocultos.

Las credenciales se configuran en el panel de Railway como variables de entorno del servicio, nunca visibles en el repositorio de código.

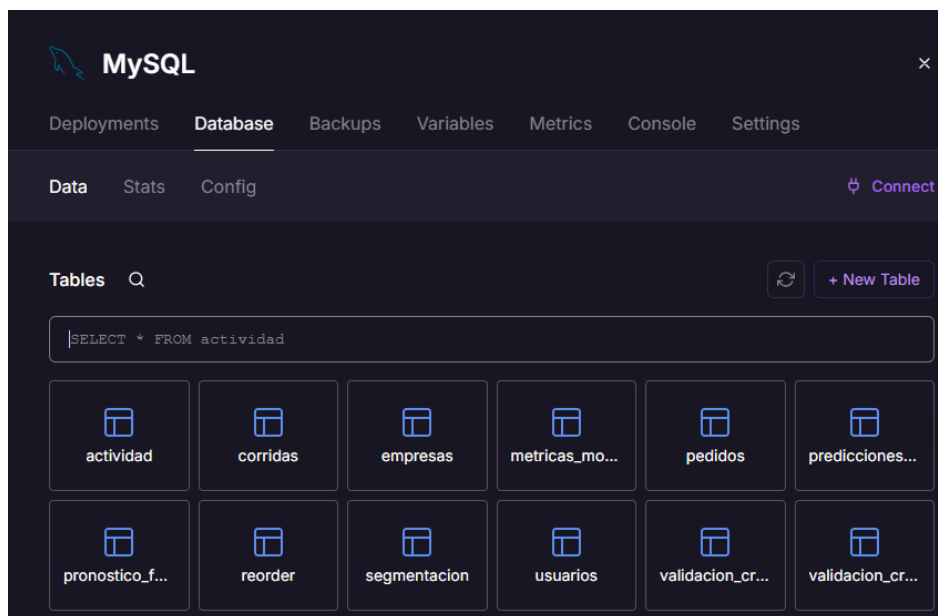


Fig. 31. Tablas del esquema relacional de Thary en el entorno de producción, vistas desde el explorador de datos de Railway.

La Fig. 31 muestra que el mismo esquema de 12 tablas que se tiene localmente, también existe en el entorno desplegado de Railway.

El servicio de Thary no cuenta con un volumen configurado, esto puede verse en el panel de

administración en el que no se ve la pestaña de "volúmenes". Esto quiere decir que los archivos que están en el servicio son efímeros, es decir, cada vez que se redespliegue el servicio, como por ejemplo, por una actualización del código o un reinicio del servicio, se crea un nuevo contenedor, por lo que se pierden por completo los archivos generados por el sistema en las corridas anteriores del pipeline. Esto se reconoce como una limitación del entorno de producción.

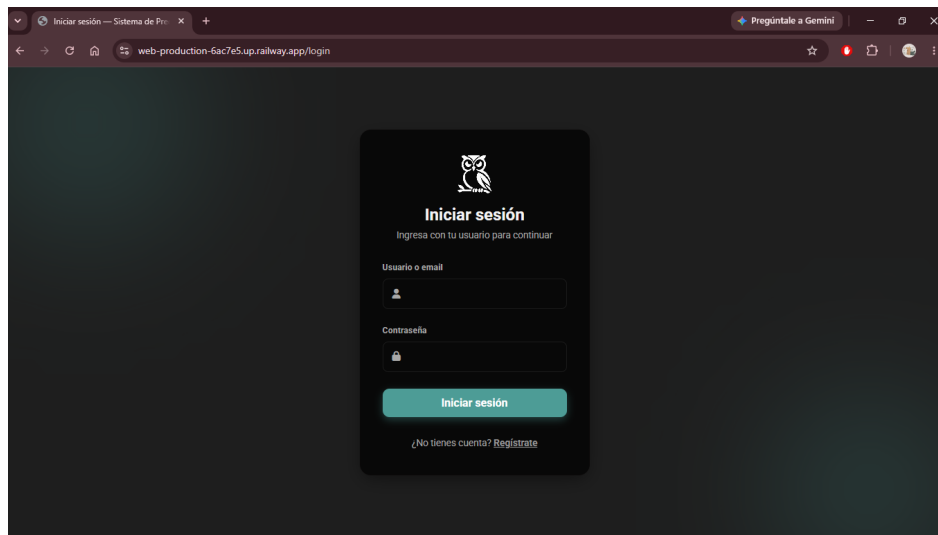


Fig. 32. Pantalla de inicio de sesión de Thary, accedida desde su URL pública en Railway.

Finalmente, la figura muestra la aplicación funcionando correctamente en el entorno de producción a través de la URL pública asignada por Railway.

3.4 Plan de Verificación y Validación

3.4.1 Estrategia de verificación

La estrategia de verificación del sistema Thary consiste la realización de pruebas mediante el uso de pytest. Las pruebas se realizan sobre aquellos componentes del sistema que de fallar, podrían comprometer la calidad y confiabilidad de las predicciones de los productos o toda la funcionalidad del sistema general. Las pruebas se implementaron en el directorio `tests` en 4 archivos diferentes cada uno enfocado en verificar el correcto funcionamiento de uno de los componentes del sistema:

- `test_carga_datos.py`: En esta prueba se verifica de que el sistema le permita a los usuarios el poder cargar archivos CSV de forma correcta. Se verifica que el sistema no se rompa al recibir archivos con datos de entrada incorrectos, incompletos o faltantes. De existir alguno de estos errores, el sistema debe mostrar un mensaje de error al usuario para cada caso en el que se le indique de forma clara que errores son los que tiene el CSV y que debe corregir

antes de cargarlo nuevamente. Por último, en esta prueba también se verifica que el sistema cumpla con el RNF-02 y que sea capaz de reconocer columnas heterogeneas (en español e ingles) y que las procese correctamente.

- `test_modelos.py`: En esta prueba se verifica que la lógica de selección del mejor modelo por producto funcione correctamente, para esto se garantiza que la validación que se hace con la prueba de Diebold-Mariano con la corrección de Harvey-Leybourne-Newbold para confirmar que la diferencia de error entre modelos es realmente significativa, también se ejecute sin problemas.
- `test_reposicion.py`: Durante esta prueba se verifica que los calculos de: Stock de seguridad, EOQ (calculos de stock de seguridad), punto de reorden y costo de ruptura (stockout) se realicen correctamente para posteriormente, ser mostrados al usuario en la interfaz de una manera clara y comprensible.
- `test_combinacion.py`: En esta prueba se verifica que el peso que se usa para combinar los resultados que dan los modelos XgBoost y media móvil, sean calculados correctamente. También se comprueba que no halla errores en casos como en el que las dos predicciones combinadas son identicas.

Para los calculos que tienen una única respuesta correcta (`test_reposicion.py`, `test_combinacion.py`, y `test_modelos.py`), se aplica un enfoque llamado *oraculo conocido*, el cual consiste en antes de hacer una prueba, primero calcular manualmente con base a la fórmula correspondiente, cual es el resultado correcto que el sistema debería dar, y después se verifica si el sistema entrego o no dicho resultado.

Adicional a esto, el sistema también implementó el uso de GitHub Actions (`.github/workflows/tests.yml`), para poder ejecutar todas estas pruebas de manera automática cada que se haga un commit en el repositorio donde se encuentra el sistema. De este manera, se automatiza el proceso y se evita tener que depender de que alguien tenga que ejecutar las pruebas manualmente cada cierto tiempo.

3.4.2 Estrategia de validación del modelo de predicción

En esta etapa de verificación, se busca garantizar que los resultados dados por el sistema sean confiables en comparación con los datos reales.

Teniendo en cuenta que en el sistema se está trabajando con historiales de consumo mensual, es decir, series de tiempo, se decidió implementar una estrategia de validación que respetara el orden cronológico de la información, en la que los datos más antiguos siempre son los datos de entrenamiento y los más recientes se usan como datos de prueba, los cuales no se le muestran al sistema durante el entrenamiento, ya que son estos los que posteriormente tendrá que predecir.

Es por esto que se decidió implementar un sistema de validación cruzada para los datos, en el que se repite el proceso de entrenamiento y evaluación sobre distintas ventanas de tiempo sucesivas, para verificar que el funcionamiento del sistema sea estable y consistente a lo largo del tiempo, y que no dependa de las particularidades de un periodo específico para dar buenos resultados. Si al finalizar la validación cruzada se observa que el sistema tuvo un desempeño similar en todos los periodos evaluados, se puede concluir que ese modelo es confiable; sin embargo, si ocurre lo contrario y solo tuvo un desempeño decente en un periodo específico, dicho modelo no es confiable, y solo tuvo buenos resultados en ese periodo por casualidad o por condiciones particulares del mismo.

Para cada ventana de tiempo de la validación cruzada se calculan las métricas de MAE, RMSE, Mediana del Error, R^2 , MAPE y WAPE, comparando el desempeño de los tres modelos candidatos (Media Móvil, Regresión Lineal y XGBoost) entre sí.

Adicionalmente, también se evalúa la sección en la que se combinan los resultados de predicción de los modelos XGBoost y Media Móvil, en el que se estima si el promedio de ambos modelos reduce el error en comparación a usar ambos modelos por separado. Esto permite determinar si la decisión de implementar un sistema con tres modelos de predicción, y escoger el de mejor desempeño para cada producto, aporta valor en comparación con simplemente usar un solo modelo para todos los productos.

En cada evaluación varía el tamaño de prueba (función `tamano_test_backtest`), si se cuenta con un historial de consumo con una cantidad de datos más amplia, se puede usar un periodo de prueba más grande. Esto también es conveniente a la hora de evaluar la prueba de Diebold-Mariano correspondiente a la selección del mejor modelo, ya que esta prueba ayuda a determinar si la selección de dicho modelo se dio porque de verdad es el modelo con mejores resultados o si la diferencia que tiene con el segundo mejor modelo podría tratarse solo de ruido.

En general, la estrategia de selección del modelo ganador aplica parte de la estrategia de validación, ya que solo se puede considerar un modelo como ganador para un producto si existe suficiente evidencia estadística para afirmar que su ventaja en comparación con los demás modelos es real.

3.4.3 Criterios de aceptación

El principal criterio de aceptación que se establece para Thary es que el componente principal, es decir, el método de selección del mejor modelo por producto, debe tener un mejor resultado que la Media Móvil aplicada a todo el catálogo, y esto debe cumplirse en todos los periodos evaluados de la validación cruzada, no solo en el promedio general.

Esta comparación se hace con la métrica WAPE, promediada sobre los distintos periodos de la validación cruzada. Se eligió esta métrica porque representa el error total del pronóstico como porcentaje de la demanda total, y porque no se infla con productos de demanda baja, como si le ocurre al MAPE, que además no se puede calcular cuando la demanda es cero.

Otro factor que se tiene en cuenta para el cumplimiento del criterio es que, cuando se hace la

selección del modelo ganador para cada producto, se selecciona con base en cual de los tres cuenta con el menor MAE en el periodo de prueba. Una vez hecho esto, se aplica la prueba de Diebold-Mariano con la corrección de Harvey-Leybourne-Newbold, para verificar que su ventaja no sea por ruido en los datos o otros factores externos. Si no se encuentra suficiente evidencia de que la ventaja es real, se usa Media Móvil como respaldo en vez de aceptar ese resultado.

Adicionalmente, también se definieron los siguientes criterios generales de aceptación para el sistema:

- Todas las pruebas automatizadas hechas con Pytest deben pasar sin errores, tanto en la computadora local como en GitHub Actions.
- El flujo completo de Thary: cargar un archivo, procesarlo, ver los resultados y confirmar un pedido, debe poder hacerse de principio a fin sin errores.
- El sistema debe poder generar una predicción incluso si un producto tiene poco historial (menos de 4 meses), usando media móvil como método de respaldo en vez de fallar o excluir ese producto.

3.4.4 Plan de validación con usuarios

El plan de validación de usuarios se incluyó como otra validación a la verificación del sistema en la que se busca poder evaluar si Thary es un sistema intuitivo y fácil de usar para los usuarios finales, y que la información sea fácil de comprender e interpretar, de manera que esta le ayude al usuario a tomar decisiones.

La validación se realiza para los dos perfiles de usuario implementados: Administrador y Personal de Inventario.

Para su realización, se va a determinar si el usuario puede completar el flujo completo de uso del sistema sin asistencia adicional más allá de la que brinda la propia interfaz:

- **Administrador:** inicia sesión, carga y procesa el historial de consumo, consulta el panel técnico de comparación de modelos, crea una cuenta de personal de inventario, y revisa la bitacora de actividad de su empresa.
- **Personal de Inventario:** inicia sesión, consulta el dashboard de demanda pronosticada, el estado del inventario, el calendario de pedidos y el presupuesto disponible, confirma un pedido sugerido a partir de las recomendaciones generadas, y revierte dicho pedido para verificar que la acción se pueda deshacer correctamente. Finalmente, consulta su propio perfil.

Durante este recorrido se observa si el usuario logra completar cada tarea sin asistencia adicional a la propia interfaz, prestando especial atención a los siguientes aspectos:

- Si las alertas de reposición y las recomendaciones de compra son comprensibles sin necesidad de conocimientos técnicos de modelado predictivo.
- Si el usuario logra distinguir con claridad las pantallas y funcionalidades disponibles según su rol.
- Si el flujo de confirmar y, de ser necesario, deshacer un pedido resulta intuitivo y sin ambigüedad sobre el estado final del pedido.
- Si la información del calendario de pedidos y del costo de ruptura (stockout) es interpretada correctamente como una alerta de urgencia, y no como un dato meramente informativo.

Al finalizar el recorrido, la percepción del usuario se recopila mediante una breve conversación en la que se le pregunta si encontró alguna pantalla o mensaje confuso y si considera que la información presentada sería útil para tomar decisiones reales de abastecimiento en su contexto laboral. Los resultados de esta validación se documentan en el Capítulo 4.

4 Capítulo 4: Análisis de Resultados y Juicio de Ingeniería

4.1 Resultados de Verificación (Pruebas de Software)

La verificación del sistema de Thary se llevó a cabo mediante pruebas automatizadas escritas con *pytest*, ya explicadas en la Sección 3.4. Los resultados de estas pruebas se obtuvieron ejecutando el comando `pytest tests/ -v` en la terminal.

```
tests/test_carga_datos.py::test_archivo_vacio_da_mensaje_claro PASSED [ 2%]
tests/test_carga_datos.py::test_archivo_solo_con_encabezados_da_mensaje_claro PASSED [ 4%]
tests/test_carga_datos.py::test_falta_columna_obligatoria_la_nombre_en_el_mensaje PASSED [ 6%]
tests/test_carga_datos.py::test_acepta_nombres_de_columna_en_ingles PASSED [ 9%]
tests/test_carga_datos.py::test_reconoce_la_columna_precio_en_espanol PASSED [ 11%]
tests/test_carga_datos.py::test_fechas_y_demanda_invalidas_en_todas_las_filas PASSED [ 13%]
tests/test_carga_datos.py::test_algunas_filas_invalidas_se_descartan_pero_el_resto_se_procesa PASSED [ 16%]
tests/test_carga_datos.py::test_acepta_codigo_y_cantidad_como_nombres_de_columna PASSED [ 18%]
tests/test_carga_datos.py::test_producto_es_el_nombre_si_el_archivo_trae_un_codigo PASSED [ 20%]
tests/test_carga_datos.py::test_producto_solo_sigue_siendo_el_codigo PASSED [ 23%]
tests/test_carga_datos.py::test_reconoce_tiempo_de_entrega_y_lead_time PASSED [ 25%]
tests/test_carga_datos.py::test_encabezados_sin_importar_mayusculas_ni_tildes PASSED [ 27%]
tests/test_carga_datos.py::test_si_hay_dos_columnas_de_codigo_gana_product_id PASSED [ 30%]
tests/test_combinacion.py::test_peso_todo_para_el_modelo_perfecto PASSED [ 32%]
tests/test_combinacion.py::test_peso_cero_si_el_otro_modelo_es_el_perfecto PASSED [ 34%]
tests/test_combinacion.py::test_peso_a_medias_si_se_equivocan_igual_pero_para_lados_opuestos PASSED [ 37%]
tests/test_combinacion.py::test_peso_se_recorta_a_cero_uno PASSED [ 39%]
tests/test_combinacion.py::test_peso_con_predicciones_identicas_es_un_medio PASSED [ 41%]
tests/test_combinacion.py::test_evaluar_combinacion_devuelve_las_tres OPCIONES_y_el_peso PASSED [ 44%]
tests/test_combinacion.py::test_evaluar_combinacion_sin_predicciones_devuelve_none PASSED [ 46%]
tests/test_modelos.py::test_tamano_test_backtest_archivo_corto_usa_default PASSED [ 48%]
tests/test_modelos.py::test_tamano_test_backtest_justo_en_el_umbral_usa_test_grande PASSED [ 51%]
tests/test_modelos.py::test_tamano_test_backtest_un_mes_antes_del_umbral_usa_default PASSED [ 53%]
tests/test_modelos.py::test_elegir_campeon_por_producto_diferencia_no_significativa_usa_media_movil PASSED [ 55%]
tests/test_modelos.py::test_elegir_campeon_por_producto_diferencia_clara_y_sostenida_si_gana PASSED [ 58%]
tests/test_modelos.py::test_elegir_campeon_por_producto_empate_exacto_usa_media_movil PASSED [ 60%]
tests/test_modelos.py::test_seleccionar_prediccion_final PASSED [ 62%]
tests/test_modelos.py::test_seleccionar_prediccion_final_cubre_los_tres_metodos PASSED [ 65%]
tests/test_reposicion.py::test_presupuesto_se_reparte_en_partes_iguales PASSED [ 67%]
tests/test_reposicion.py::TestA100StockDeSeguridadYPuntoDeReorden::test_stock_de_seguridad PASSED [ 69%]
tests/test_reposicion.py::TestA100StockDeSeguridadYPuntoDeReorden::test_punto_de_reorden PASSED [ 72%]
tests/test_reposicion.py::TestA100StockDeSeguridadYPuntoDeReorden::test_sin_datos_de_costo_no_hay_eoq_y_cae_al_metodo_alternativo PASSED [ 74%]
tests/test_reposicion.py::TestA100StockDeSeguridadYPuntoDeReorden::test_alerta_de_pedido_cuando_inventario_esta_por_debajo_del_punto_de_reorden PASSED [ 76%]
tests/test_reposicion.py::TestB200EOQSinLimiteDePresupuesto::test_eoq PASSED [ 79%]
tests/test_reposicion.py::TestB200EOQSinLimiteDePresupuesto::test_cantidad_sugerida_es_el_eoq_completo_sin_recortar PASSED [ 81%]
tests/test_reposicion.py::TestB200EOQSinLimiteDePresupuesto::test_costo_estimado_del_pedido PASSED [ 83%]
tests/test_reposicion.py::TestB200EOQSinLimiteDePresupuesto::test_valor_del_inventario_actual PASSED [ 86%]
tests/test_reposicion.py::TestB200EOQSinLimiteDePresupuesto::test_no_hace_falta_pedir_todavia PASSED [ 88%]
tests/test_reposicion.py::TestB200EOQSinLimiteDePresupuesto::test_costo_de_ruptura_estimado PASSED [ 90%]
tests/test_reposicion.py::TestC300EOQLimitadoPorElPresupuesto::test_eoq_ideal_es_mayor_al_limite_de_presupuesto PASSED [ 93%]
tests/test_reposicion.py::TestC300EOQLimitadoPorElPresupuesto::test_cantidad_sugerida_queda_acotada_por_el_presupuesto PASSED [ 95%]
tests/test_reposicion.py::TestC300EOQLimitadoPorElPresupuesto::test_sin_datos_de_inventario_no_hay_alerta_ni_fecha_estimada PASSED [ 97%]
tests/test_reposicion.py::TestC300EOQLimitadoPorElPresupuesto::test_sin_precio_no_hay_costo_de_ruptura PASSED [ 100%]
```

Fig. 33. Ejecución detallada de la suite de pruebas mediante `pytest tests/ -v`, mostrando cada prueba individual.

TABLA IX
Resultados de la suite de pruebas automatizadas por archivo.

Archivo	Componente verificado	Resultado
test_carga_datos.py	Carga y limpieza de datos, incluyendo el reconocimiento de columnas heterogeneas	13/13 exitosas
test_combinacion.py	Combinación de pronósticos	7/7 exitosas
test_modelos.py	Selección del modelo campeón y prueba Diebold-Mariano	8/8 exitosas
test_reposicion.py	Calculo de reposición (stock de seguridad, punto de reorden, EOQ, costo de ruptura)	15/15 exitosas
Total		43/43 exitosas

```
(venv) PS C:\Users\Home\Desktop\proyecto_tesis\sistema_prediccion_inventario> pytest tests/ -q
..... [100%]
===== warnings summary =====
tests/test_carga_datos.py::test_fechas_y_demanda_invalidas_en_todas_las_filas
  C:\Users\Home\Desktop\proyecto_tesis\sistema_prediccion_inventario\app\services\prediccion\carga_datos.py:131: UserWarning: Could not infer format, so each element will be parsed individually, falling back to 'dateutil'. To ensure parsing is consistent and as-expected, please specify a format.
    df["fecha"] = pd.to_datetime(df["fecha"], errors="coerce")

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
43 passed, 1 warning in 5.56s
```

Fig. 34. Resumen final de la ejecución mediante `pytest tests/ -q`.

Como se puede ver en la Fig. 33, la Tabla IX y la Fig. 34, las 43 pruebas se ejecutaron correctamente en un tiempo de 5.56 segundos. Se registro una única advertencia (*warning*) relacionada con el formato de fechas de pandas al procesar un archivo de prueba con fechas invalidas a proposito, sin que esto afecte el resultado de la prueba.

Se detecto un bug menor en el que la columna de "Precio" en español no era reconocida por el sistema a pesar de que la interfaz de carga la anunciaba como aceptada. Se arreglo el problema y dentro de `test_carga_datos.py` se incluyó la prueba `test_reconoce_la_columna_precio_en_espanol`, cuyo resultado exitoso confirma que la corrección aplicada sigue vigente.

El mecanismo de reconocimiento de columnas se reforzó con seis pruebas adicionales que verifican, entre otros casos: que el sistema ignore mayusculas y tildes en los encabezados, que resuelva correctamente la ambigüedad entre "Producto" como código o como nombre del producto (según si el archivo ya trae una columna de código clara), y que priorice la columna correcta cuando un archivo trae más de un candidato para el mismo destino (por ejemplo, un "Id" de transaccion junto a un "Product_ID" real). Además de las pruebas automáticas, se revisó de forma manual que la función principal de Thary se hiciera bien de principio a fin. Es decir, que el sistema dejara cargar

un archivo, procesarlo, mostrar los resultados y confirmar un pedido sin errores. Las etapas de segmentación de productos, construcción de variables, y las rutas y autenticación de la aplicación web no tienen pruebas automáticas. Esto se reconoce como una limitación de la revisión.

4.2 Resultados de la Validación del Modelo de Predicción

A continuación, se presentan los resultados obtenidos en la validación del sistema Thary. Estos resultados se obtuvieron luego de aplicar la estrategia de validación ya definida anteriormente, sobre el historial real de consumo de 856 productos (medicamentos) del año 2021 - 2022 en la Clínica Nuestra Señora de los remedios.

Las métricas obtenidas al evaluar los tres modelos candidatos, junto con el método de selección de modelo por producto. Todo esto sobre el último periodo de prueba disponible, que corresponde al trimestre abril-junio de 2022, con seis meses de entrenamiento previo.

TABLA X
Métricas del holdout final por modelo.

Modelo	MAE	RMSE	Mediana Error	MAPE (%)	WAPE (%)
Media Móvil (3 meses)	48.79	243.65	5.33	80.32	23.86
Regresión Lineal	56.43	276.41	5.25	95.30	27.60
XGBoost (log1p)	50.59	254.82	3.53	65.12	24.74
Sistema final (selección por producto)	48.47	243.55	—	79.33	23.71

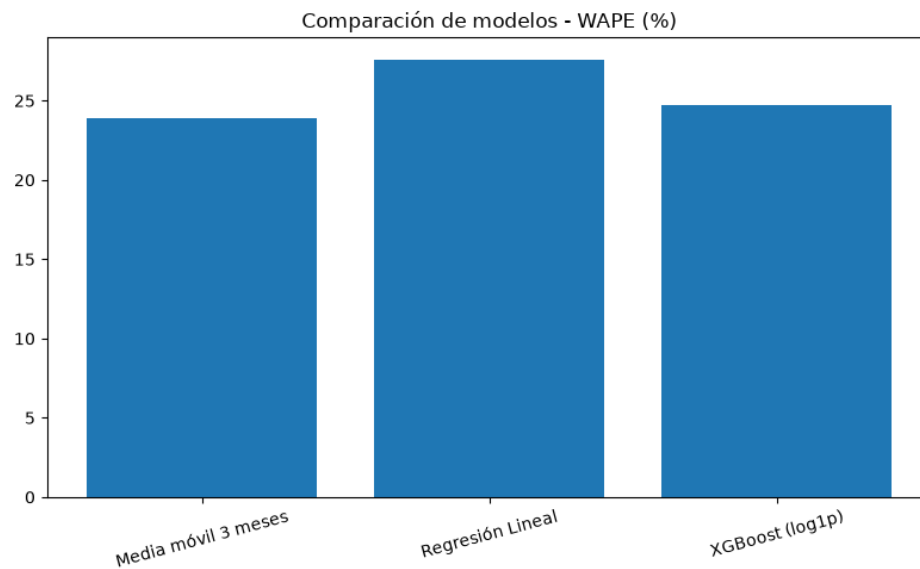


Fig. 35. Comparación de WAPE entre los tres modelos candidatos (holdout final).

4.2.1 Consistencia mediante validación cruzada temporal

Después de obtener los resultados anteriores, se propuso el uso de una validación cruzada para descartar que el resultado obtenido dependiera de las condiciones y particularidades de un único periodo de prueba.

La idea principal, es probar el modelo varias veces simulando estar en un mes distinto al pasado, para poder evaluar si acierta de forma consistente y no solo una vez por casualidad. Para poder predecir un mes, solo se puede usar los meses que vinieron antes, no se pueden usar los datos que serían "Futuros", porque queremos simular un escenario real, y en ese caso, no se tendría esa información.

1. Prueba 1 (Fold 1): Es el más antiguo, se le enseña al modelo con los 4 meses previos a febrero de 2022, y se hace que el sistema adivine febrero, marzo y abril de 2022. Se compara lo que adivino contra los resultados reales de esos meses y se registra que tan grande fue el error.
2. Prueba 2 (Fold 2): En esta segunda prueba o Fold, se le enseña al modelo con 5 meses en vez de 4, es decir, los meses previos a marzo de 2022, y se le pide que prediga marzo, abril y mayo de 2022, una vez más, se compara y se registra el error.
3. Prueba 3 (Fold 3): Por último, se le enseña al modelo con los 6 meses previos a abril de 2022 y se le solicita al sistema que prediga abril, mayo y junio de 2022, se compara y, una vez más, se registra el error. Este último fold corresponde al mismo holdout final presentado en la Tabla X.

Dicha validación cruzada temporal se implementó en el sistema en el archivo `app/services/prediccion/validacion.py`.

Código 45. Constantes y función `_armar_folds` de la validación cruzada temporal.

```
TEST_SIZE_MESES = 3
MIN_TRAIN_MESES = 3
MAX_FOLDS = 3

def _armar_folds(meses_ordenados):
    n = len(meses_ordenados)
    folds = []
    fin_test = n
    while len(folds) < MAX_FOLDS:
        inicio_test = fin_test - TEST_SIZE_MESES
        if inicio_test < MIN_TRAIN_MESES:
            break
        folds.append((meses_ordenados[:inicio_test], meses_ordenados[inicio_test:fin_test]))
        fin_test -= 1
    folds.reverse()
    return folds
```

Se definen tres constantes, `TEST_SIZE_MESES`, `MIN_TRAIN_MESES` y `MAX_FOLDS`, que definen el tamaño de la ventana de prueba, el tamaño mínimo de la ventana de entrenamiento y el número máximo de folds a generar, respectivamente.

Se arma la función que genera las particiones `_armar_folds`, esta recibe una lista de todos los meses disponibles organizados cronológicamente. Se define el fold más reciente, es decir, los últimos meses disponibles `TEST_SIZE_MESES`. En cada iteración se retrocede el punto de corte de un mes hacia atrás en el tiempo, de esta manera, se crea un nuevo Fold cuyo entrenamiento crece en un mes respecto al anterior. Este proceso se repite hasta completar `MAX_FOLDS`, o hasta que ya no quede suficiente historial y el tamaño de la ventana de entrenamiento sea menor a `MIN_TRAIN_MESES` meses de entrenamiento.

Al finalizar, los folds se ordenan en orden cronológico del más antiguo al más nuevo antes de ser devueltos.

Al ya tener los folds organizados, el sistema entrena nuevamente los tres modelos candidatos, y selecciona el mejor modelo por producto, evaluando el resultado bajo la etiqueta de "Sistema final". Es decir, es la estrategia que usa Thary para predecir el consumo, en vez de escoger uno solo, para

cada producto, el sistema entrena los 3 modelos y selecciona el que se equivocó menos con ese producto en particular.

Un aspecto importante es que, la elección de qué modelo usar para cada producto se hace solo con los datos de entrenamiento, antes de ver los datos de prueba. Esto demuestra que no se está "haciendo trampa", ya que el sistema no tiene acceso a los datos de prueba, y por lo tanto, no puede "aprender" de ellos.

Aun así, solo esto no basta para confiar en el resultado, ya que aun si el sistema está eligiendo el modelo de forma honesta, un único experimento podría haber salido bien por casualidad y no porque el sistema de verdad sea bueno.

Es por eso que la validación cruzada aportó valor al sistema, ya que al repetir este mismo proceso (entrenamiento, elección del modelo y evaluación) en diferentes periodos de prueba, se confirma que el resultado obtenido en el holdout final no fue un resultado aislado, sino que se repite de forma consistente en distintos periodos de prueba.

En la siguiente tabla se describen el promedio y la desviación estándar de las métricas obtenidas en cada fold, para cada modelo candidato y el sistema final:

TABLA XI

Resumen de la validación cruzada temporal (promedio $\pm$ desviación, 3 folds).

Modelo	MAE	RMSE	MAPE (%)	WAPE (%)
Media Móvil	44.27 $\pm$ 3.70	200.82 $\pm$ 41.31	82.38 $\pm$ 1.85	22.31 $\pm$ 1.29
Regresión Lineal	58.13 $\pm$ 2.21	270.35 $\pm$ 6.32	106.24 $\pm$ 10.64	29.38 $\pm$ 1.84
XGBoost	46.58 $\pm$ 3.46	223.64 $\pm$ 31.40	64.93 $\pm$ 2.25	23.48 $\pm$ 1.15
Sistema final	43.42 $\pm$ 3.77	190.50 $\pm$ 40.63	81.13 $\pm$ 1.56	21.88 $\pm$ 1.34

De la tabla se puede observar que:

- El sistema final es el que obtiene el mejor promedio en las cuatro métricas evaluadas. En cuanto a la estabilidad entre folds, presenta la menor desviación estándar en RMSE y MAPE, aunque una desviación ligeramente mayor a la de la Media Móvil en MAE y WAPE. Esto es significativo, porque demuestra que vale la pena la complejidad de elegir un modelo distinto para cada producto: el sistema final es consistentemente más preciso en promedio, aunque su ventaja en estabilidad no se cumple de forma uniforme en las cuatro métricas.
- La media móvil es el segundo mejor modelo, seguido por XGBoost y finalmente la regresión lineal. Cuando se comparan los tres modelos candidatos de forma individual, la media móvil es la que obtiene un mejor MAE, RMSE y WAPE, mientras que solo XGBoost la supera en MAPE. El hecho de que ninguno de los dos modelos más sofisticados domine de forma

general sobre la línea base avala la decisión de seleccionar un modelo distinto por producto, ya que no existe un único modelo que sea el mejor en todos los casos.

- La Regresión Lineal es la que tiene los peores resultados en las cuatro métricas, además de que su MAPE es el que más varía entre los 3 folds. Esto es coherente con la tendencia de este modelo a sobreestimar los productos de alta demanda, observada previamente en el análisis de dispersión entre modelos, lo que ayuda a explicar por que su error resulta más alto que el de los otros dos modelos.
- Finalmente, se observa que la efectividad del sistema final no se debe solamente a "elegir el mejor modelo en general", que en este caso sería la media móvil, sino a elegir un modelo distinto para cada producto. En caso de haber elegido media móvil para todos los productos, el error hubiera sido de 22.31% de WAPE, mientras que al elegir el mejor modelo por producto, validado mediante la prueba de Diebold-Mariano, el error se reduce a 21.88%

4.3 Resultados de Validación (Evaluación del Usuario)

Para validar la usabilidad de Thary, se siguió el plan de validación ya definido. La realización de la retroalimentación fue hecha por el desarrollador del sistema y por una persona encargada de la gestión de inventarios en el dominio de salud, esto con el fin de obtener una perspectiva externa a la del equipo de desarrollo.

Para un usuario registrado como administrador el flujo del sistema se pudo completar sin dificultad e incluyó: inicio de sesión, carga y procesamiento del historial de consumo, consulta del dashboard de demanda pronosticada, del estado del inventario, del presupuesto y del calendario de pedidos y confirmación (y reversión) de un pedido sugerido.

Una vez completado todo el proceso, la interfaz fue percibida como intuitiva y fácil de usar, no fue necesario que existiera alguna explicación adicional más allá de la que la propia interfaz del sistema proporcionaba a la hora de completar cada una de las tareas.

Como resultado de esta validación, se aportó una mejora de la usabilidad del sistema. El gestor de inventario que probó el sistema sugirió una mejora concreta: incorporar una plantilla de archivo CSV descargable desde la propia interfaz que le sirviera a los usuarios de guía sobre el formato y columnas esperadas antes de cargar su propio histórico de consumo. Esta sugerencia se identificó como valiosa porque, si bien Thary ya estaba diseñado para aceptar distintas variantes de nombres de columna, y en la pantalla de carga se incluye un manual con los nombres de las columnas, la incorporación de una plantilla facilita aún más este proceso al mostrarle al usuario el formato esperado antes de que intente cargar su propio archivo.

Esta sugerencia se implementó en el sistema incluyendo el archivo `plantilla_thary.csv`, el cual está disponible para descarga desde la interfaz de carga de archivo.

Se reconoce que esta validación tuvo un carácter informal y con un número reducido de participantes (dos personas), no obstante, al incluir a una persona con experiencia real en gestión de inventario en el dominio de salud, la retroalimentación obtenida se considera representativa del tipo de usuario final para el cual Thary fue diseñado, y se identifica como una línea de trabajo futuro la realización de una validación de usabilidad más extensa, con un número mayor de participantes.

4.4 Evaluación de la Seguridad y Estándares

La seguridad de Thary fue evaluada en base al rendimiento que tienen las mitigaciones implementadas en el sistema frente a las amenazas identificadas mediante el modelo STRIDE.

4.4.1 Resultados del análisis de amenazas (STRIDE)

En la siguiente tabla se pueden ver los resultados del estado de cada una de las partes de STRIDE, indicando si la amenaza fue mitigada o no y en el caso de que no lo haya sido, se indica el riesgo asociado a la amenaza.

TABLA XII
Estado de mitigación de las amenazas identificadas mediante STRIDE.

Amenaza	Estado	Evidencia
Suplantacion de identidad	Mitigada	Contraseñas con hash <i>scrypt</i> , nunca en texto plano; clave de sesión (SECRET_KEY) leída desde variable de entorno
Manipulacion de datos	Mitigada	Cantidad y costo del pedido tomados del servidor, no del formulario
Repudio	Mitigada	Tabla actividad (subir archivo, confirmar/deshacer pedido, crear usuario) y registro de usuario/fecha en pedidos
Divulgacion de información	Mitigada	Aislamiento de datos por <code>empresa_id</code> mediante clave foranea
Denegación de servicio	Parcialmente mitigada	Límite de tamaño de archivo (MAX_CONTENT_LENGTH) y límite de tasa en login/registro (<i>Flask-Limiter</i> , 10 solicitudes/minuto)
Elevacion de privilegios	Mitigada	Control de acceso por roles (<code>admin_required</code>) en las siete rutas administrativas del sistema

Cinco de las seis amenazas fueron mitigadas exitosamente. La única amenaza cuya mitigación

podría considerarse parcial es "Denegación de servicio". Si bien se cuenta con mitigaciones como el límite para el tamaño de archivo y el límite de tasa en las rutas de login y registro, existe un problema en como se aplica este último, ya que aunque se configuró un límite de intentos por minuto, el servidor corre con varios procesos distintos, y cada uno lleva su propia cuenta de intentos por separado sin compartirla entre si, ya que ese conteo se guarda en la memoria de cada proceso individual. Esto significa que una misma persona podría hacer más intentos de los permitidos repartiendo los entre los distintos procesos, por lo que el límite real termina siendo mayor al configurado. Tampoco existe un mecanismo que bloquee a un usuario después de varios intentos fallidos de contraseña, más allá de este límite general de peticiones.

En cuanto a la mitigación de repudio, esta solo cubre el registro de las actividades que los usuarios realizan estando ya dentro del sistema, como subir un archivo, confirmar o deshacer un pedido, o crear un nuevo usuario, por lo que no se registran eventos como el inicio de sesión.

4.4.2 Nivel de cumplimiento frente a los estándares declarados

Por otro lado, también se realizó la evaluación de cumplimiento frente a los estándares ISO/IEC/IEEE 12207, ISO/IEC/IEEE 29148, Modelo C4, ISO/IEC 25010, Modelado STRIDE. Resultados los cuales se resumieron en la siguiente tabla:

TABLA XIII
Nivel de cumplimiento frente a los estándares de ingeniería declarados.

Estándar	Nivel	Evidencia
ISO/IEC/IEEE 12207	Alto	Procesos técnicos aplicados en su totalidad: stakeholders, requisitos, arquitectura, diseño detallado, amenazas
ISO/IEC/IEEE 29148	Alto	SRS estructurado en requisitos funcionales, no funcionales y casos de uso, con especificación completa en el Anexo A
ISO/IEC 25010	Medio	Aplicado de forma implícita mediante los escenarios arquitecturales; no se realizó una evaluación formal por cada característica de calidad
Modelo C4	Alto	Diagramas de contexto, contenedores y componentes, reflejando la arquitectura real del sistema
Modelado STRIDE	Alto	Seis categorías de amenaza evaluadas; cinco mitigadas, una parcialmente mitigada (Tabla XII)

En general, después de esta evaluación de seguridad y estándares, se puede determinar que el sistema Thary cuenta con un nivel de cumplimiento alto en cuanto a los estándares de ISO/IEC/IEEE 12207, ISO/IEC/IEEE 29148, Modelo C4 y el Modelado STRIDE, los cuales representan tanto el proceso de ingeniería de requisitos como de arquitectura y análisis de amenazas. En cuanto al ISO/IEC 25010, este solo fue aplicado a través de los escenarios arquitecturales, no se realizó una evaluación

formal para verificar cada característica, por lo que se considera una oportunidad de mejora para trabajos futuros.

4.5 Análisis de Impactos de la Solución

En el siguiente capítulo se detallan los impactos que el sistema Thary tiene desde distintos campos que abarcan: impacto técnico, impacto social, impacto económico, impacto ambiental e impacto ético.

Con este análisis se busca darle al lector una visión general de la propuesta de valor del sistema y como su existencia supondría un cambio positivo en la forma en la que se gestionan los inventarios en cualquier tipo de dominio de cualquier organización.

Para esto, se compara la evidencia real obtenida al ejecutar el sistema con los impactos iniciales que fueron declarados, con el fin de determinar si los impactos esperados fueron alcanzados.

4.5.1 Impacto técnico

Los impactos técnicos hacen referencia a la mejora en la eficiencia que la tecnología implementada en el sistema Thary aporta al proceso de gestión de inventarios mediante la predicción de demanda. Sin el sistema, el personal de inventario tiene que revisar producto por producto la cantidad de productos existentes, el historial de consumo y el tiempo de entrega de cada proveedor para posteriormente, en base a esa información, generar reportes y tomar decisiones de compra.

Thary se encarga de automatizar todo este proceso. Cuando el sistema procesa un historial completo, en este caso, se probó con el de la Clínica Nuestra Señora de los Remedios (con 856 productos), Thary genera de forma automática la clasificación ABC y de variabilidad, el pronóstico de demanda, y las recomendaciones de reposición para la totalidad del catálogo en una sola ejecución.

El mecanismo de selección de modelo por producto y sus resultados se validan estadísticamente mediante el uso de la prueba de Diebold-Mariano. Sus resultados demostraron una mejora respecto a utilizar un único modelo para todo el catálogo: el WAPE promedio del sistema final (21.88%) fue consistentemente menor al de la Media Móvil aplicada de forma generalizada (22.31%) a lo largo de los tres periodos de la validación cruzada temporal, representando una mejora relativa de aproximadamente 2%.

4.5.2 Impacto social

En cuanto a impacto social, se puede considerar que el principal impacto de Thary en este aspecto es que gracias a las herramientas que ofrece, agiliza un proceso que de hacerse manual, además de ser lento, es propenso a errores humanos.

En general, Thary ayuda a mejorar la disponibilidad de productos, la eficiencia general del personal y las ventas mediante una mejor planificación de inventarios.

En el caso de estudio utilizado para validar el sistema, se identificaron de forma temprana los

productos en riesgo de agotarse (200 de 856 productos, 23.4% del catálogo, en estado de alerta). El sistema le ofrece al personal de inventario visibilidad anticipada para prevenir desabastecimientos y tomar decisiones de compra de forma oportuna.

Este impacto se confirmó durante la validación con usuarios, en la que un gestor de inventario con experiencia real en el dominio de salud completo el flujo completo del sistema sin dificultad, y su retroalimentación llevó a una mejora en la usabilidad (la inclusión de una plantilla de CSV descargable), evidenciando que, efectivamente, el sistema es fácil de usar por su usuario final objetivo.

4.5.3 Impacto económico

Respecto a impactos económicos, Thary dispone de una pestaña específica para costos y presupuestos. Al ejecutar el sistema sobre el catálogo completo del caso de estudio se reportó un valor total de inventario actual de \$270.810.382 (calculado como la suma del inventario disponible multiplicado por el costo unitario de cada producto, sobre 772 de los 856 productos que contaban con costo unitario disponible en el archivo CSV). Esto permitió saber cuanta cantidad de capital estaba inmovilizada en el inventario en ese momento específico.

También se incluyó el cálculo del costo de ruptura (stockout), que estima cuánto dinero se dejaría de percibir si un producto se agota antes de recibir el siguiente pedido. Para poder calcular esto, se necesita conocer el precio de venta de cada producto, y esta es una columna que el archivo del caso de estudio no incluye, por lo que no fue posible calcular un costo de ruptura real para la Clínica Nuestra Señora de los Remedios.

Es por esto que, para poder verificar que la función funcionara correctamente, se generaron precios de venta estimados (ficticios) a partir del costo unitario de cada producto, aplicando un margen supuesto según su categoría. Con estos precios ilustrativos, el sistema estimó un costo de ruptura total de \$17.859.670 sobre 118 de los 200 productos en alerta que contaban con margen unitario disponible. Esta cifra confirma que la función calcula bien lo que tiene que calcular, pero no se debe interpretar como el impacto económico real del caso de estudio, ya que los precios usados no son los reales de la clínica.

4.5.4 Impacto ambiental

Los impactos ambientales se resumen en que, al evitar el sobre-stock y el desabastecimiento, se reduce el riesgo de que productos con fecha de vencimiento caduquen en el inventario antes de ser utilizados, lo cual sería un desperdicio tanto económico como de recursos. Aunque es un impacto que no se midió de manera formal o cuantitativa, se considera que es otro de los impactos positivos que Thary ofrece.

4.5.5 Impacto ético

Finalmente, en cuanto a impactos éticos, se considera que un criterio ético con el que cuenta Thary es el hecho de no exponer información sensible de las organizaciones que decidan usar el sistema a terceros, teniendo en cuenta la sensibilidad de los datos manejados (historial de consumo, información de usuarios) desde etapas tempranas del desarrollo del sistema.

La seguridad de los datos es un factor que se consideró desde el diseño del sistema, mediante el análisis de amenazas STRIDE y las mitigaciones correspondientes, las cuales incluyen conceptos como el aislamiento de datos entre organizaciones y el cifrado de credenciales.

4.6 Conclusiones

La correcta gestión de inventarios es una actividad crítica para cualquier organización, ya que un inventario que no se gestione de manera adecuada puede generar problemas como costos innecesarios, pérdida de ventas y clientes insatisfechos, entre otros. Es por eso que, el poder contar con una herramienta que permita realizar una acertada predicción de demanda de los productos es un elemento clave para la gestión de inventarios, ya que permite obtener un análisis detallado de las cantidades necesarias de cada producto y en base a eso, tomar decisiones informadas.

Es en este contexto en el que se plantea la siguiente pregunta de investigación: ¿Cómo se puede diseñar e implementar un sistema de apoyo a la decisión que, mediante modelos de predicción de demanda, permita estimar las cantidades necesarias de productos en un sistema de gestión de inventarios?

Dicha pregunta se respondió a través del cumplimiento del siguiente objetivo general: Desarrollar un sistema que le permita a los usuarios poder predecir cuál va a ser la demanda mensual de productos mediante el análisis de datos históricos, para garantizar su correcta planificación y abastecimiento. A partir del desarrollo, implementación y posterior evaluación de los resultados del sistema (Capítulos 3 y 4), se puede concluir que si es posible realizar el diseño de un sistema con dichas características y que este le ofrezca resultados confiables y fáciles de entender a los usuarios. Esto es posible siempre y cuando el sistema cumpla con las siguientes características:

- un enfoque empírico de selección de modelos por producto en lugar de un único modelo generalizado
- un mecanismo de validación estadística que confirme que las diferencias observadas entre modelos sean reales y no producto del azar
- una arquitectura de software que traduzca esas predicciones en recomendaciones operativas concretas (punto de reorden, cantidad económica de pedido, y alertas de reposición)

También se logró el cumplimiento de todos los objetivos específicos que fueron planteados en la primera etapa del desarrollo del sistema:

Identificar las fuentes de datos relevantes, incluyendo el historial de consumo y las variables del inventario necesarias para la predicción de la demanda de productos.

Se concluyó que las fuentes de datos relevantes para el funcionamiento básico de Thary, además del historial de demanda, que incluye rezagos de 1, 2, 3 y 6 meses, y medias móviles de 3 y 6 meses, son:

- variables de calendario
- estadísticas históricas del propio producto (media y desviación)
- variables de inventario como el nivel actual de existencias y el costo unitario

Por otra parte, otras variables que también se incorporaron para complementar el análisis con recomendaciones de reposición, pero que no son necesarias para el funcionamiento de predicción de demanda básico del sistema, son:

- el tiempo de entrega de los proveedores (*lead time*) y sus nombres (proveedor principal y alterno)
- el presupuesto de capital de trabajo disponible, usado para acotar la cantidad sugerida de pedido cuando este no alcanza para cubrir la cantidad económica de pedido (EOQ) ideal
- el margen unitario del producto, usado para estimar el costo de ruptura (*stockout*) ante un posible desabastecimiento

Seleccionar modelos de predicción de demanda adecuados para estimar el consumo o venta mensual de productos en diferentes contextos de los inventarios.

Se concluyó que aunque no existe un único modelo que sea universalmente superior, se pueden aprovechar las ventajas de cada uno de los modelos seleccionados: Media Móvil, Regresión Lineal y XGBoost. La estrategia adoptada consiste en seleccionar el modelo con menor error por cada producto individualmente, y validar dicha selección mediante la prueba de Diebold-Mariano.

Al aplicar esta estrategia, el sistema obtuvo un WAPE promedio de 21.88%, una cifra inferior al 22.31% que obtuvo la Media Móvil aplicada a todo el catálogo en los tres periodos de la validación cruzada temporal. Esto confirma que la selección de modelos es más efectiva cuando se hace por producto, y no mediante una elección única para todo un catálogo.

Incorporar variables como el tiempo de entrega de proveedores y el punto de reposición dentro del sistema para complementar las estimaciones de demanda.

Se incorporaron variables como el tiempo de entrega (*lead time*) y el nivel de inventario actual, las cuales permitieron calcular el stock de seguridad y el punto de reorden, complementados con la cantidad económica de pedido (EOQ) y el costo de ruptura (*stockout*) estimado.

Al ejecutar el sistema sobre el caso de estudio, se pudieron identificar 200 de 856 productos (23.4% del catálogo) en estado de alerta de reposición, con un valor total de inventario de \$270.810.382. En cuanto al costo de ruptura, este no se pudo calcular sobre datos reales, ya que el archivo del caso de estudio no incluye precios de venta. Por esto, se usó precios estimados de forma ilustrativa, con los que el sistema calculó un costo de ruptura de \$17.859.670 para los productos en alerta con margen unitario disponible, lo cual confirma que esta función funciona correctamente, aunque esta cifra en específico no representa el impacto económico real del caso de estudio.

Diseñar un módulo de visualización que presente las predicciones de manera clara y comprensible para apoyar la toma de decisiones del usuario.

El diseño de las interfaces se realizó con el objetivo de que sean comprensibles incluso para usuarios que no cuenten con conocimientos técnicos, y que a la vez les permita tomar decisiones informadas sobre la gestión de inventarios.

Se diseñaron pantallas diferenciadas por rol, en la que el administrador tiene acceso a información más sensible como las pantallas para subir un archivo de datos csv o la creación de nuevos usuarios y el historial de movimientos de la empresa, por su parte el empleado solo podría acceder a las pestañas que cuenten con los análisis de los resultados. Ambos roles pueden acceder a las pantallas de visualización con alertas visuales codificadas por color, tablas y gráficas, que permiten identificar rápidamente los productos que requieren atención.

La validación de este módulo con un gestor de inventario real del dominio de salud confirmó que la interfaz por sí sola resulta lo suficientemente comprensible para un usuario, sin necesidad de guía adicional ni conocimientos técnicos de modelado predictivo.

Evaluar el desempeño del sistema desarrollado mediante la realización de pruebas con datos históricos, con el fin de analizar la precisión de las predicciones generadas y verificar su utilidad en el apoyo a la toma de decisiones en la gestión del inventario.

Por último, el sistema se evaluó mediante la realización de 37 pruebas automatizadas que resultaron exitosas. También se realizó una validación cruzada temporal sobre el historial real de la clínica (que fue el caso de estudio con el que se hicieron las pruebas), confirmando una mejora estadísticamente significativa del sistema final frente a la utilización única de la Media Móvil. En cuanto a la utilidad práctica del sistema en la toma de decisiones, esta se validó mediante la retroalimentación y aprobación dada por un usuario real.

En conclusión, la evidencia recolectada a lo largo de este documento permite concluir que Thary responde satisfactoriamente a la pregunta de investigación planteada, ya que es posible diseñar e implementar un sistema de apoyo a la decisión que, mediante predicción de demanda, estime las

cantidades necesarias de productos en un sistema de gestión de inventarios, siempre que se adopte un enfoque de selección de modelos por producto, un mecanismo de validación estadística riguroso, y una arquitectura de software capaz de traducir esas predicciones en recomendaciones concretas. Esto se logró con algunas limitaciones reconocidas que se documentan como líneas de trabajo futuro en la siguiente sección.

4.7 Recomendaciones de Operación y Mantenibilidad

Las recomendaciones para la operación y mantenibilidad de Thary se realizan a partir de los hallazgos encontrados durante la evaluación de seguridad y del análisis de despliegue. El objetivo de estas recomendaciones es poder garantizar la continuidad del servicio, la seguridad de los datos y la mantenibilidad del sistema a lo largo del tiempo.

4.7.1 Continuidad y recuperación de datos

Para garantizar que en caso de que el sistema falle se puedan recuperar los datos, se recomienda implementar un plan de respaldo y restauración de datos que abarque los siguientes puntos, teniendo en cuenta que el sistema se encuentra actualmente desplegado en Railway, un proveedor de nube que ofrece servicios de base de datos y despliegue de aplicaciones web:

- Activar la funcionalidad de *backups* automáticos diarios de Railway sobre el servicio de MySQL, priorizando las tablas `usuarios`, `empresas` y `pedidos`.
- Configurar un volumen de almacenamiento persistente para el servicio web de Thary, para asegurar que los archivos que sean generados durante la ejecución como el CSV cargado, el modelo entrenado, gráficas, etc. no se pierdan ante cada redespliegue.
- Complementar los *backups* nativos de Railway con una exportación periódica (`mysqldump`) hacia un proveedor de nube distinto, esto con el fin de tener una copia de seguridad adicional y evitar depender de un único proveedor. De esta manera, de fallar Railway, se podrá restaurar la base de datos en otro proveedor de nube y continuar con la operación del sistema.
- Documentar el proceso de recuperación de datos, especificando el orden de prioridad de restauración. Las tablas de empresas y usuarios tienen prioridad, por lo que deben de restaurarse antes que otras tablas como resultados, dado que estas dependen de aquellas por clave foránea. Asimismo, se debe especificar los responsables de ejecutarlo.
- Cada tres meses debe de ejecutarse una prueba de restauración real sobre un entorno distinto al de producción. Esto con el fin de confirmar que los *backups* generados efectivamente pueden restaurarse.

4.7.2 Seguridad

Para garantizar la seguridad tanto de los datos como de la información de los usuarios, se recomienda:

- Implementar una política de complejidad para las contraseñas de los usuarios al momento del registro que incluya requisitos como un mínimo de 8 caracteres, la inclusión de letras mayúsculas, minúsculas, números y caracteres especiales,
- Hacer que los distintos procesos del servidor compartan un mismo conteo de intentos de inicio de sesión, en vez de que cada uno lleve su propia cuenta por separado. Así, se garantiza que el límite de intentos de inicio de sesión que está configurado se cumpla de verdad, sin importar cuántos procesos estén corriendo.
- Registrar los inicios de sesión dentro de la bitacora de actividades, complementando las demás acciones de los usuarios que ya se registran.

4.7.3 Verificación continua

- Ampliar el alcance de las pruebas automatizadas para que también cubran las etapas faltantes: segmentación de productos, construcción de variables, las rutas y autenticación de la aplicación web.
- Configurar el flujo de integración continua de GitHub Actions para que, además de ejecutar las pruebas, también calcule y reporte la cobertura de código en cada *push*, facilitando el seguimiento de esta ampliación en el tiempo.
- Establecer un ambiente de pruebas (*staging*) que este separado del entorno de producción y cuente con su propia instancia de base de datos, para poder realizar pruebas de seguridad y validar cambios antes de desplegarlos a producción.

4.7.4 Accesibilidad y mantenibilidad de la interfaz

- Incorporar nuevos mecanismos básicos para garantizar que las pantallas de interfaz del sistema sean inclusivas y accesibles para todo tipo de usuarios, entre los mecanismos que convendría incluir se encuentran: etiquetas para lectores de pantalla, navegación completa por teclado, entre otros.
- Mantener actualizada la documentación del diccionario de datos del Anexo B cada vez que se agregue una nueva columna al esquema de la base de datos.

4.8 Trabajos Futuros

Después de completar todo el desarrollo de Thary, se identificaron diferentes oportunidades de mejora que podrían hacer que el sistema mejore su desempeño, seguridad, eficiencia y en general, ser un mejor producto para el usuario final. Aunque dichas oportunidades fueron identificadas, no pudieron ser implementadas en esta versión del sistema debido al alcance del proyecto y demás limitaciones. Aun así, se consideran aportes valiosos que valen la pena ser documentados para futuras actualizaciones y versiones del sistema.

4.8.1 Realización de validación con usuarios

La validación de usabilidad fue realizada y documentada en el capítulo 4, sin embargo, esta consto con un carácter informal y con un número bastante reducido de participantes. Es por eso que se propone como trabajo futuro realizar un estudio de usabilidad más formal, el cual cuente con un mayor número de participantes con el perfil del usuario objetivo (es decir, personas relacionadas con la administración de productos en inventarios, ya sea con el rol de personal de inventario o de administrador). También convendría incluir métricas cuantitativas de satisfacción y facilidad de uso como implementar cuestionarios estandarizados como el System Usability Scale, más allá de solo pedir que prueben el sistema y recopilar feedback.

4.8.2 Conservación del historial de artefactos por corrida

El sistema está configurado de manera que cada vez que se procesa un archivo, MySQL guarda un registro nuevo sin borrar los anteriores, sin embargo, los archivos asociados a esa corrida como el modelo entrenado y las gráficas, se guardan siempre en la misma carpeta, por lo que se sobrescriben con cada nueva ejecución. Esto significa que solo se guardan los archivos de la ejecución más reciente, aunque en la base de datos si exista el historial completo de todas las ejecuciones anteriores. Es por esto que seria valioso encontrar alguna manera para poder guardar diferentes archivos por cada corrida, para en el futuro, poder comparar como se comportaba el modelo entrenado en distintos momentos, y no solo ver sus métricas ya calculadas.

4.8.3 Validación en otros dominios de inventario

Uno de los requerimientos no funcionales más importantes de Thary es que el sistema debe de poder ser utilizado para adaptarse a distintos dominios de inventarios, sin embargo, a la hora de probar y validar el sistema, solo se utilizó como caso de estudio un único dominio: el de la farmacia hospitalaria, más específicamente, el consumo de medicamentos e insumos medicos de la Clínica Nuestra Señora de los Remedios del año 2021. Esto se hizo porque era el único dominio para el cual se contaba con un dataset real que representará el comportamiento de la demanda de una serie de productos de manera realista y organica. A pesar de que no se descarta que el sistema pueda ser utilizado en otros dominios de inventario, como retail, ferreteria, alimentos, etc., el

realizar más pruebas con datasets reales de dichos dominios variados sería beneficioso para probar la adaptabilidad del sistema y confirmar que el desempeño del sistema se mantiene en otros contextos.

4.8.4 Exploración de modelos de aprendizaje profundo

Durante la revisión de literatura se exploraron diferentes técnicas de Deep Learning con las que es posible realizar predicciones de series de tiempo. Entre ellas, se descartaron LSTM y GRU como modelos candidatos para el sistema Thary, aun así, en un trabajo futuro se podría rescatar la posibilidad de integrar alguna de estas técnicas como una cuarta o quinta alternativa al mecanismo de selección de mejor modelo por producto. Principalmente en escenarios donde se cuente con un historial de consumo más extenso al utilizado en el caso de estudio actual, dado que este tipo de modelos suele requerir mayor cantidad de datos para superar a técnicas más simples.

4.8.5 Reevaluación del asistente conversacional

Durante una primera versión de Thary, se implementó un asistente conversacional basado en un modelo de lenguaje ejecutado localmente (Ollama), el cual fue posteriormente descartado como una de las funcionalidades del sistema en la versión final. Se podría considerar en el futuro reevaluar la reactivación de esta funcionalidad, incluyendo un análisis más detallado de su valor real para el usuario final.

4.8.6 Combinación de pronósticos con métodos más sofisticados

La combinación de pronósticos implementada en Thary utiliza un único peso fijo, calculado por mínimos cuadrados sobre todo el periodo de backtest, que es el método más simple dentro de la literatura de combinación de pronósticos [21]. Se propone como trabajo futuro explorar métodos más sofisticados, como pesos que varíen en el tiempo o combinaciones que incluyan los tres modelos candidatos en lugar de solo dos.

4.9 Declaración de Uso Responsable de IA

El desarrollo de Thary se apoyó mediante el uso de la herramienta de inteligencia artificial Claude (Anthropic), incluida su versión para el entorno de programación (Claude Code), como asistente de apoyo en diferentes etapas. El alcance de dicho uso se detalla a continuación:

- Apoyo en la corrección ortográfica y gramatical de los capítulos, ofreciendo sugerencias de cómo reorganizar las secciones de la documentación y redactando propuestas de texto a partir de las ideas y explicaciones del autor.
- Verificación constante del código fuente real del sistema Thary, con el fin de asegurar que las explicaciones y la documentación del documento coincidieran con las últimas versiones del código. Ya que debido a las constantes actualizaciones que recibió el sistema, fue común

que varios capítulos necesitaran ser actualizados y reescritos para reflejar el estado actual del código.

- Apoyo en la escritura y creación de código de algunas funcionalidades del sistema, así como en la corrección rápida de errores de sintaxis y de ejecución.
- Apoyo en la corrección de errores de compilación y sintaxis del documento en $\text{\LaTeX}$.

Aun así, se verificaron manualmente todas las aportaciones que dio la IA con el fin de asegurar que fuesen correctas, también se revisó el contenido generado para asegurar que no se introdujeran conceptos nuevos que no están implementados en el sistema.

Referencias Bibliográficas

- [1] P. MEANA COALLA, *Gestión de inventarios* (Comercio y Marketing). Ediciones Paraninfo, S.A, 2024, ISBN: 9788428363570. [Internet]. available from <https://books.google.com.co/books?id=RfgyEQAAQBAJ>.
- [2] IBM, (2024). “¿Qué es la gestión de inventario?”, IBM Think. [Accessed: Aug. 5, 2026]. [Internet]. available from <https://www.ibm.com/mx-es/think/topics/inventory-management>.
- [3] H. A. Setyadi, B. Al Amin, and P. Widodo, “Implementation economic order quantity and reorder point methods in inventory management information systems”, *Journal of Information Systems and Informatics*, vol. 6, no. 1, pp. 103–117, 2024, Disponible en: <https://doaj.org/article/ec81c372e4604ad39a6690acfae59380>. doi: 10.51519/journalisi.v6i1.647
- [4] E. Mahendrawathi, E. Nurul Laili, and R. P. Kusumawardani, “Classification of hospital pharmaceutical drug inventory items by combining abc analysis and fuzzy classification”, in *ICACISIS 2011 - 2011 International Conference on Advanced Computer Science and Information Systems, Proceedings*, Disponible en: <https://www.scopus.com/inward/record.uri?eid=2-s2.0-84857290700>, 2011, pp. 215–220.
- [5] A. A. Syntetos, J. E. Boylan, and J. D. Croston, “On the categorization of demand patterns”, *Journal of the Operational Research Society*, vol. 56, no. 5, pp. 495–503, 2005. doi: 10.1057/palgrave.jors.2601841
- [6] T. Finn and I. Smalley, (2024). “¿Qué es la previsión de demanda?”, IBM Think. [Accessed: Aug. 5, 2026]. [Internet]. available from <https://www.ibm.com/es-es/think/topics/demand-forecasting>.
- [7] M. Valencia Cárdenas, F. J. Díaz Serna, and J. C. Correa Morales, “Inventory planning with dynamic demand. A state of art review”, *DYNA*, vol. 82, no. 190, pp. 182–191, Mar. 2015. doi: 10.15446/dyna.v82n190.42828 [Internet]. available from <https://revistas.unal.edu.co/index.php/dyna/article/view/42828>.
- [8] S. F. Sánchez Cuervo and C. C. Ortega Betancur, *Análisis de series temporales para predicción de demanda de productos*, Nov. 2024. [Internet]. available from <https://repository.unad.edu.co/handle/10596/69801>.
- [9] R. Verma, “Demand forecasting of spare parts store by moving average method and verification by exponential method”, *MIJBR-MITS International Journal of Research*, vol. 6, no. 1, p. 51, 2019, Disponible en: https://mits.ac.in/mits_new/public/uploads/jntua/d8cd460ac347bfb463616048e32ada5f.pdf, ISSN: 2394-4161.

-
- [10] A. Kochak and S. Sharma, “Demand forecasting using neural network for supply chain management”, *International journal of mechanical engineering and robotics research*, vol. 4, no. 1, pp. 96–104, 2015, Disponible en: https://www.ijmerr.com/v4n1/ijmerr_v4n1_10.pdf.
- [11] X. Wen and W. Li, “Time series prediction based on lstm-attention-lstm model”, *IEEE Access*, vol. 11, pp. 48 322–48 331, 2023. DOI: 10.1109/ACCESS.2023.3276628
- [12] F. M. Salem, “Gated rnn: The gated recurrent unit (gru) rnn”, in *Recurrent Neural Networks: From Simple to Gated Architectures*, Cham: Springer International Publishing, 2022, pp. 85–100, ISBN: 978-3-030-89929-5. DOI: 10.1007/978-3-030-89929-5_5 [Internet]. available from https://doi.org/10.1007/978-3-030-89929-5_5.
- [13] I. M. Peláez, “Modelos de regresión: Lineal simple y regresión logística”, *Revista Seden*, vol. 14, pp. 195–214, 2016.
- [14] I. Guyon, J. Weston, S. Barnhill, and V. Vapnik, “Gene selection for cancer classification using support vector machines”, *Machine Learning*, vol. 46, no. 1–3, pp. 389–422, 2002, Disponible en: <https://doi.org/10.1023/A:1012487302797>. DOI: 10.1023/A:1012487302797
- [15] R. M. O’Brien, “A caution regarding rules of thumb for variance inflation factors”, *Quality & Quantity*, vol. 41, no. 5, pp. 673–690, 2007, Disponible en: <https://doi.org/10.1007/s11135-006-9018-6>. DOI: 10.1007/s11135-006-9018-6
- [16] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system”, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16, Disponible en: <https://doi.org/10.1145/2939672.2939785>, New York, NY, USA: ACM, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785
- [17] XGBoost Developers, *XGBoost Parameters*, Documentacion oficial de XGBoost, Disponible en: https://xgboost.readthedocs.io/en/release_2.1.0/parameter.html, 2024.
- [18] H. Hewamalage, C. Bergmeir, and K. Bandara, “Recurrent neural networks for time series forecasting: Current status and future directions”, *International Journal of Forecasting*, vol. 37, no. 1, pp. 388–427, 2021, Disponible en: <https://arxiv.org/abs/1909.00590>.
- [19] H. Lütkepohl and F. Xu, “The role of the log transformation in forecasting economic variables”, CESifo, CESifo Working Paper 2591, 2009, Disponible en: <https://www.cesifo.org/en/publications/2009/working-paper/role-log-transformation-forecasting-economic-variables>.
- [20] M. A. Jahin, A. Shahriar, and M. A. Amin, “MCDFN: Supply chain demand forecasting via an explainable multi-channel data fusion network model”, *Evolutionary Intelligence*, 2025, Disponible en: <https://doi.org/10.1007/s12065-025-01053-7>. DOI: 10.1007/s12065-025-01053-7

-
- [21] X. Wang, R. J. Hyndman, F. Li, and Y. Kang, “Forecast combinations: An over 50-year review”, *International Journal of Forecasting*, vol. 39, no. 4, pp. 1518–1547, 2023, Disponible en: <https://doi.org/10.1016/j.ijforecast.2022.11.005>. DOI: 10.1016/j.ijforecast.2022.11.005
- [22] J. M. Bates and C. W. J. Granger, “The combination of forecasts”, *Operational Research Quarterly*, vol. 20, no. 4, pp. 451–468, 1969.
- [23] C. W. J. Granger and R. Ramanathan, “Improved methods of combining forecasts”, *Journal of Forecasting*, vol. 3, no. 2, pp. 197–204, 1984, Disponible en: <https://doi.org/10.1002/for.3980030207>. DOI: 10.1002/for.3980030207
- [24] H. Hewamalage, K. Ackermann, and C. Bergmeir, “Forecast evaluation for data scientists: Common pitfalls and best practices”, *Data Mining and Knowledge Discovery*, vol. 37, no. 2, pp. 788–832, 2023, Disponible en: <https://doi.org/10.1007/s10618-022-00894-5>. Copia gratuita: <https://arxiv.org/abs/2203.10716>. DOI: 10.1007/s10618-022-00894-5
- [25] S. Kim and H. Kim, “A new metric of absolute percentage error for intermittent demand forecasts”, *International Journal of Forecasting*, vol. 32, no. 3, pp. 669–679, 2016, Disponible en: <https://ideas.repec.org/a/eee/intfor/v32y2016i3p669-679.html>.
- [26] F. X. Diebold, “Comparing predictive accuracy, twenty years later: A personal perspective on the use and abuse of diebold-mariano tests”, *Journal of Business & Economic Statistics*, vol. 33, no. 1, pp. 1–9, 2015, Disponible en: <https://doi.org/10.1080/07350015.2014.983236>. Copia gratuita: <https://www.nber.org/papers/w18391.pdf>. DOI: 10.1080/07350015.2014.983236
- [27] F. X. Diebold and R. S. Mariano, “Comparing predictive accuracy”, *Journal of Business & Economic Statistics*, vol. 13, no. 3, pp. 253–263, 1995, Disponible en: <https://doi.org/10.1080/07350015.1995.10524599>. Copia gratuita del autor: <https://www.sas.upenn.edu/~fdiebold/papers/paper68/pa.dm.pdf>. DOI: 10.1080/07350015.1995.10524599
- [28] D. Harvey, S. Leybourne, and P. Newbold, “Testing the equality of prediction mean squared errors”, *International Journal of Forecasting*, vol. 13, no. 2, pp. 281–291, 1997, Disponible en: <https://ideas.repec.org/a/eee/intfor/v13y1997i2p281-291.html>.
- [29] M. Yadav, R. Patil, B. Bhirud, P. Chavle, and S. Bhosale, *Lstm model based demand forecasting in supply chain using efficient optimizer*, Disponible en: https://www.researchgate.net/publication/373555323_LSTM_Model_based_Demand_Forecasting_in_Supply_Chain_using_Efficient_Optimizer, Sep. 2023.

-
- [30] Y. Lin, “Application of abc analysis and linear regression model in inventory management”, *Advances in Economics, Management and Political Sciences*, vol. 126, pp. 111–119, Dec. 2024, Disponible en: https://www.researchgate.net/publication/387236283_Application_of_ABC_Analysis_and_Linear_Regression_Model_in_Inventory_Management. DOI: 10.54254/2754-1169/2024.18326
- [31] U. Gupta, V. Bhattacharjee, and P. S. Bishnu, “Stocknet—gru based stock index prediction”, *Expert Systems with Applications*, vol. 207, p. 117986, 2022, ISSN: 0957-4174. DOI: 10.1016/j.eswa.2022.117986
- [32] S. Nagadevi, G. Abirami, R. Vidhya, S. Selvakumar, and C. Vijayalakshmi, “Xgboost-based demand forecasting in supply chain management using machine learning algorithm”, *International Journal of Interactive Mobile Technologies (iJIM)*, vol. 19, no. 18, pp. 161–174, Sep. 2025. DOI: 10.3991/ijim.v19i18.57253 [Internet]. available from <https://online-journals.org/index.php/i-jim/article/view/57253>.
- [33] D. Shah, *Demand Forecasting & Inventory Optimization for Consumer Products*, GitHub repository, Disponible en: <https://github.com/dhruvi002/demand-forecast-inventory-optimizer>, 2026.
- [34] S. Saci, (Aug. 2020). “Machine Learning for Retail Demand Forecasting: Comparative Study of Demand Forecasting Methods for a Retail Store (XGBoost Model vs. Rolling Mean)”. Disponible en: <https://www.samirsaci.com/inventory-management-for-retail-stochastic-demand-2/>.
- [35] ksmooi, *Forecasting inventory demand using xgboost*, Kaggle Notebook, Disponible en: <https://www.kaggle.com/code/ksmooi/inventory-demand-forecast-xgboost-reg>, 2024.
- [36] T. Verma, *Linear regression: Demand forecasting*, Notebook sobre el dataset BoomBikes (demanda de bicicletas compartidas). Disponible en: https://github.com/21-f3000932/LinearRegression_Demand-forecasting/tree/main, 2021.
- [37] ISO/IEC/IEEE, *ISO/IEC/IEEE 12207:2026 – Systems and software engineering – Software life cycle processes*, International Standard, Disponible en: <https://www.iso.org/standard/90219.html>, Apr. 2026.
- [38] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 – Systems and software engineering – Life cycle processes – Requirements engineering*, International Standard, Revisada y confirmada en 2024. Disponible en: <https://www.iso.org/standard/72089.html>, 2018.
- [39] ISO/IEC, *ISO/IEC 25010:2023 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, International Standard, Disponible en: <https://www.iso.org/standard/78176.html>, 2023.

-
- [40] S. Ben Taieb, G. Bontempi, A. F. Atiya, and A. Sorjamaa, “A review and comparison of strategies for multi-step ahead time series forecasting based on the nn5 forecasting competition”, *Expert Systems with Applications*, vol. 39, no. 8, pp. 7067–7083, 2012. DOI: 10.1016/j.eswa.2012.01.039 [Internet]. available from <https://www.sciencedirect.com/science/article/pii/S0957417412000528>.

ANEXOS

Anexo A Documento completo SRS (historias de usuario / backlog)

TABLA XIV
Especificación del caso de uso CU-01: Iniciar sesión

Elemento	Especificación
ID	CU-01
Nombre	Iniciar sesión
Actor principal	Usuario registrado
Requisitos relacionados	RF-10
Objetivo	El objetivo es permitir que los usuarios registrados accedan al sistema utilizando sus credenciales.
Precondiciones	Para poder iniciar sesión, el usuario debe encontrarse previamente registrado en el sistema.
Postcondiciones	El usuario accede exitosamente al sistema y puede acceder a las funcionalidades a las que tenga acceso dependiendo de su rol.
Flujo principal	1. El usuario ingresa su usuario y contraseña. 2. El sistema valida que la información sea correcta. 3. El sistema identifica el perfil del usuario. 4. El sistema permite el acceso a Thary.
Flujos alternativos	1. Si el usuario o la contraseña introducidos son incorrectas, el sistema le informa al usuario que los datos ingresados no son válidos. 2. Si el usuario no está registrado, el sistema rechaza el acceso y el usuario tiene la oportunidad de registrarse.

TABLA XV
Especificación del caso de uso CU-02: Cargar historial de consumo

Elemento	Especificación
ID	CU-02
Nombre	Cargar historial de consumo
Actor principal	Administrador
Requisitos relacionados	RF-01, RF-07
Objetivo	El objetivo del caso de uso es permitirle al Administrador poder cargar un archivo CSV que contenga el historial de consumo de los productos del inventario.
Precondiciones	• Para acceder a esta funcionalidad, el usuario debe haber iniciado sesión con perfil de Administrador. • El archivo debe subirse en formato de CSV.
Postcondiciones	El archivo es recibido por el sistema para su posterior procesamiento
Flujo principal	1. El Administrador inicia sesión y es recibido por la pantalla inicial que le da la opción de subir un archivo csv. 2. El Administrador selecciona un archivo CSV. 3. El sistema recibe el archivo. 4. El sistema se encarga de validar el formato y que contenga los campos mínimos requeridos para el funcionamiento del sistema como: fecha, ID del producto y cantidad demandada de cada uno. 5. El sistema informa que el archivo fue cargado correctamente.
Flujos alternativos	1. Si el archivo no tiene formato CSV, el sistema informa el error con el mensaje "El archivo debe tener extensión .csv" 2. Si faltan campos obligatorios, el sistema rechaza el archivo e informa cuáles son los datos requeridos, ejemplo: Falta la columna obligatoria: fecha" 3. Si el usuario no cuenta con los permisos de Administrador, Thary simplemente no le muestra la opción de carga archivo, limitandose con solo mostrarle el dashboard con los resultados y demas pantallas con información.

TABLA XVI
Especificación del caso de uso CU-03: Procesar archivo

Elemento	Especificación
ID	CU-03
Nombre	Procesar archivo
Actor principal	Administrador
Requisitos relacionados	RF-02, RF-07
Objetivo	Iniciar el procesamiento del historial de consumo previamente cargado para generar las estimaciones de demanda.
Precondiciones	• El Administrador debe haber iniciado sesión. • Debe existir un archivo de historial cargado y válido.
Postcondiciones	El sistema procesa los datos, ejecuta el proceso de predicción y genera los resultados de demanda para los productos.
Flujo principal	1. El Administrador accede al panel de procesamiento. 2. El sistema verifica que exista un archivo válido. 3. El Administrador selecciona la opción “Procesar archivo”. 4. El sistema procesa y prepara los datos. 5. El sistema ejecuta el proceso de predicción. 6. El sistema genera las estimaciones de demanda mensual para los productos. 7. El sistema genera los resultados necesarios para las etapas posteriores.
Flujos alternativos	1. Si no existe un archivo válido, el sistema informa al Administrador que debe cargar un historial. 2. Si ocurre un error durante el procesamiento, el sistema informa que no fue posible completar la operación. 3. Si el usuario no es Administrador, el sistema impide la ejecución.

TABLA XVII
Especificación del caso de uso CU-04: Consultar demanda pronosticada

Elemento	Especificación
ID	CU-04
Nombre	Consultar demanda pronosticada
Actor principal	Personal de Inventario
Requisitos relacionados	RF-02, RF-05
Objetivo	Permitirle al los empleados del inventario consultar cual es la demanda mensual estimada para cada uno de los producto.
Precondiciones	Como precondición, ya debe de existir un archivo procesado anteriormente por el administrador, también ya deben haber resultados de predicción disponibles.
Postcondiciones	El usuario visualiza las estimaciones de demanda correspondientes a los productos.
Flujo principal	1. El usuario accede al dashboard con las predicciones. 2. El sistema consulta los resultados generados. 3. El sistema le muestra al usuario, sin importar si es administrador o usuario, la demanda pronosticada de los productos. 4. El usuario consulta la información presentada.
Flujos alternativos	Si no existen resultados de predicción, el sistema informa que primero debe realizarse el procesamiento del archivo mediante el siguiente mensaje "Todavía no hay datos cargados. En cuanto tu administrador suba un archivo, vas a poder ver el pronóstico, el inventario y confirmar pedidos."

TABLA XVIII
Especificación del caso de uso CU-05: Consultar estado del inventario

Elemento	Especificación
ID	CU-05
Nombre	Consultar estado del inventario
Actor principal	Personal de Inventario
Requisitos relacionados	RF-04, RF-05
Objetivo	El objetivo es que se pueda consultar el inventario actual de los productos e identificar fácilmente a aquellos que requieran atención o esten en mayor riesgo de desabatecimiento.
Precondiciones	Como precondición para este caso de uso, es importante que ya exista información del inventario disponible y resultados del procesamiento.
Postcondiciones	Las postcondiciones son que el usuario puede visualizar sin problemas el estado actual del inventario y las alertas correspondientes para cada producto.
Flujo principal	1. El usuario accede al módulo de "Estado de inventario". 2. El sistema consulta la información actual del inventario. 3. El sistema se encarga de comparar el inventario disponible con el punto de reposición de cada producto. 4. El sistema identifica los productos que se encuentran en o por debajo de su punto de reposición. 5. El sistema muestra las alertas visuales correspondientes dependiendo de cada caso, se marcaran de color rojo en caso de estar por debajo del punto de reposición.
Flujos alternativos	Si no existe información de inventario para un producto, el sistema informa que no se dispone de los datos necesarios para determinar su estado.

TABLA XIX

Especificación del caso de uso CU-06: Consultar recomendaciones de reposición y presupuesto

Elemento	Especificación
ID	CU-06
Nombre	Consultar recomendaciones de reposición y presupuesto
Actor principal	Personal de Inventario
Requisitos relacionados	RF-03, RF-05
Objetivo	Permitir consultar las recomendaciones de reposición generadas para cada producto, teniendo en cuenta distintos datos como la demanda estimada, el inventario que ya se tiene disponible, el tiempo de entrega, que varía de producto a producto, y el presupuesto establecido.
Precondiciones	La principal precondición es que debe existir un archivo csv ya procesado, y el sistema ya debe contar con toda la información necesaria para poder calcular las recomendaciones.
Postcondiciones	El usuario visualiza la cantidad recomendada de compra de una manera clara mediante el uso de gráficas y tablas, también puede ver otros datos relevantes como el punto de reposición y información acerca del presupuesto.
Flujo principal	1. El usuario accede al módulo de recomendaciones, es decir, el dashboard principal. 2. El sistema consulta las estimaciones de demanda. 3. El sistema obtiene el inventario actual y los parámetros de reposición. 4. El sistema considera el tiempo de entrega del proveedor y las restricciones presupuestales. 5. El sistema calcula la recomendación de reposición. 6. El sistema le muestra los resultados al usuario.
Flujos alternativos	Si no existen datos suficientes para realizar el cálculo, el sistema informa qué información se encuentra pendiente.

TABLA XX
Especificación del caso de uso CU-07: Registrar pedido

Elemento	Especificación
ID	CU-07
Nombre	Registrar pedido
Actor principal	Personal de Inventario
Requisitos relacionados	RF-06
Objetivo	El objetivo es permitirle al Personal de Inventario registrar un pedido de reposición a partir de las recomendaciones generadas.
Precondiciones	El usuario debe haber iniciado sesión y debe existir información del producto y de la cantidad a solicitar.
Postcondiciones	El pedido queda registrado con la información correspondiente y asociado con el nombre del usuario responsable.
Flujo principal	1. El usuario consulta la página de "Pedidos" del sistema. 2. Selecciona uno de los pedidos para registrarlos. 3. Selecciona el proveedor. 4. A partir de esto, el sistema registra la fecha del pedido. 5. El sistema identifica al usuario responsable. 6. El sistema guarda la información del pedido. 7. El sistema confirma el registro.
Flujos alternativos	Si no se selecciona ningún producto, pero de todas maneras se hace click en el boton para registrar, saldra el siguiente mensaje: No seleccionaste ningún producto para confirmar"

TABLA XXI
Especificación del caso de uso CU-08: Gestionar usuarios

Elemento	Especificación
ID	CU-08
Nombre	Gestionar usuarios
Actor principal	Administrador
Requisitos relacionados	RF-08
Objetivo	Permitirle al Administrador poder crear cuentas para sus empleados con menos privilegios a los de un administrador.
Precondiciones	El usuario debe haber iniciado sesión con perfil de Administrador.
Postcondiciones	El nuevo empleado queda registrado y puede utilizar el sistema con permisos restringidos.
Flujo principal	1. El Administrador accede a la gestión de usuarios. 2. Selecciona la opción para crear un usuario. 3. Ingresa la información requerida. 4. Define el perfil o nivel de privilegios. 5. El sistema valida la información. 6. El sistema registra al nuevo usuario.
Flujos alternativos	1. Si la información es inválida o incompleta, el sistema solicita corregirla mediante un mensaje de error. 2. Si el usuario ya existe, el sistema informa que no puede crearse una cuenta duplicada.

TABLA XXII
Especificación del caso de uso CU-09: Deshacer pedido

Elemento	Especificación
ID	CU-09
Nombre	Deshacer pedido
Actor principal	Personal de Inventario
Requisitos relacionados	RF-10
Objetivo	El objetivo es permitirle al Personal de Inventario revertir un pedido previamente confirmado, en caso de haberse equivocado o de que la confirmación ya no sea necesaria.
Precondiciones	• El usuario debe haber iniciado sesión. • Debe existir al menos un pedido previamente confirmado por el mismo usuario.
Postcondiciones	El pedido queda marcado como no confirmado, y vuelve a aparecer entre las recomendaciones pendientes de confirmar.
Flujo principal	1. El usuario accede a la página de "Pedidos" del sistema. 2. Consulta el historial de pedidos confirmados. 3. Selecciona el pedido que desea deshacer. 4. El sistema solicita confirmación de la acción. 5. El usuario confirma que desea deshacer el pedido. 6. El sistema revierte el estado del pedido y lo registra en la bitacora de actividad.
Flujos alternativos	1. Si el usuario cancela la confirmación, el sistema no realiza ningún cambio y el pedido permanece confirmado. 2. Si el pedido ya fue deshecho previamente por otro usuario, el sistema informa que la acción ya no es posible.

TABLA XXIII
Especificación del caso de uso CU-10: Consultar actividad de la empresa

Elemento	Especificación
ID	CU-10
Nombre	Consultar actividad de la empresa
Actor principal	Administrador
Requisitos relacionados	RF-11
Objetivo	Permitirle al Administrador consultar la bitacora completa de acciones realizadas por todos los usuarios de su empresa, con fines de auditoria.
Precondiciones	• El usuario debe haber iniciado sesión con perfil de Administrador. • Debe existir al menos un registro de actividad generado previamente en la empresa.
Postcondiciones	El administrador visualiza el listado de acciones realizadas por los usuarios de su empresa, ordenadas cronologicamente.
Flujo principal	1. El Administrador accede al módulo de "Actividad". 2. El sistema consulta el registro de actividad de la empresa del administrador. 3. El sistema muestra el listado de acciones, indicando el usuario responsable, el tipo de acción y la fecha y hora en que ocurrió.
Flujos alternativos	Si aún no se ha registrado ninguna actividad en la empresa, el sistema informa que no hay eventos para mostrar.

TABLA XXIV
Especificación del caso de uso CU-11: Consultar calendario de pedidos

Elemento	Especificación
ID	CU-11
Nombre	Consultar calendario de pedidos
Actor principal	Personal de Inventario
Requisitos relacionados	RF-12
Objetivo	Permitirle al usuario visualizar en un calendario las fechas estimadas en las que debe realizarse el pedido de cada producto, según las recomendaciones generadas por el sistema.
Precondiciones	Debe existir un archivo previamente procesado, con resultados de reposición disponibles.
Postcondiciones	El usuario visualiza, organizadas por fecha, las recomendaciones de pedido proximas a vencer.
Flujo principal	1. El usuario accede al módulo de "Calendario de pedidos". 2. El sistema consulta las recomendaciones de reposición vigentes. 3. El sistema organiza dichas recomendaciones según la fecha estimada en la que deberían realizarse. 4. El sistema muestra el calendario con los pedidos proximos, por producto.
Flujos alternativos	Si no existen recomendaciones de reposición disponibles, el sistema informa que primero debe procesarse un archivo.

TABLA XXV
Especificación del caso de uso CU-12: Consultar perfil

Elemento	Especificación
ID	CU-12
Nombre	Consultar perfil
Actor principal	Usuario registrado
Requisitos relacionados	RF-13
Objetivo	Permitirle a cualquier usuario, sin importar su rol, consultar los datos de su propia cuenta y el historial de las acciones que el mismo ha realizado en el sistema.
Precondiciones	El usuario debe haber iniciado sesión.
Postcondiciones	El usuario visualiza la información de su cuenta y sus propios movimientos dentro del sistema.
Flujo principal	1. El usuario accede al módulo de "Perfil". 2. El sistema consulta los datos de la cuenta del usuario. 3. El sistema consulta los movimientos propios del usuario (pedidos confirmados o revertidos, y si es administrador, también sus archivos cargados y los usuarios que ha creado). 4. El sistema muestra la información recopilada.
Flujos alternativos	Si el usuario aún no ha realizado ninguna acción registrada en el sistema, el sistema muestra los datos de la cuenta sin movimientos asociados.

Anexo B Modelos de datos extensos, diccionarios de datos, mockups

Este anexo presenta el diccionario de datos completo del esquema relacional de Thary, complementando la descripción resumida presentada en el capítulo 3. Todas las tablas se implementan sobre MySQL, con codificación `utf8mb4` y motor de almacenamiento InnoDB.

Tabla empresas

Campo	Tipo	Restricción	Descripción
id	VARCHAR(32)	PRIMARY KEY	Identificador único de la organización
nombre_empresa	VARCHAR(120)		Nombre de la organización, visible en el encabezado
fecha_creación	DATETIME	DEFAULT CURRENT_ TIMESTAMP	Fecha de registro de la empresa

Tabla usuarios

Campo	Tipo	Restricción	Descripción
id	INT	PRIMARY KEY, AUTO_ INCREMENT	Identificador único del usuario
nombre	VARCHAR(120)	NOT NULL	Nombre completo del usuario
username	VARCHAR(80)	NOT NULL, UNIQUE	Nombre de usuario para inicio de sesión
email	VARCHAR(160)	NOT NULL, UNIQUE	Correo electrónico del usuario
password_hash	VARCHAR(255)	NOT NULL	Hash de la contraseña, generado con <i>script</i>
rol	ENUM('admin','empleado')	NOT NULL, DEFAULT 'empleado'	Rol del usuario dentro de la empresa
empresa_id	VARCHAR(32)	NOT NULL, FK → empresas.id	Empresa a la que pertenece el usuario

Tabla pedidos

Campo	Tipo	Restricción	Descripción
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador único del pedido
empresa_id	VARCHAR(32)	NOT NULL, FK → empresas.id	Empresa a la que pertenece el pedido
producto_id	VARCHAR(64)	NOT NULL	Identificador del producto pedido
proveedor	VARCHAR(160)		Proveedor elegido para el pedido
cantidad	DOUBLE		Cantidad confirmada
costo	DOUBLE		Costo estimado del pedido
usuario	VARCHAR(120)		Usuario responsable de confirmar el pedido
fecha_hora	DATETIME		Fecha y hora de confirmación

Clave única: (empresa_id, producto_id)

Tabla actividad

Campo	Tipo	Restricción	Descripción
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador único del evento
empresa_id	VARCHAR(32)	NOT NULL, FK → empresas.id	Empresa asociada al evento
username	VARCHAR(80)	NOT NULL	Usuario que realizó la acción
nombre_usuario	VARCHAR(120)	NOT NULL	Nombre del usuario que realizó la acción
tipo	VARCHAR(30)	NOT NULL	Tipo de acción (archivo_subido, pedido_confirmado, etc.)
descripcion	VARCHAR(300)	NOT NULL	Descripción legible del evento
fecha	DATETIME	DEFAULT CURRENT_TIMESTAMP	Fecha y hora del evento

Tabla corridas

Campo	Tipo	Restricción	Descripción
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificador de la corrida (tabla padre)
empresa_id	VARCHAR(32)	NOT NULL, FK → empresas.id	Empresa que ejecutó la corrida
fecha_corrida	DATETIME	DEFAULT CURRENT_TIMESTAMP	Fecha de la corrida
horizonte_meses	INT		Meses hacia adelante pronosticados
presupuesto_capital_trabajo	DOUBLE		Presupuesto total disponible
presupuesto_por_producto	DOUBLE		Presupuesto asignado por producto
mape_final	DOUBLE		MAPE del sistema final
wape_final	DOUBLE		WAPE del sistema final
costo_total_pedidos_urgentes	DOUBLE		Costo total de productos en alerta
valor_total_inventario	DOUBLE		Valor total del inventario en la corrida
productos_sin_dato_inventario	INT		Cantidad de productos sin inventario registrado
productos_en_alerta	INT		Cantidad de productos en o bajo el punto de reorden
features_regresión_lineal	TEXT		Variables seleccionadas por RFE/VIF (serializado)

Tabla segmentacion

Campo	Tipo	Restricción	Descripción
id	BIGINT	PRIMARY KEY, AUTO_ INCREMENT	Identificador único
corrida_id	INT	NOT NULL, FK → corridas.id (CASCADE)	Corrida a la que pertenece
producto_id	VARCHAR(64)		Identificador del producto
nombre_producto	VARCHAR(200)		Nombre del producto
categoría	VARCHAR(120)		Categoría o rubro del producto
demanda_total	DOUBLE		Demanda total histórica
media	DOUBLE		Demanda promedio mensual
desviación	DOUBLE		Desviación estándar de la demanda
cv	DOUBLE		Coefficiente de variación
variabilidad	VARCHAR(20)		Clasificación Alta/Media/Baja
demanda_ acumulada_pct	DOUBLE		Porcentaje acumulado de demanda
categoría_abc	VARCHAR(5)		Clasificación A/B/C
método_campeón	VARCHAR(40)		Modelo seleccionado como campeón
proveedor_ principal	VARCHAR(160)		Proveedor principal del producto
proveedor_ alterno	VARCHAR(160)		Proveedor alterno del producto

Tabla `metricas_modelos`

Campo	Tipo	Restricción	Descripción
id	BIGINT	PRIMARY KEY, AUTO_ INCREMENT	Identificador único
corrida_id	INT	NOT NULL, FK → corridas.id (CASCADE)	Corrida a la que pertenece
modelo	VARCHAR(60)		Nombre del modelo evaluado
mae	DOUBLE		Error absoluto medio
rmse	DOUBLE		Raíz del error cuadrático medio
r2	DOUBLE		Coefficiente de determinación
mediana_error	DOUBLE		Mediana del error absoluto
mape	DOUBLE		Error porcentual absoluto medio
wape	DOUBLE		Error porcentual absoluto ponderado

Tabla predicciones_backtest

Campo	Tipo	Restricción	Descripción
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador único
corrida_id	INT	NOT NULL, FK → corridas.id (CASCADE)	Corrida a la que pertenece
fecha	VARCHAR(10)		Fecha del mes evaluado (AAAA-MM-DD)
producto_id	VARCHAR(64)		Identificador del producto
demanda	DOUBLE		Demanda real observada
categoría_abc	VARCHAR(5)		Clasificación ABC del producto
variabilidad	VARCHAR(20)		Clasificación de variabilidad
predicción_media_móvil	DOUBLE		Predicción del modelo Media Móvil
predicción_regresión_lineal	DOUBLE		Predicción del modelo Regresión Lineal
predicción_xgboost	DOUBLE		Predicción del modelo XGBoost
predicción_final	DOUBLE		Predicción del método campeón seleccionado
error_final	DOUBLE		Error de la predicción final
método_usado	VARCHAR(40)		Modelo seleccionado para ese producto y mes

Tabla pronostico_futuro

Campo	Tipo	Restricción	Descripción
id	BIGINT	PRIMARY KEY, AUTO_ INCREMENT	Identificador único
corrida_id	INT	NOT NULL, FK → corridas.id (CASCADE)	Corrida a la que pertenece
producto_id	VARCHAR(64)		Identificador del producto
nombre_producto	VARCHAR(200)		Nombre del producto
categoría	VARCHAR(120)		Categoría del producto
mes_ pronosticado	VARCHAR(7)		Mes futuro pronosticado (AAAA-MM)
horizonte	INT		Número de mes dentro del horizonte
predicción_ xgboost	DOUBLE		Predicción de XGBoost
predicción_ regresión_lineal	DOUBLE		Predicción de Regresión Lineal
predicción_ baseline	DOUBLE		Predicción de Media Móvil
predicción_final	DOUBLE		Predicción del método campeón
método_usado	VARCHAR(40)		Modelo seleccionado para ese producto

Tabla reorder

Campo	Tipo	Restricción	Descripción
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador único
corrida_id	INT	NOT NULL, FK → corridas.id (CASCADE)	Corrida a la que pertenece
producto_id	VARCHAR(64)		Identificador del producto
nombre_producto	VARCHAR(200)		Nombre del producto
categoría	VARCHAR(120)		Categoría del producto
proveedor_principal	VARCHAR(160)		Proveedor principal
proveedor_alterno	VARCHAR(160)		Proveedor alterno
categoría_abc	VARCHAR(5)		Clasificación ABC
variabilidad	VARCHAR(20)		Clasificación de variabilidad
método_pronóstico	VARCHAR(40)		Modelo usado para pronosticar
demanda_promedio_mensual_pronosticada	DOUBLE		Demanda promedio pronosticada
lead_time_semanas	DOUBLE		Tiempo de entrega en semanas
lead_time_meses	DOUBLE		Tiempo de entrega en meses
stock_seguridad	DOUBLE		Stock de seguridad calculado
punto_reorden	DOUBLE		Punto de reorden calculado
inventario_actual	DOUBLE		Inventario disponible actual
eoq	DOUBLE		Cantidad económica de pedido
límite_presupuesto_unidades	DOUBLE		Límite de unidades según presupuesto
cantidad_sugerida_pedido	DOUBLE		Cantidad final sugerida
costo_unitario	DOUBLE		Costo unitario del producto
costo_estimado_pedido	DOUBLE		Costo total estimado del pedido
valor_inventario_actual	DOUBLE		Valor del inventario disponible

Tabla validacion_cruzada_folds

Campo	Tipo	Restricción	Descripción
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador único
corrida_id	INT	NOT NULL, FK → corridas.id (CASCADE)	Corrida a la que pertenece
fold	INT		Número de fold (1, 2, 3...)
meses_entrenamiento	INT		Meses usados para entrenar en ese fold
periodo_test	VARCHAR(30)		Periodo evaluado en ese fold
modelo	VARCHAR(40)		Modelo evaluado (incluye “Sistema final”)
mae	DOUBLE		MAE obtenido en ese fold
rmse	DOUBLE		RMSE obtenido en ese fold
mape	DOUBLE		MAPE obtenido en ese fold
wape	DOUBLE		WAPE obtenido en ese fold

Tabla validacion_cruzada_resumen

Campo	Tipo	Restricción	Descripción
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Identificador único
corrida_id	INT	NOT NULL, FK → corridas.id (CASCADE)	Corrida a la que pertenece
modelo	VARCHAR(40)		Modelo resumido
métrica	VARCHAR(20)		Métrica resumida (MAE, RMSE, MAPE, WAPE)
promedio	DOUBLE		Promedio de la métrica entre folds
desvio	DOUBLE		Desviación estándar entre folds
folds	INT		Número de folds considerados

Anexo C Manual de usuario y/o guía de instalación y despliegue

Instalación en entorno local

1. Clonar el repositorio del proyecto y acceder a la carpeta creada:

```
git clone https://github.com/danna0723/Thary_Proyecto.git
cd Thary_Proyecto
```

Se requiere tener instalado Python 3.12 o superior.

2. Crear un entorno virtual e instalar las dependencias:

```
python -m venv venv
venv\Scripts\activate          # En Windows
pip install -r requirements.txt
```

3. Tener un servidor MySQL corriendo localmente (o accesible mediante las variables del siguiente paso) y un usuario con permisos sobre la base de datos del sistema. Por ejemplo:

```
CREATE USER 'thary_app'@'localhost' IDENTIFIED BY '<su_contraseña>'
GRANT ALL PRIVILEGES ON sistema_inventario.* TO 'thary_app'@'localhost'
```

Al iniciar, el sistema crea automáticamente la base de datos (si el usuario tiene permiso para hacerlo) y todas las tablas necesarias.

4. Configurar las variables de entorno para la conexión a MySQL (DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME). Si no se definen, el sistema usa los valores 127.0.0.1, puerto 3307, usuario thary_app y base de datos sistema_inventario, con contraseña vacía. Por ejemplo, en PowerShell:

```
$env:DB_PASSWORD = "<su_contraseña>"
```

Las variables definidas de esta forma solo duran mientras la terminal esté abierta.

5. Ejecutar la aplicación. En este modo de desarrollo no hace falta definir SECRET_KEY, porque se usa una clave de desarrollo por defecto:

```
python run.py
```

6. Acceder desde el navegador a `http://localhost:5000`.

Despliegue en producción (Railway)

1. Crear un proyecto en Railway y agregar dos servicios: la aplicación web (conectada al repositorio de GitHub) y una instancia de MySQL.
2. Configurar las variables de entorno del servicio web:
 - Las de conexión a la base de datos (DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME), apuntando a las credenciales del servicio MySQL de Railway.
 - SECRET_KEY: una cadena aleatoria larga que se usa para firmar las sesiones de los usuarios. Sin esta variable la aplicación no arranca, y no debe compartirse ni escribirse en el código.

Los nombres de las variables no deben tener espacios al inicio ni al final, porque eso hace fallar la construcción del servicio.

3. El archivo `Procfile` ya define el comando de arranque en producción mediante Gunicorn:

```
web: gunicorn run:app --bind 0.0.0.0:$PORT --workers 2 --timeout 1
```

4. Railway detecta automáticamente la versión de Python especificada en `.python-version` e instala las dependencias de `requirements.txt`.
5. Al iniciar, la aplicación crea automáticamente las tablas necesarias en la base de datos (función `crear_tablas()`).
6. Se recomienda activar un volumen persistente para el servicio web y la funcionalidad de *backups* automáticos para el servicio de MySQL (Sección 4.7.1).

Manual de usuario

Administrador

1. Registrarse: quien crea una cuenta desde la pantalla de registro obtiene una empresa propia y queda como su administrador. Después, iniciar sesión.
2. Desde el menú lateral, entrar a “Subir archivo”. Allí se indica el número de meses a pronosticar y el presupuesto disponible, y se carga un archivo CSV con el historial de consumo. Las columnas obligatorias son la fecha, el código del producto y la cantidad vendida; las opcionales habilitan funciones adicionales (por ejemplo, el inventario activa las alertas de pedido). La misma pantalla incluye el detalle de las columnas que reconoce el sistema y una plantilla de ejemplo descargable.

3. Seleccionar “Procesar archivo” para ejecutar el pipeline de predicción.
4. Consultar el “Panel técnico” para revisar la comparación de modelos y la segmentación de productos.
5. Crear cuentas nuevas para el personal de inventario desde “Usuarios”.
6. Consultar la bitácora de actividad de la empresa desde “Actividad”.

Personal de inventario

1. Iniciar sesión con las credenciales proporcionadas por el administrador.
2. Consultar, dentro de “Análisis”, el pronóstico de demanda, el estado del inventario, el presupuesto y costos, y el calendario de pedidos. El ícono de la campana, en la parte superior, indica cuántos productos necesitan pedido.
3. Descargar el pronóstico de demanda en formato CSV desde la pantalla de pronóstico, con la opción de elegir un rango de meses.
4. Confirmar un pedido sugerido desde la pantalla “Pedido”, seleccionando el proveedor correspondiente.
5. Deshacer un pedido confirmado por error con el botón “Deshacer” de la misma pantalla.
6. Consultar su propio perfil y el historial de sus acciones desde “Mi perfil”.

Anexo D Enlaces a repositorios de código fuente y tableros de gestión (Azure DevOps, Jira, GitHub)

El código fuente de Thary se gestionó con el sistema de control de versiones Git, y el repositorio se alojó en GitHub. A continuación se presentan los enlaces a los recursos del proyecto.

Recurso	Enlace
Repositorio de código fuente (GitHub)	https://github.com/danna0723/Thary_Proyecto
Flujo de integración continua (GitHub Actions)	https://github.com/danna0723/Thary_Proyecto/actions
Historial de cambios (commits)	https://github.com/danna0723/Thary_Proyecto/commits/main
Sistema desplegado (Railway)	https://web-production-6ac7e5.up.railway.app

Repositorio de código fuente

El repositorio contiene la aplicación completa (código en `app/`), las pruebas automatizadas (`tests/`), la definición del flujo de integración continua (`.github/workflows/tests.yml`) y los archivos de configuración del despliegue (`Procfile` y `requirements.txt`). El desarrollo se realizó sobre una única rama principal (`main`), con un historial de 16 commits entre el 14 y el 26 de septiembre de 2026 al momento de redactar este documento.

Integración y despliegue continuos

Cada envío de cambios (*push*) o *pull request* al repositorio ejecuta automáticamente las pruebas automatizadas mediante GitHub Actions, con Python 3.12. Además, el servicio en Railway está conectado al repositorio de GitHub, de modo que cada cambio enviado a la rama principal inicia un nuevo despliegue.

Tableros de gestión

Para el seguimiento del desarrollo del sistema no se utilizó alguna herramienta externa de gestión de proyectos como Jira o Azure DevOps, ya que el avance del desarrollo se registró mediante el historial de commits en GitHub, cuyos mensajes describen los cambios realizados en cada etapa.

Anexo E Protocolo y seguimiento de la revisión sistemática de literatura (si aplica)

Este anexo no aplica al presente trabajo. La revisión de literatura presentada en el Capítulo 2 se realizó mediante un proceso de búsqueda y selección tradicional, con los criterios de inclusión y exclusión descritos en dicho capítulo, pero sin seguir un protocolo formal de revisión sistemática.